Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial 2026/2027

Disusun Oleh: 13524016 - Wafiq Hibban Robbany

13524025 - Moh. Hafizh Irham Perdana

13524070 - A. Fawwaz Azam Wicaksono

DAFTAR ISI

DAFTAR ISI **2**
BAB I **5**
DESKRIPSI PERSOALAN **5**
A. Deskripsi Umum 5
B. Batasan Persoalan 5
C. Fungsi Objektif 6
BAB II **7**
IMPLEMENTASI ALGORITMA **7**
A. Representasi State 7
B. Pembentukan Initial State 12
C. Objective Function & Constraint 16
1. Objective Function 17
2. Constraints 18
D. Pembentukan Neighbor 19
E. Algoritma Hill Climbing 20
1. Steepest Ascent Hill Climbing 21
2. Sideways Move Hill Climbing 22
3. Random Restart Hill Climbing 25
4. Stochastic Hill Climbing 28
F. Algoritma Simulated Annealing 31
G. Genetic Algorithm 35
BAB III **40**
HASIL PENGUJIAN DAN PEMBAHASAN **40**
A. Algoritma Hill Climbing 40
1. Pengujian HC-SA-01 40
2. Pengujian HC-SA-02 42
3. Pengujian HC-SA-03 43
4. Pengujian HC-SM-01 45
5. Pengujian HC-SM-02 47
6. Pengujian HC-SM-03 48
7. Pengujian HC-RR-01 50
8. Pengujian HC-RR-02 53
9. Pengujian HC-RR-03 56
10. Pengujian HC-S-01 59
11. Pengujian HC-S-02 61
12. Pengujian HC-S-03 62
B. Simulated Annealing 64
1. Pengujian SA-01 64

2. Pengujian SA-02.....66
3. Pengujian SA-03.....67
C. Genetic Algorithm.....69
1. Pengujian GA-01.....70
2. Pengujian GA-02.....75
3. Pengujian GA-03.....79
4. Pengujian GA-04.....84
5. Pengujian GA-05.....88
6. Pengujian GA-06.....93
D. Pengujian Bonus Multi Truck.....97
1. Pengujian MT-01.....98
2. Pengujian MT-02.....99
3. Pengujian MT-03.....101
E. Pengujian Bonus Objective Function (ETA).....103
1. Pengujian ETA-01.....104
2. Pengujian ETA-02.....105
3. Pengujian ETA-03.....107
F. Analisis Algoritma.....109
BAB IV.....112
KESIMPULAN DAN SARAN.....112
A. Kesimpulan.....112
B. Saran.....112
REFERENSI.....113
LAMPIRAN.....114

DAFTAR TABEL

Tabel 2.1 Tabel Daftar Kelas Representasi State..... 7
Tabel 2.2 Tabel Daftar Fungsi Pembentukan Initial State..... 12
Tabel 2.3 Tabel Daftar Fungsi Objective Function & Constraint..... 16
Tabel 2.4 Tabel Daftar Fungsi Pembentukan Neighbor..... 20
Tabel 2.5 Tabel Daftar Fungsi Steepest Ascent Hill Climbing.....21
Tabel 2.6 Tabel Daftar Fungsi Sideways Move Hill Climbing..... 23
Tabel 2.7 Tabel Daftar Fungsi Random Restart Hill Climbing..... 25
Tabel 2.8 Tabel Daftar Fungsi Stochastic Hill Climbing.....28
Tabel 2.9 Tabel Daftar Fungsi Simulated Annealing..... 32
Tabel 2.10 Tabel Daftar Fungsi Genetic Algorithm..... 36
Tabel 3.1 Variasi Parameter Pengujian Steepest Ascent..... 40
Tabel 3.2 Variasi Parameter Pengujian Sideways Move.....45
Tabel 3.3 Variasi Parameter Pengujian Random Restart..... 50
Tabel 3.4 Variasi Parameter Pengujian Stochastic HC.....59
Tabel 3.5 Variasi Parameter Pengujian Simulated Annealing.....64
Tabel 3.6 Parameter Pengujian GA dengan Kontrol Populasi.....70
Tabel 3.7 Parameter Pengujian GA dengan Kontrol Iterasi..... 70
Tabel 3.8 Variasi Parameter Pengujian Multi Truck.....97
Tabel 3.9 Variasi Parameter Pengujian Simulated Annealing dengan ETA..... 104

BAB I

DESKRIPSI PERSOALAN

A. Deskripsi Umum

Diberikan sebuah truk dengan ruang muat berbentuk balok dan kapasitas berat maksimum tertentu, serta sekumpulan paket yang akan dimuat. Tidak semua paket dapat dimuat karena ruang dan kapasitas berat truk terbatas. Tugasnya adalah memilih paket mana yang dimuat, di posisi mana, dan dengan orientasi apa, sehingga total nilai paket yang terangkut sebesar mungkin. Pada eksperimen utama digunakan satu truk dengan kapasitas berat 100 dan 30 paket.

Setiap paket memiliki atribut berikut:

- identitas unik (*id*);
- dimensi asli (lebar, panjang, dan tinggi);
- nilai (*value*);
- berat;
- status pecah-belah (*fragile*);
- estimasi sisa hari kedatangan (ETA), yang digunakan pada fungsi objektif bonus.

Posisi paket dinyatakan sebagai koordinat bilangan bulat (x, y, z) pada sudut kiri-bawah-belakang paket. Paket boleh dirotasi sebesar 90° sehingga sisi-sisinya dapat dipetakan ke sumbu yang berbeda. Paket yang tidak dimuat tetap menjadi bagian dari *state* dengan posisi di luar truk.

B. Batasan Persoalan

Sebuah konfigurasi dianggap valid apabila memenuhi seluruh batasan berikut.

Batasan	Ketentuan
Kapasitas berat	Total berat paket yang dimuat tidak melebihi kapasitas maksimum truk.
Batas ruang	Setiap paket berada sepenuhnya di dalam atau sepenuhnya di luar truk; tidak boleh menembus dinding..

Batasan	Ketentuan
Tidak tumpang tindih	Volume dua paket yang dimuat tidak boleh beririsan.
Penopang (support)	Paket berada di lantai truk atau sebagian alasnya ditopang oleh permukaan atas paket lain, sehingga tidak melayang.
Paket pecah-belah	Paket fragile tidak boleh menjadi penopang bagi paket lain.

C. Fungsi Objektif

Fungsi objektif utama adalah memaksimalkan total nilai paket yang termuat sepenuhnya di dalam truk. Paket yang berada di luar truk tidak memberikan kontribusi. Sebagai bonus, tersedia fungsi objektif alternatif berbasis ETA yang memberi skor lebih besar kepada paket dengan ETA lebih kecil, yaitu 1/(ETA + 1), sehingga paket yang lebih mendesak diprioritaskan. Aturan batasan tetap sama untuk kedua fungsi objektif.

BAB II

IMPLEMENTASI ALGORITMA

A. Representasi State

State merepresentasikan konfigurasi penempatan seluruh paket, baik yang berada di dalam maupun di luar truk. Representasi ini diimplementasikan menggunakan `dataclass` pada modul `src/core/state.py`. Setiap paket menyimpan posisi dan orientasinya, sedangkan truk menyimpan dimensi ruang muat serta kapasitas berat maksimum.

Posisi paket dinyatakan sebagai koordinat bilangan bulat `(x, y, z)` pada sudut kiri-bawah-belakang paket. Orientasi menentukan pemetaan sisi asli paket, yaitu `w`, `l`, dan `h`, terhadap sumbu `x`, `y`, dan `z`. Dengan demikian, rotasi dapat direpresentasikan tanpa mengubah dimensi asli paket.

Untuk satu truk, konfigurasi disimpan menggunakan kelas `State`. Dukungan beberapa truk menggunakan kelas `FleetState`, yang menggabungkan sejumlah objek `State` menjadi satu konfigurasi armada.

Tabel 2.1 Tabel Daftar Kelas Representasi State

Nama Kelas	Atribut	Keterangan
`Package`	`id`, `dimensions`, `value`, `weight`, `isFragile`, `position`, `orientation`, `eta`	Merepresentasikan paket beserta identitas, dimensi asli, nilai, berat, status pecah-belah, posisi, orientasi, dan estimasi sisa hari kedatangan.
`Truck`	`dimensions`, `max_capacity`	Merepresentasikan dimensi ruang muat dan kapasitas berat maksimum sebuah truk.

Nama Kelas	Atribut	Keterangan
`State`	`truck, packages`	Merepresentasikan konfigurasi satu truk beserta seluruh paket yang ditugaskan kepadanya, termasuk paket yang berada di luar truk.
`FleetState`	`trucks`	Merepresentasikan konfigurasi armada sebagai daftar objek `State`. Setiap ID paket hanya boleh muncul sekali dalam seluruh armada.

Kelas `Package` dan `Truck` menyimpan data dasar yang digunakan dalam evaluasi maupun pembentukan neighbor. Potongan kode berikut menunjukkan atribut kedua kelas tersebut.

```
@dataclass
class Package:
    id: str
    dimensions: dict[str, int]
    value: float
    weight: float
    isFragile: bool = False
    position: tuple[int, int, int] = (0, 0, 0)
    orientation: dict[str, str] = field(
        default_factory=lambda: {
            "w": "x", "l": "y", "h": "z"
        }
    )
    eta: int = 0


@dataclass
class Truck:
    dimensions: dict[str, int]
    max_capacity: float
```

Atribut `dimensions` menggunakan dictionary dengan key `w`, `l`, dan `h`. Pada Package, nilai tersebut merupakan dimensi asli paket. Ukuran paket pada setiap sumbu setelah rotasi dihitung berdasarkan atribut `orientation`.

Metode `__post_init__()` memeriksa data saat objek dibentuk. Pemeriksaan mencakup dimensi bilangan bulat positif, posisi berupa tuple tiga bilangan bulat, orientasi yang diperbolehkan, ETA bilangan bulat nonnegatif, serta penolakan nilai negatif pada value, berat, dan kapasitas.

Kelas `State` menghubungkan data truk dengan daftar paket serta menyediakan operasi penyalinan dan pemeriksaan batas geometris. Potongan kode berikut menunjukkan struktur kelas dan metode untuk menentukan kedudukan paket terhadap ruang muat.

```
@dataclass
class State:
    truck: Truck
    packages: list[Package]

    def __post_init__(self):
        ids = [package.id for package in self.packages]
        if len(set(ids)) != len(ids):
            raise ValueError("ID paket harus unik.")

    def copy(self) -> "State":
        return State(
            replace(self.truck),
            [replace(package) for package in self.packages],
        )

    def bounds(self, package: Package):
        x, y, z = package.position
        width, length, height = package.oriented_dimensions()
        return (
            (x, y, z),
            (x + width, y + length, z + height),
        )

    def is_inside(self, package: Package) -> bool:
        start, end = self.bounds(package)
        limits = tuple(
            self.truck.dimensions[side]
            for side in ("w", "l", "h")
        )
        return all(
            0 <= start[i] and end[i] <= limits[i]
```

```
            for i in range(3)
        )

    def is_outside(self, package: Package) -> bool:
        start, end = self.bounds(package)
        limits = tuple(
            self.truck.dimensions[side]
            for side in ("w", "l", "h")
        )
        return any(
            end[i] <= 0 or start[i] >= limits[i]
            for i in range(3)
        )
```

Metode `bounds()` merepresentasikan volume paket menggunakan interval setengah terbuka pada setiap sumbu. Paket dengan batas (x_1, y_1, z_1) dan (x_2, y_2, z_2) menempati ruang $[x_1, x_2) \times [y_1, y_2) \times [z_1, z_2)$. Representasi ini memungkinkan dua paket bersentuhan pada permukaan tanpa dianggap overlap.

Paket berada sepenuhnya di dalam truk apabila seluruh koordinat awal tidak negatif dan seluruh batas akhirnya tidak melebihi dimensi truk. Paket berada sepenuhnya di luar apabila intervalnya terpisah dari ruang muat pada minimal satu sumbu. Jika kedua pemeriksaan menghasilkan `False`, paket berada sebagian di dalam dan sebagian di luar sehingga menembus dinding.

Metode `copy()` membentuk objek truk dan paket baru menggunakan `replace()`. Dictionary dimensi dan orientasi turut disalin melalui proses inisialisasi objek. Dengan demikian, perubahan pada kandidat neighbor tidak mengubah konfigurasi asal.

Untuk mendukung beberapa truk, `FleetState` menyimpan daftar state truk sebagai satu solusi gabungan. Setiap paket ditugaskan kepada tepat satu state truk, meskipun secara geometris paket tersebut masih dapat berada di luar ruang muat. Potongan kode berikut menunjukkan struktur armada, validasi keunikan paket, dan pencarian truk pemilik paket.

```
@dataclass
class FleetState:
    trucks: list[State]
```

```
    def __post_init__(self):
        if not self.trucks:
            raise ValueError(
                "Armada harus memiliki minimal satu truk."
            )

        ids = [
            p.id
            for truck_state in self.trucks
            for p in truck_state.packages
        ]
        if len(ids) != len(set(ids)):
            raise ValueError(
                "Setiap paket hanya boleh ditugaskan "
                "ke satu truk."
            )

    @property
    def packages(self) -> list[Package]:
        return [
            p
            for truck_state in self.trucks
            for p in truck_state.packages
        ]

    def copy(self) -> "FleetState":
        return FleetState([
            state.copy() for state in self.trucks
        ])

    def locate(self, package_id: str) -> tuple[State, int]:
        for truck_state in self.trucks:
            for index, package in enumerate(
                truck_state.packages
            ):
                if package.id == package_id:
                    return truck_state, index

        raise KeyError(package_id)
```

Properti `packages` menyediakan daftar gabungan paket sehingga algoritma dapat mengakses seluruh paket dalam armada. Metode `locate()` menentukan state truk

yang memiliki sebuah paket. Pemeriksaan `is_inside()`, `is_outside()`, dan `bounds()` kemudian diteruskan kepada state truk tersebut.

Koordinat setiap paket bersifat lokal terhadap truk pemiliknya. Oleh karena itu, paket pada truk berbeda dapat memiliki koordinat yang sama tanpa saling overlap.

B. Pembentukan Initial State

Initial state dibentuk secara acak untuk menghasilkan konfigurasi awal pencarian yang memenuhi constraint. Pengacakan mencakup urutan penempatan paket, orientasi, dan koordinat kandidat. Paket yang gagal ditempatkan setelah sejumlah percobaan tetap berada di luar truk.

Untuk beberapa truk, daftar paket terlebih dahulu diacak dan dibagikan secara bergiliran kepada setiap truk. Konfigurasi penempatan pada masing-masing truk kemudian dibentuk menggunakan prosedur inisialisasi satu truk.

Tabel 2.2 Tabel Daftar Fungsi Pembentukan Initial State

Nama Fungsi	Parameter	Keterangan
`outside_position()`	`state: State, package_index: int`	Menerima state dan indeks paket untuk menentukan posisi representatif di luar truk. Mengembalikan koordinat `(x, 0, 0)` dengan `x` dimulai dari batas lebar truk dan ditambah ukuran maksimum paket-paket sebelumnya.
`random_initial_state()`	`truck: Truck | list[Truck], packages: list[Package], seed: int | None = None, max_attempts: int = 50`	Menerima satu truk atau daftar truk, daftar paket, seed pengacakan, dan batas percobaan penempatan per paket. Menghasilkan konfigurasi awal berupa `State` untuk satu truk atau `FleetState` untuk armada.

Fungsi `outside_position()` menyediakan koordinat untuk paket yang belum atau tidak berhasil dimuat. Potongan kode berikut menunjukkan penentuan posisi tersebut berdasarkan batas truk dan ukuran paket sebelumnya.

```
def outside_position(
    state: State, package_index: int,
) -> tuple[int, int, int]:
    x = state.truck.dimensions["w"]

    for package in state.packages[:package_index]:
        x += max(package.dimensions.values())

    return x, 0, 0
```

Koordinat x paling kecil sama dengan lebar truk, sehingga seluruh bagian paket berada di luar ruang muat. Penambahan ukuran maksimum setiap paket sebelumnya memberikan jarak antarpaket di luar truk yang tetap memadai untuk berbagai orientasi.

Fungsi `random_initial_state()` mengatur pengacakan paket, pembentukan kandidat penempatan, dan pemeriksaan validitasnya. Potongan kode berikut menunjukkan proses inisialisasi untuk satu truk maupun armada.

```
def random_initial_state(
    truck: Truck | list[Truck],
    packages: list[Package],
    seed: int | None = None,
    max_attempts: int = 50,
) -> State | FleetState:
    if type(max_attempts) is not int or max_attempts <= 0:
        raise ValueError(
            "Max attempts harus berupa bilangan bulat positif."
        )

    if isinstance(truck, list):
        if not truck:
            raise ValueError(
                "Armada harus memiliki minimal satu truk."
            )

        rng = random.Random(seed)
        shuffled_packages = [
```

```
            replace(package) for package in packages
        ]
        rng.shuffle(shuffled_packages)

        assigned: list[list[Package]] = [[] for _ in truck]
        for index, package in enumerate(shuffled_packages):
            assigned[index % len(truck)].append(package)

        states = [
            random_initial_state(
                vehicle,
                package_subset,
                seed=rng.randrange(2**32),
                max_attempts=max_attempts,
            )
            for vehicle, package_subset in zip(truck, assigned)
        ]
        return FleetState(states)

    rng = random.Random(seed)
    state = State(truck, packages).copy()

    for i, package in enumerate(state.packages):
        package.position = outside_position(state, i)
        package.orientation = rng.choice(ORIENTATIONS).copy()

    order = list(range(len(state.packages)))
    rng.shuffle(order)

    for i in order:
        package = state.packages[i]

        for _ in range(max_attempts):
            package.orientation = rng.choice(
                ORIENTATIONS
            ).copy()
            width, length, height = (
                package.oriented_dimensions()
            )

            max_x = state.truck.dimensions["w"] - width
            max_y = state.truck.dimensions["l"] - length
            max_z = state.truck.dimensions["h"] - height

            if min(max_x, max_y, max_z) < 0:
```

```
                continue

            levels = {0}
            for other in state.packages:
                if (
                    other.id != package.id
                    and state.is_inside(other)
                    and not other.isFragile
                ):
                    top = state.bounds(other)[1][2]
                    if top <= max_z:
                        levels.add(top)

            package.position = (
                rng.randint(0, max_x),
                rng.randint(0, max_y),
                rng.choice(sorted(levels)),
            )

            if is_valid(state):
                break

            package.position = outside_position(state, i)

    return state
```

Proses diawali dengan pemeriksaan bahwa `max_attempts` merupakan bilangan bulat positif. Generator `random.Random(seed)` digunakan agar konfigurasi dapat dibentuk ulang dengan input dan seed yang sama.

Pada satu truk, fungsi menyalin state dan menempatkan seluruh paket di luar ruang muat terlebih dahulu. Setiap paket diberi orientasi acak, kemudian urutan percobaan penempatannya diacak agar paket pada awal daftar tidak selalu mendapatkan prioritas.

Dalam setiap percobaan, orientasi paket dipilih kembali. Batas koordinat dihitung dari selisih dimensi truk dengan ukuran paket setelah orientasi. Jika terdapat batas negatif, orientasi tersebut tidak dapat dimuat sehingga percobaan dilanjutkan tanpa melakukan penempatan.

Kandidat ketinggian pada `levels` mencakup lantai truk dan permukaan atas paket non-fragile yang telah berada di dalam truk. Pemilihan level ini mempersempit kandidat

pada ketinggian yang berpotensi memiliki support. Namun, kesesuaian posisi horizontal dan seluruh constraint tetap diperiksa melalui `is_valid()`.

Jika kandidat valid, posisi paket dipertahankan dan proses beralih ke paket berikutnya. Jika tidak valid, paket dikembalikan ke luar truk sebelum percobaan selanjutnya. Setelah batas percobaan tercapai, paket yang belum berhasil dimuat tetap disimpan sebagai bagian state dengan posisi di luar truk. Batas percobaan mencegah inisialisasi terus mencari penempatan bagi paket yang sulit atau tidak mungkin dimuat.

Pada armada, daftar paket disalin dan diacak, kemudian dibagikan menggunakan pola *round-robin*. Pembagian ini menyeimbangkan jumlah paket yang ditugaskan, tanpa menjamin keseimbangan berat, volume, atau value. Setiap truk selanjutnya diinisialisasi secara terpisah dengan seed turunan, lalu seluruh hasil digabungkan menjadi `FleetState`.

C. Objective Function & Constraint

Objective function digunakan untuk menilai kualitas konfigurasi pemuatan paket, sedangkan constraint menentukan validitas konfigurasi tersebut. Keduanya diimplementasikan pada modul `src/core/evaluation.py`.

Implementasi menyediakan dua pilihan objective, yaitu total value sebagai objective utama dan prioritas ETA sebagai objective bonus. Kedua objective hanya memperhitungkan paket yang seluruh bagiannya berada di dalam truk. Constraint yang berlaku tetap sama untuk kedua pilihan tersebut.

Tabel 2.3 Tabel Daftar Fungsi Objective Function & Constraint

Nama Fungsi	Parameter	Keterangan
`objective_value()`	`state: State | FleetState, objective: str = "value"`	Menerima konfigurasi satu truk atau armada dan pilihan objective. Mengembalikan total value paket termuat untuk "value" atau jumlah skor prioritas ETA untuk "eta". Pilihan yang tidak dikenal menghasilkan ValueError.

Nama Fungsi	Parameter	Keterangan
`is_valid()`	`state: State | FleetState`	Menerima konfigurasi satu truk atau armada dan memeriksa kapasitas berat, batas ruang muat, overlap, support, serta larangan paket fragile menjadi support. Mengembalikan `True` jika seluruh pemeriksaan terpenuhi dan `False` jika terdapat pelanggaran.

1. Objective Function

Objective utama memaksimalkan total value paket yang berhasil dimuat. Misalkan $P_m(S)$ merupakan himpunan paket yang seluruh bagiannya berada di dalam truk pada state S, maka:

$$f_{value}(S) = \sum_{i \in P_m(S)} Value_i$$

Paket yang berada di luar truk tidak memberikan kontribusi terhadap objective. Pada armada, penjumlahan mencakup paket termuat pada seluruh truk.

Sebagai bonus, implementasi menyediakan objective prioritas ETA. Objective ini memberikan skor lebih besar kepada paket dengan estimasi sisa hari kedatangan yang lebih kecil:

$$f_{ETA}(S) = \sum_{i \in P_m(S)} \frac{1}{ETA_i + 1}$$

Penambahan satu pada penyebut menghindari pembagian dengan nol ketika ETA bernilai nol. Sebagai contoh, paket dengan ETA 0 memperoleh skor 1, sedangkan paket dengan ETA 3 memperoleh skor 0,25. Objective ini memaksimalkan jumlah skor urgensi paket termuat; nilainya tidak digabungkan dengan total value.

Fungsi `objective_value()` memilih perhitungan berdasarkan parameter `objective` dan menyaring paket menggunakan `is_inside()`. Potongan kode berikut menunjukkan implementasi kedua objective tersebut.

```
def objective_value(
    state: State | FleetState,
    objective: str = "value",
) -> float:
    if objective == "value":
        return sum(
            package.value
            for package in state.packages
            if state.is_inside(package)
        )

    if objective == "eta":
        return sum(
            1 / (package.eta + 1)
            for package in state.packages
            if state.is_inside(package)
        )

    raise ValueError(
        f"Objective tidak dikenal: {objective}"
    )
```

Pemeriksaan validitas dipisahkan dari perhitungan objective. Oleh karena itu, `objective_value()` hanya menghitung skor berdasarkan paket yang berada di dalam truk, sedangkan kelayakan konfigurasi diperiksa melalui `is_valid()`.

2. Constraints

Constraint	Ketentuan
Kapasitas berat	Total berat paket termuat tidak melebihi kapasitas maksimum truk.
Batas ruang muat	Setiap paket harus sepenuhnya berada di dalam atau sepenuhnya berada di luar truk.
Tidak overlap	Volume dua paket termuat tidak boleh saling beririsan.

Support	Paket berada di lantai atau memiliki sebagian alas yang ditopang permukaan atas paket lain.
Fragile	Paket pecah-belah tidak boleh menjadi support bagi paket lain.

Untuk setiap truk, batas berat dinyatakan sebagai:

$$\sum_{i \in P_m(S)} Weight_i \leq MaxCapacity$$

Pemeriksaan overlap menggunakan batas awal dan akhir paket pada ketiga sumbu. Dua paket overlap apabila interval volumenya beririsan pada sumbu x, y, dan z secara bersamaan. Ketidaksamaan yang digunakan bersifat ketat, sehingga paket yang hanya bersentuhan pada permukaan tidak dianggap overlap.

Pemeriksaan support mengikuti definisi bahwa paket tidak melayang apabila terdapat minimal sebagian alas yang ditopang. Seluruh alas tidak harus tertutup oleh paket penopang. Namun, setiap paket yang menjadi penopang harus bukan paket fragile.

D. Pembentukan Neighbor

Neighbor merupakan konfigurasi kandidat yang dihasilkan melalui perubahan terhadap state saat ini. Pembentukan neighbor diimplementasikan pada modul `src/core/operators.py` melalui tiga operasi utama, yaitu pemindahan paket, pertukaran posisi dua paket, dan rotasi paket sebesar 90°.

Setiap operator membentuk salinan state sebelum melakukan perubahan sehingga state asal tetap terjaga. Kandidat kemudian diperiksa menggunakan `is_valid()`. Hanya konfigurasi yang memenuhi seluruh constraint yang dikembalikan sebagai neighbor.

Tabel 2.4 Tabel Daftar Fungsi Pembentukan Neighbor

Nama Fungsi	Parameter	Keterangan
`move_package()`	`state: State, package_index : int, position: tuple[int, int, int]`	Menerima state, indeks paket, dan koordinat tujuan untuk membentuk kandidat pemindahan paket. Mengembalikan neighbor valid atau `None` jika posisi tidak berubah atau kandidat melanggar constraint.
`swap_packages()`	`state: State, first: int, second: int`	Menerima state dan indeks dua paket untuk menukar posisinya. Mengembalikan neighbor valid atau `None` jika pertukaran tidak mengubah konfigurasi atau melanggar constraint.
`rotate_package()`	`state: State, package_index : int, axis: str`	package_index: int, axis: str Menerima state, indeks paket, dan sumbu rotasi "x", "y", atau "z". Mengembalikan neighbor valid setelah perubahan orientasi, atau None jika hasilnya melanggar constraint.
`generate_neighbors()`	`state: State`	Menerima konfigurasi satu truk atau armada dan menghasilkan neighbor valid satu per satu menggunakan generator. Pada armada, fungsi juga membentuk kandidat perpindahan paket antatruk.

E. Algoritma Hill Climbing

Hill Climbing melakukan pencarian solusi melalui perpindahan dari state saat ini menuju neighbor berdasarkan nilai objective function. Pada permasalahan pemuatan paket, setiap state merepresentasikan konfigurasi penempatan dan orientasi paket. Neighbor dibentuk melalui operasi pemindahan, pertukaran posisi, dan rotasi paket.

Implementasi terdiri atas empat varian, yaitu **Steepest Ascent**, **Stochastic**, **Sideways Move**, dan **Random Restart**. Seluruh varian menggunakan fungsi `objective_value()` untuk mengevaluasi state. Pada objective utama, nilai state merupakan jumlah value paket yang berada sepenuhnya di dalam truk.

Modul Hill Climbing menggunakan fungsi tanpa mendefinisikan kelas baru. Representasi solusi menggunakan kelas `State` dan, untuk pencarian beberapa truk, `FleetState` yang dijelaskan pada subbab representasi state.

1. Steepest Ascent Hill Climbing

Steepest Ascent mengevaluasi seluruh neighbor valid dari state saat ini dan memilih neighbor dengan objective tertinggi yang memberikan peningkatan. Pencarian berhenti apabila tidak terdapat neighbor dengan objective lebih tinggi.

Apabila beberapa neighbor memiliki nilai terbaik yang sama, implementasi mempertahankan neighbor pertama yang ditemukan. Hal ini terjadi karena kandidat terbaik hanya diperbarui ketika ditemukan nilai yang lebih besar.

Tabel 2.5 Tabel Daftar Fungsi Steepest Ascent Hill Climbing

Nama Fungsi	Parameter	Keterangan
`steepest_ascent()`	`initial_state: State, objective: str = "value"`	Menerima `initial_state` sebagai konfigurasi awal pencarian dan `objective` sebagai pilihan fungsi penilaian, dengan default `"value"` untuk memaksimalkan total value paket di dalam truk. Pada setiap iterasi, fungsi memilih neighbor dengan objective tertinggi yang memberikan peningkatan. Pencarian berhenti ketika tidak ada peningkatan, lalu mengembalikan tuple berisi state akhir dan dictionary riwayat pencarian.

Fungsi utama Simulated Annealing mengatur perubahan suhu, pemilihan neighbor, peluang penerimaan, dan pencatatan riwayat pencarian. Potongan kode berikut menunjukkan proses tersebut dalam implementasi `simulated_annealing()`.

```
def steepest_ascent(initial_state: State, objective: str = "value") ->
tuple[State, dict]:
    start = perf_counter()
    current = _start(initial_state)
    values = [objective_value(current, objective)]
    while True:
        best, best_value = None, values[-1]
```

```
        for neighbor in generate_neighbors(current):
            value = objective_value(neighbor, objective)
            if value > best_value:
                best, best_value = neighbor, value
        if best is None:
            break
        current = best
        values.append(best_value)
    return current, _history(initial_state, current, values, start,
"no_improvement", objective)
```

Proses diawali dengan pencatatan waktu menggunakan `perf_counter()`. Fungsi `_start()` memvalidasi dan menyalin state awal agar pencarian tidak mengubah input. Objective state awal kemudian disimpan sebagai elemen pertama pada `values`, yang merepresentasikan iterasi ke-0.

Pada setiap iterasi, `best_value` diinisialisasi dengan objective state saat ini. Seluruh neighbor valid diperiksa, dan kandidat terbaik diperbarui hanya apabila ditemukan objective yang lebih tinggi. Jika beberapa neighbor memiliki nilai terbaik yang sama, neighbor pertama yang ditemukan dipertahankan.

Apabila kandidat terbaik ditemukan, `current` diperbarui dan objective-nya ditambahkan ke riwayat. Sebaliknya, jika `best` tetap bernilai `None`, pencarian dihentikan dengan alasan `no_improvement`. Jumlah iterasi yang dicatat merupakan jumlah perpindahan yang diterima, tanpa menghitung pemeriksaan terakhir yang tidak menghasilkan perpindahan.

Fungsi mengembalikan state akhir beserta riwayat yang memuat state awal dan akhir, objective akhir, riwayat objective, jumlah iterasi, durasi, dan alasan berhenti. Karena hanya menerima peningkatan, algoritma dapat berhenti pada optimum lokal atau plateau dan tidak menjamin tercapainya optimum global.

2. Sideways Move Hill Climbing

Hill Climbing with Sideways Move memperbolehkan perpindahan menuju neighbor yang memiliki objective sama dengan state saat ini. Mekanisme ini memungkinkan pencarian menelusuri konfigurasi lain pada plateau yang mungkin membuka jalan menuju peningkatan objective.

Pada setiap iterasi, algoritma mengevaluasi seluruh neighbor valid dan memilih kandidat dengan objective tertinggi yang tidak lebih rendah daripada state saat ini. Jika terdapat beberapa kandidat dengan nilai terbaik yang sama, salah satunya dipilih secara acak.

Tabel 2.6 Tabel Daftar Fungsi Sideways Move Hill Climbing

Nama Fungsi	Parameter	Keterangan
`sideways()`	`initial_state: State, max_sideways: int = 100, seed: int | None = None, objective: str = "value"`	Menerima `initial_state` sebagai konfigurasi awal, `max_sideways` sebagai batas total perpindahan dengan objective sama, `seed` untuk mengatur pemilihan acak, dan `objective` sebagai pilihan fungsi penilaian. Fungsi menerima perpindahan yang meningkatkan atau mempertahankan objective, lalu mengembalikan tuple berisi state akhir dan dictionary riwayat pencarian.

Fungsi utama Sideways Move mengatur pemilihan neighbor terbaik, penerimaan perpindahan dengan objective sama, dan pembatasan jumlah sideways move. Potongan kode berikut menunjukkan proses tersebut beserta pencatatan riwayat dalam implementasi `sideways()`.

```
def sideways(
    initial_state: State, max_sideways: int = 100, seed: int | None = None,
objective: str = "value",
) -> tuple[State, dict]:
    start = perf_counter()
    _check_limit("max_sideways", max_sideways)
    current = _start(initial_state)
    rng = random.Random(seed)
    values = [objective_value(current, objective)]
    sideways_moves = 0
    while True:
        best, best_value, ties = None, values[-1], 0
        for neighbor in generate_neighbors(current):
            value = objective_value(neighbor, objective)
            if value > best_value:
                best, best_value, ties = neighbor, value, 1
            elif value == best_value:
```

```
                ties += 1
                if rng.randrange(ties) == 0:
                    best = neighbor
        if best is None:
            reason = "no_non_decreasing_neighbor"
            break
        equal = best_value == values[-1]
        if equal and sideways_moves == max_sideways:
            reason = "max_sideways"
            break
        current = best
        values.append(best_value)
        if equal:
            sideways_moves += 1
            if sideways_moves == max_sideways:
                reason = "max_sideways"
                break
    history = _history(initial_state, current, values, start, reason,
objective)
    history.update(sideways_moves=sideways_moves,
sideways_limit_mode="total", seed=seed)
    return current, history
```

Proses diawali dengan validasi `max_sideways` sebagai bilangan bulat nonnegatif. State awal divalidasi dan disalin menggunakan `_start()`, kemudian objective awal dicatat sebagai iterasi ke-0. Generator `random.Random(seed)` digunakan agar pemilihan kandidat acak dapat diulang dengan seed yang sama.

Dalam pemeriksaan neighbor, `best_value` menyimpan objective terbaik sementara. Ketika ditemukan nilai lebih tinggi, kandidat terbaik diperbarui dan penghitung `ties` diatur menjadi satu. Kandidat lain dengan nilai sama dipilih menggunakan *reservoir sampling*, sehingga setiap kandidat dengan nilai terbaik memiliki peluang yang sama untuk terpilih.

Setelah kandidat dipilih, variabel `equal` menunjukkan apakah perpindahan mempertahankan objective. Jika perpindahan diterima, state diperbarui dan objective dicatat. Penghitung `sideways_moves` hanya bertambah untuk perpindahan dengan objective sama. Batas ini berlaku terhadap total sideways move selama pencarian dan tidak direset setelah terjadi peningkatan.

Pencarian berhenti apabila tidak terdapat neighbor yang mempertahankan atau meningkatkan objective, atau batas sideways move tercapai. Jika `max_sideways` bernilai nol, algoritma tetap menerima peningkatan, tetapi menolak perpindahan dengan objective sama.

Fungsi mengembalikan state akhir beserta riwayat pencarian yang mencakup jumlah iterasi, durasi, alasan berhenti, jumlah sideways move, dan seed. Jumlah iterasi menunjukkan seluruh perpindahan yang diterima, baik berupa peningkatan maupun sideways move.

3. Random Restart Hill Climbing

Random Restart Hill Climbing menjalankan Steepest Ascent dari beberapa state awal untuk memperluas eksplorasi pencarian. Run pertama menggunakan state awal yang diberikan, sedangkan setiap restart membentuk state awal baru secara acak. Hasil terbaik dari seluruh run dipilih sebagai solusi akhir.

Tabel 2.7 Tabel Daftar Fungsi Random Restart Hill Climbing

Nama Fungsi	Parameter	Keterangan
`random_restart()`	`initial_state: State, max_restarts: int = 10, seed: int | None = None, max_attempts: int = 50, objective: str = "value"`	Menerima `initial_state` sebagai konfigurasi run pertama, `max_restarts` sebagai batas restart setelah run pertama, `seed` untuk mengatur pembentukan state acak, `max_attempts` sebagai batas percobaan penempatan per paket saat inisialisasi restart, dan `objective` sebagai pilihan fungsi penilaian. Mengembalikan tuple berisi state terbaik dari seluruh run dan dictionary riwayat pencarian.

Fungsi utama Random Restart mengatur pembentukan state awal baru, pelaksanaan Steepest Ascent pada setiap run, dan penyimpanan solusi terbaik dari seluruh run. Potongan kode berikut menunjukkan proses tersebut beserta pencatatan statistik setiap run dalam implementasi `random_restart()`.

```
def random_restart(
```

```
    initial_state: State,
    max_restarts: int = 10,
    seed: int | None = None,
    max_attempts: int = 50,
    objective: str = "value",
) -> tuple[State, dict]:
    start = perf_counter()
    _check_limit("max_restarts", max_restarts)
    if type(max_attempts) is not int or max_attempts <= 0:
        raise ValueError("max_attempts harus berupa bilangan bulat
positif.")
    current = _start(initial_state)
    rng = random.Random(seed)
    seed_base = rng.getrandbits(128)
    best, best_value, best_run = current.copy(), objective_value(current,
objective), 0
    runs, values, iterations, run_indices = [], [], [], []
    accepted = 0
    for run_index in range(max_restarts + 1):
        restart_seed = None if run_index == 0 else seed_base + run_index
        if run_index:
            current = random_initial_state(
                [s.truck for s in initial_state.trucks] if
isinstance(initial_state, FleetState) else initial_state.truck,
                initial_state.packages,
                seed=restart_seed, max_attempts=max_attempts,
            )
        final, history = steepest_ascent(current, objective)
        history.update(run_index=run_index, seed=restart_seed)
        runs.append(history)
        if run_index == 0 or history["final_objective"] > best_value:
            best, best_value, best_run = final.copy(),
history["final_objective"], run_index
        values.extend(history["objective"])
        iterations.extend(accepted + i for i in history["iteration"])
        run_indices.extend([run_index] * len(history["objective"]))
        accepted += history["iterations"]
    history = _history(initial_state, best, values, start, "max_restarts",
objective)
    history.update(
        final_objective=best_value, iterations=accepted,
iteration=iterations,
        run_index=run_indices, runs=runs, restarts=max_restarts,
        iterations_per_run=[run["iterations"] for run in runs],
        best_run_index=best_run, seed=seed,
```

```
    )
    return best, history
```

Proses diawali dengan validasi parameter. `max_restarts` harus berupa bilangan bulat nonnegatif, sedangkan `max_attempts` harus berupa bilangan bulat positif. State awal kemudian divalidasi dan disalin melalui `_start()`. Generator acak membentuk `seed_base` yang digunakan untuk menghasilkan seed berbeda pada setiap restart.

Perulangan dijalankan sebanyak `max_restarts + 1`, karena run pertama tidak dihitung sebagai restart. Run ke-0 menggunakan state awal masukan. Pada run berikutnya, `random_initial_state()` membentuk konfigurasi baru menggunakan data truk dan paket yang sama. Untuk representasi `FleetState`, inisialisasi menerima daftar truk dalam armada.

Setiap run menjalankan `steepest_ascent()` hingga tidak ditemukan peningkatan. Objective akhir run dibandingkan dengan `best_value`. Jika lebih tinggi, state terbaik dan indeks run diperbarui. Apabila nilainya sama, hasil terbaik yang lebih dahulu ditemukan dipertahankan.

Riwayat setiap run disimpan pada `runs`, sedangkan `values` menggabungkan riwayat objective seluruh run. Nilai objective dapat turun ketika memasuki restart karena konfigurasi awal dibentuk ulang. Variabel `accepted` menjumlahkan seluruh perpindahan yang diterima oleh Steepest Ascent, sehingga proses restart itu sendiri tidak dihitung sebagai iterasi perpindahan.

Pencarian berhenti setelah run awal dan seluruh restart selesai. Hasil yang dikembalikan merupakan state terbaik dari seluruh run, yang dapat berbeda dari state akhir run terakhir. Riwayat hasil juga mencatat jumlah restart, iterasi setiap run melalui `iterations_per_run`, indeks run terbaik melalui `best_run_index`, serta durasi keseluruhan pencarian.

4. Stochastic Hill Climbing

Stochastic Hill Climbing memilih kandidat perpindahan secara acak dan menerima kandidat tersebut hanya apabila objective meningkat. Pemilihan acak

memungkinkan pencarian dilakukan tanpa mengevaluasi seluruh neighbor pada setiap iterasi untuk kasus satu truk. Jumlah percobaan dibatasi oleh parameter `nmax`.

Tabel 2.8 Tabel Daftar Fungsi Stochastic Hill Climbing

Nama Fungsi	Parameter	Keterangan
`stochastic()`	`initial_state : State, seed: int | None = None, nmax: int = 100, objective: str = "value"`	Menerima `initial_state` sebagai konfigurasi awal pencarian, `seed` untuk mengatur pemilihan acak, `nmax` sebagai batas percobaan proposal, dan `objective` sebagai pilihan fungsi penilaian. Menerima kandidat yang meningkatkan objective dan mengembalikan tuple berisi state akhir serta dictionary riwayat pencarian.
`_random_successor()`	`state: State, rng: random.Random`	Menerima state saat ini dan generator acak untuk memilih kandidat perpindahan. Pada satu truk, fungsi memilih paket dan operasi secara acak. Pada armada, fungsi memilih dari daftar neighbor valid. Mengembalikan kandidat state atau `None` jika proposal tidak menghasilkan perpindahan valid.

Fungsi pendukung pembentukan kandidat acak mengatur pemilihan paket, jenis operasi, dan kandidat perpindahan yang akan dievaluasi oleh Stochastic Hill Climbing. Potongan kode berikut menunjukkan proses tersebut dalam implementasi `_random_successor()`.

```
def _random_successor(state: State, rng: random.Random) -> State | None:
    if isinstance(state, FleetState):
        neighbors = list(generate_neighbors(state))
        return rng.choice(neighbors) if neighbors else None
    index = rng.randrange(len(state.packages))
    operation = rng.choice(("move", "rotate", "swap"))
    if operation == "rotate":
        return rotate_package(state, index, rng.choice(("x", "y", "z")))
    if operation == "swap":
        if len(state.packages) < 2:
            return None
        other = rng.randrange(len(state.packages) - 1)
```

```
        if other >= index:
            other += 1
        return swap_packages(state, index, other)

    sizes = state.packages[index].oriented_dimensions()
    counts = [max(0, state.truck.dimensions[side] - size + 1)
              for side, size in zip(("w", "l", "h"), sizes)]
    inside_count = counts[0] * counts[1] * counts[2]
    target = rng.randrange(inside_count + 1)
    if target == inside_count:
        position = outside_position(state, index)
    else:
        x, remainder = divmod(target, counts[1] * counts[2])
        y, z = divmod(remainder, counts[2])
        position = (x, y, z)
    return move_package(state, index, position)
```

Pada kasus satu truk, fungsi memilih satu paket, kemudian memilih operasi `move`, `rotate`, atau `swap` dengan peluang yang sama. Operasi rotasi memilih salah satu sumbu secara acak, sedangkan pertukaran memilih paket kedua yang berbeda dari paket pertama.

Untuk operasi pemindahan, `counts` menyatakan jumlah kemungkinan koordinat pada setiap sumbu berdasarkan ukuran paket setelah orientasi. Kandidat tujuan mencakup seluruh koordinat yang secara dimensi memungkinkan paket berada di dalam truk, serta satu posisi representatif di luar truk. Validitas penempatan selanjutnya diperiksa oleh operator terkait.

Pemilihan tersebut tidak menghasilkan distribusi seragam atas seluruh neighbor valid karena pengacakan dilakukan berdasarkan paket dan jenis operasi. Pada kasus `FleetState`, implementasi menggunakan pendekatan berbeda, yaitu membentuk seluruh neighbor valid terlebih dahulu dan memilih salah satunya secara acak.

Fungsi utama Stochastic Hill Climbing mengatur pengambilan kandidat acak, penerimaan perpindahan yang meningkatkan objective, dan pencatatan setiap percobaan hingga batas pencarian tercapai. Potongan kode berikut menunjukkan proses tersebut dalam implementasi `stochastic()`.

```
def stochastic(
    initial_state: State, seed: int | None = None, nmax: int = 100,
objective: str = "value",
) -> tuple[State, dict]:
    start = perf_counter()
    _check_limit("nmax", nmax)
    current = _start(initial_state)
    rng = random.Random(seed)
    values = [objective_value(current, objective)]
    accepted_attempts = []
    invalid_proposals = 0
    reason = "nmax" if current.packages or nmax == 0 else "no_neighbors"
    for attempt in range(1, nmax + 1):
        if not current.packages:
            break
        selected = _random_successor(current, rng)
        current_value = values[-1]
        if selected is None:
            invalid_proposals += 1
        else:
            value = objective_value(selected, objective)
            if value > current_value:
                current = selected
                current_value = value
                accepted_attempts.append(attempt)
        values.append(current_value)
    history = _history(initial_state, current, values, start, reason,
objective)
    history.update(
        seed=seed, nmax=nmax, selection="random_operation_proposal",
        attempts=len(values) - 1, attempt_objective=values.copy(),
        accepted_moves=len(accepted_attempts),
accepted_attempts=accepted_attempts,
        invalid_proposals=invalid_proposals,
    )
    return current, history
```

Proses diawali dengan validasi `nmax` sebagai bilangan bulat nonnegatif. State awal divalidasi dan disalin melalui `_start()`, kemudian objective awal disimpan sebagai iterasi ke-0. Generator `random.Random(seed)` mengatur pengambilan keputusan acak agar pencarian dapat diulang dengan input dan seed yang sama.

Pada setiap percobaan, `_random_successor()` menghasilkan kandidat. Kandidat diterima hanya jika objective-nya lebih tinggi daripada state saat ini. Proposal dengan objective sama atau lebih rendah ditolak. Apabila fungsi mengembalikan None, penghitung `invalid_proposals` bertambah; kondisi ini juga mencakup proposal yang tidak mengubah posisi.

Setiap percobaan tetap dicatat dalam riwayat, termasuk proposal yang ditolak atau tidak valid. Oleh karena itu, jumlah iterasi menunjukkan jumlah percobaan, sedangkan `accepted_moves` menunjukkan jumlah perpindahan yang diterima. Nomor percobaan yang menghasilkan peningkatan disimpan pada `accepted_attempts`.

Pencarian berhenti setelah batas nmax tercapai atau ketika state tidak memiliki paket. Kegagalan satu proposal tidak langsung menghentikan pencarian karena proposal berikutnya masih mungkin menghasilkan peningkatan. State akhir beserta riwayat objective, durasi, jumlah percobaan, dan statistik penerimaan kemudian dikembalikan sebagai hasil pencarian.

F. Algoritma Simulated Annealing

Simulated Annealing (SA) digunakan untuk mencari susunan paket yang memaksimalkan nilai objektif. Pada setiap iterasi, algoritma memilih satu state tetangga yang valid secara acak. Jika objective tetangga lebih tinggi daripada objective state saat ini, state tersebut diterima. Jika lebih rendah, state itu masih mungkin diterima berdasarkan suhu saat itu. Mekanisme tersebut memberi kesempatan bagi pencarian untuk keluar dari local optimum.

Suhu diturunkan secara geometris pada setiap iterasi, menggunakan rumus $T_i = T_0 \cdot \alpha^{\{i-1\}}$, dengan T_0 sebagai suhu awal dan α sebagai faktor pendinginan. Untuk objective maksimisasi, perubahan nilai dihitung sebagai $\Delta E = f(S_{next}) - f(S_{current})$. Peluang penerimaan langkah dihitung dengan $P(accept) = min(1, e^{\Delta E / T_i})$. Dengan demikian, langkah yang memperbaiki objective diterima, sedangkan langkah yang menurunkannya memiliki peluang diterima yang mengecil saat suhu turun.

Dalam implementasi ini, state terbaik yang pernah ditemukan disimpan terpisah dari state yang sedang dijelajahi. Karena itu, algoritma dapat mengembalikan state terbaik meskipun langkah terakhir yang diterima memiliki objective lebih rendah. Untuk fungsi objektif utama, nilai state adalah jumlah value paket yang seluruhnya berada di dalam truk.

Tabel 2.9 Tabel Daftar Fungsi Simulated Annealing

Nama Fungsi	Parameter	Keterangan
`simulated_annealing()`	`initial_state: State, initial_temperature: float, cooling_rate: float, max_iterations: int, objective: str = "value"`	Menjalankan Simulated Annealing dari state awal. Pada setiap iterasi, fungsi menghitung suhu, memilih satu neighbor valid secara acak, lalu menerima neighbor berdasarkan perubahan objective dan peluang $min(1, e^{(\Delta E/T)})$. Fungsi menyimpan state terbaik yang ditemukan serta riwayat objective, suhu, nilai $e^{(\Delta E/T)}$, dan frekuensi stuck. Mengembalikan tuple (best_state, history).
`generate_neighbors()`	`state: State`	Menghasilkan neighbor valid dari state saat ini melalui pemindahan, rotasi, atau pertukaran paket. SA memilih satu neighbor secara acak dari hasil fungsi ini.
`objective_value()`	`state: State | FleetState, objective: str = "value"`	Menghitung nilai objective state. Untuk objective default `"value"`, nilai yang dihitung adalah total value paket yang seluruhnya berada di dalam truk. Fungsi juga mendukung objective `"eta"`.

Fungsi utama Simulated Annealing mengatur perubahan suhu, pemilihan neighbor, peluang penerimaan, dan pencatatan riwayat pencarian. Potongan kode berikut menunjukkan proses tersebut dalam implementasi `simulated_annealing()`.

```
def simulated_annealing(
    initial_state: State,
    initial_temperature: float,
    cooling_rate: float,
    max_iterations: int,
    objective: str = "value",
```

```
) -> tuple[State, dict[str, Any]]:
    if initial_temperature < 0:
        raise ValueError("Initial temperature tidak boleh negatif.")
    if not 0 < cooling_rate <= 1:
        raise ValueError("Cooling rate harus berada pada rentang (0, 1].")
    if type(max_iterations) is not int or max_iterations < 0:
        raise ValueError("Max iterations harus berupa bilangan bulat
non-negatif.")

    current = initial_state.copy()
    best = current.copy()
    history = {
        "objective": [],
        "objective_name": objective,
        "exp_delta_e_over_t": [],
        "temperature": [],
        "stuck": [],
        "stuck_frequency": 0,
    }

    for iteration in range(1, max_iterations + 1):
        temperature = initial_temperature * cooling_rate ** (iteration - 1)
        if temperature <= 0:
            break

        neighbors = list(generate_neighbors(current))
        if not neighbors:
            break

        current_value = objective_value(current, objective)
        is_stuck = not any(
            objective_value(neighbor, objective) > current_value
            for neighbor in neighbors
        )
        next_state = random.choice(neighbors)
        delta_e = objective_value(next_state, objective) - current_value
        try:
            exp_delta_e_over_t = math.exp(delta_e / temperature)
        except OverflowError:
            exp_delta_e_over_t = math.inf

        if min(1.0, exp_delta_e_over_t) > random.random():
            current = next_state

        if objective_value(current, objective) > objective_value(best,
```

```
objective):
            best = current.copy()

        history["objective"].append(objective_value(current, objective))
        history["exp_delta_e_over_t"].append(exp_delta_e_over_t)
        history["temperature"].append(temperature)
        history["stuck"].append(is_stuck)
        history["stuck_frequency"] += is_stuck

    return best, history
```

Sebelum pencarian dimulai, fungsi memeriksa nilai parameter: temperatur awal tidak boleh negatif, cooling rate harus berada pada rentang (0, 1], dan jumlah iterasi harus berupa bilangan bulat nonnegatif. State awal kemudian disalin sebagai `current` dan `best`. `current` menunjukkan state yang sedang dijelajahi, sedangkan `best` menyimpan state dengan objective tertinggi yang pernah ditemukan.

Pada setiap iterasi, temperatur dihitung menggunakan i`nitial_temperature * cooling_rate ** (iteration - 1)`. Fungsi lalu membentuk daftar neighbor valid dari state saat ini. Jika temperatur sudah mencapai nol atau tidak ada neighbor yang valid, pencarian dihentikan. Jika pencarian berlanjut, satu neighbor dipilih secara acak dan perubahan objective-nya dihitung sebagai selisih nilai neighbor dengan nilai state saat ini.

Nilai `exp_delta_e_over_t` digunakan untuk menentukan peluang penerimaan neighbor. Jika objective meningkat atau tetap, nilai tersebut menghasilkan peluang penerimaan sebesar satu setelah dibatasi dengan `min(1.0, exp_delta_e_over_t)`. Jika objective menurun, neighbor masih dapat diterima, tetapi peluangnya dipengaruhi oleh besar penurunan dan temperatur saat itu. Penanganan `OverflowError` memastikan perhitungan eksponensial yang sangat besar tetap dapat dilanjutkan sebagai peluang yang dibatasi satu.

Variabel `is_stuck` bernilai benar ketika tidak ada neighbor yang memiliki objective lebih tinggi daripada state saat ini. Nilai ini dicatat pada setiap iterasi, lalu dijumlahkan ke dalam `stuck_frequency`. Dengan demikian, `stuck_frequency`

menunjukkan jumlah iterasi ketika pencarian tidak menemukan langkah yang langsung meningkatkan objective, bukan jumlah penghentian algoritma.

Setelah keputusan penerimaan dibuat, state `current` diperbarui jika neighbor diterima. Jika objective `current` melampaui objective `best`, state tersebut disalin ke `best`. Riwayat menyimpan objective state saat ini setelah keputusan penerimaan, nilai `e^(ΔE/T)` untuk neighbor yang diusulkan, temperatur, serta status stuck pada setiap iterasi. Fungsi mengembalikan `best` bersama dictionary riwayat pencarian.

G. Genetic Algorithm

Genetic Algorithm (GA) adalah algoritma metaheuristik berbasis populasi yang meniru seleksi alam untuk mencari susunan pemuatan paket bernilai tinggi. Pada setiap generasi, individu diurutkan berdasarkan fitness. Elitism menyalin beberapa individu terbaik ke generasi berikutnya. Induk dipilih dengan tournament selection, lalu crossover menggabungkan informasi penempatan atau penugasan paket dari dua induk.

Mutation secara probabilistik mengubah penempatan, orientasi, status muat, atau penugasan truk. Operator repair mencoba membangun penempatan valid; paket yang tidak dapat ditempatkan tetap berada di luar truk. Validitas mencakup batas dimensi dan kapasitas truk, tidak ada tumpang tindih, dukungan alas paket, serta aturan paket fragile. GA mengembalikan individu terbaik pada populasi akhir dan riwayat fitness maksimum serta rata-rata; metode ini tidak menjamin tercapainya global optimum.

Populasi awal dibentuk dengan inisialisasi penempatan paket secara acak. Pada GA, jumlah paket yang dipertahankan di dalam truk juga diacak agar individu awal tidak semuanya langsung padat. Karena itu, state awal yang ditampilkan pada visualisasi merupakan satu individu acak yang dipilih dari populasi awal. Paket di luar truk tetap menjadi bagian dari individu dan dapat dimasukkan pada generasi berikutnya. Seed mengendalikan pengulangan hasil acak.

Tabel 2.10 Tabel Daftar Fungsi Genetic Algorithm

Nama Fungsi	Parameter	Keterangan
`_fitness(state, objective="value")`	State/FleetState; objective	Menghitung fitness dengan objective_value untuk individu.
`_randomized_initial_individual(truck, packages, rng, repair_attempts)`	Truk/armada, paket, generator acak, batas percobaan repair	Membangun individu awal acak. Mengacak paket yang dipertahankan di dalam truk sehingga populasi tidak dimulai dari solusi greedy yang sudah padat.
`_place_genes(truck, template, genes, rng, repair_attempts)`	Truk, paket template, gen posisi/orientasi/status muat	Membentuk State dari gen. Memeriksa validitas, mencoba posisi alternatif, dan membiarkan paket di luar jika tidak berhasil diperbaiki.
`_genes_from(state)`	State	Mengubah state menjadi gen paket: posisi, orientasi, dan status di dalam/di luar truk.
`_crossover(first, second, rng, repair_attempts)`	Dua induk, generator acak, batas repair	Menggabungkan gen dari dua induk. Pada armada, pewarisan mencakup penugasan paket ke truk; hasil yang tidak valid diganti salinan salah satu induk.
`_mutate(state, rng, repair_attempts)`	Individu, generator acak, batas repair	Mutasi satu bagian individu. Untuk satu truk dapat memindahkan, memutar, mengeluarkan, atau memasukkan paket; untuk armada memilih tetangga valid, termasuk pemindahan paket antartruk.
`genetic_algorithm(truck, packages, population_size=40, iterations=100, mutation_rate=0.3, elite_size=2, tournament_size=3, repair_attempts=40, seed=None, objective="value")`	Parameter utama GA; mengembalikan dictionary hasil	Menjalankan inisialisasi populasi, pencatatan fitness generasi 0 hingga akhir, elitism, tournament selection, crossover, dan mutation. Keluaran mencakup initial_population, initial_state representatif acak, best_initial_state, final_state, final_value, max_history, average_history, ukuran populasi, iterasi, durasi, dan seed.
`run_genetic_experiments(truck, packages, iteration_values, population_values, repetitions=3, fixed_population_size=40,`	Daftar variasi parameter dan opsi GA	Menjalankan eksperimen perubahan jumlah iterasi dengan ukuran populasi tetap, serta perubahan ukuran populasi dengan jumlah iterasi tetap. Setiap konfigurasi diulang sesuai repetitions; hasil dipisahkan menjadi vary_iterations dan vary_population.

Nama Fungsi	Parameter	Keterangan
`fixed_iterations=100, seed=None, **ga_options)`		
`Fungsi pendukung: random_initial_state, generate_neighbors, is_valid, objective_value, outside_position`	State/FleetState, paket, objective	Mendukung inisialisasi, pembentukan kandidat tetangga, pemeriksaan kendala, penghitungan objective, dan penempatan koordinat paket di luar truk.

Fungsi utama Genetic Algorithm mengatur inisialisasi populasi acak, seleksi, crossover, dan mutasi. Potongan kode berikut menunjukkan proses tersebut dalam implementasi `genetic_algorithm()`.

```
def genetic_algorithm(
    truck: Truck | list[Truck],
    packages: list[Package],
    population_size: int = 40,
    iterations: int = 100,
    mutation_rate: float = 0.4,
    elite_size: int = 1,
    tournament_size: int = 2,
    repair_attempts: int = 40,
    seed: int | None = None,
    objective: str = "value",
) -> dict[str, Any]:
    """Run GA and return best states plus per-generation max/average
fitness."""
    if type(population_size) is not int or population_size < 2:
        raise ValueError("population_size harus bilangan bulat minimal 2.")
    if type(iterations) is not int or iterations < 0:
        raise ValueError("iterations harus bilangan bulat nonnegatif.")
    if not 0 <= mutation_rate <= 1:
        raise ValueError("mutation_rate harus berada pada rentang 0 sampai
1.")
    if type(elite_size) is not int or not 1 <= elite_size <
population_size:
        raise ValueError("elite_size harus minimal 1 dan lebih kecil dari
population_size.")
    if type(tournament_size) is not int or tournament_size < 1:
        raise ValueError("tournament_size harus bilangan bulat positif.")
    if type(repair_attempts) is not int or repair_attempts < 1:
        raise ValueError("repair_attempts harus bilangan bulat positif.")
```

```
    rng = random.Random(seed)
    started = time.perf_counter()
    population = [
        _randomized_initial_individual(truck, packages, rng,
repair_attempts)
        for _ in range(population_size)
    ]
    if not all(is_valid(individual) for individual in population):
        raise RuntimeError("Inisialisasi menghasilkan state yang tidak
valid.")
    initial_population = [individual.copy() for individual in population]
    initial_scores = [_fitness(individual, objective) for individual in
initial_population]
    initial_best_score = max(initial_scores, default=0.0)
    non_best_initial = [
        individual for individual, score in zip(initial_population,
initial_scores)
        if score < initial_best_score
    ]
    # Keep the displayed starting individual representative of a random
    # population member rather than accidentally showing the best
individual.
    display_rng = random.Random(None if seed is None else seed ^ 0x5A17)
    displayed_initial_state = display_rng.choice(non_best_initial or
initial_population).copy()
    max_history: list[float] = []
    average_history: list[float] = []

    def record() -> None:
        scores = [_fitness(individual, objective) for individual in
population]
        max_history.append(max(scores, default=0.0))
        average_history.append(sum(scores) / len(scores) if scores else
0.0)

    record()
    for _ in range(iterations):
        ranked = sorted(population, key=lambda individual:
_fitness(individual, objective), reverse=True)
        next_population = [individual.copy() for individual in
ranked[:elite_size]]
        while len(next_population) < population_size:
            pool = rng.sample(population, min(tournament_size,
len(population)))
```

```
            parent_a = max(pool, key=lambda individual:
_fitness(individual, objective))
            pool = rng.sample(population, min(tournament_size,
len(population)))
            parent_b = max(pool, key=lambda individual:
_fitness(individual, objective))
            child = _crossover(parent_a, parent_b, rng, repair_attempts)
            if rng.random() < mutation_rate and child.packages:
                child = _mutate(child, rng, repair_attempts)
            next_population.append(child)
        population = next_population
        record()

    best = max(population, key=lambda individual: _fitness(individual,
objective))
    return {
        "initial_population": initial_population,
        # A single representative initial state should be an actual random
        # individual, not the population's best member. Population-level
best
        # and average are recorded separately in their history series.
        "initial_state": displayed_initial_state,
        "best_initial_state": max(
            initial_population, key=lambda individual: _fitness(individual,
objective)
        ).copy(),
        "final_state": best.copy(),
        "final_value": _fitness(best, objective),
        "objective_name": objective,
        "max_history": max_history,
        "average_history": average_history,
        "population_size": population_size,
        "iterations": iterations,
        "duration_seconds": time.perf_counter() - started,
        "seed": seed,
    }
```

BAB III

HASIL PENGUJIAN DAN PEMBAHASAN

A. Algoritma Hill Climbing

Pengujian Hill Climbing dilakukan terhadap empat varian, yaitu Steepest Ascent, Sideways Move, Random Restart, dan Stochastic. Setiap varian diuji sebanyak tiga kali menggunakan konfigurasi parameter yang sama dengan seed berbeda. Fungsi objektif yang digunakan adalah value, yaitu jumlah nilai paket yang berhasil dimuat ke dalam truk. Seluruh pengujian menggunakan satu truk dan konfigurasi problem yang sama.

Steepest Ascent Hill Climbing

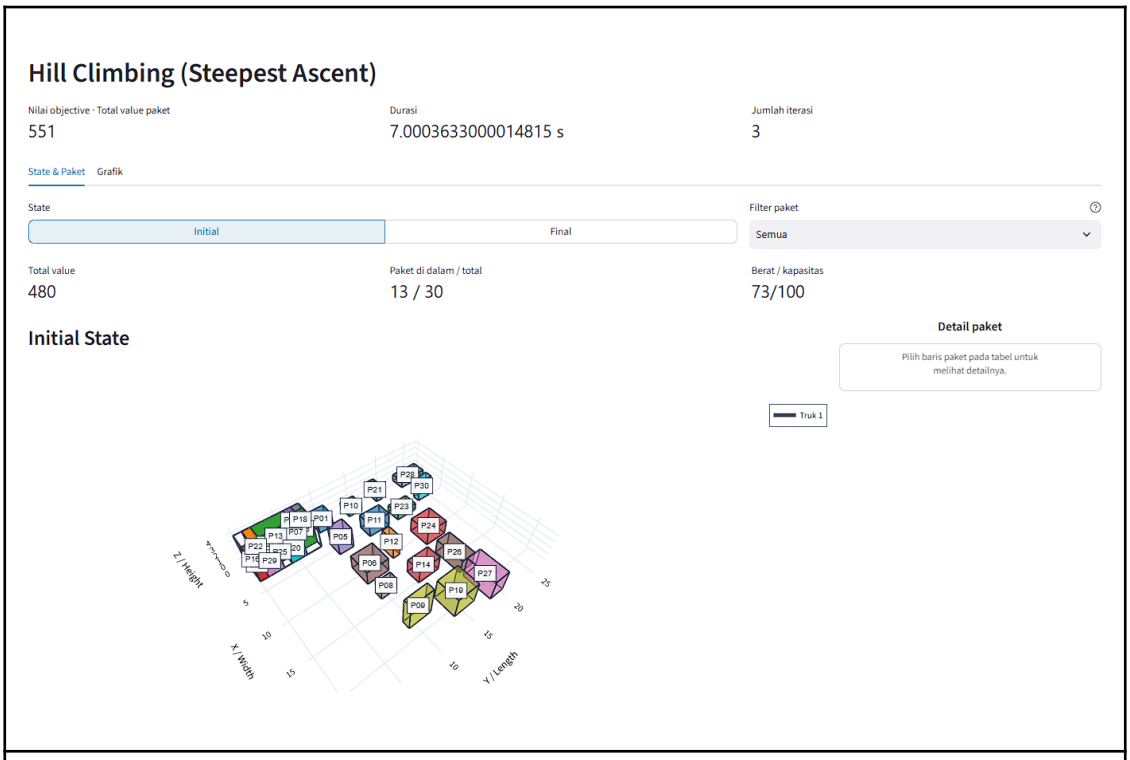
Pengujian Steepest Ascent menggunakan seed berbeda untuk menghasilkan konfigurasi awal yang berbeda. Algoritma mengevaluasi seluruh neighbor dan berhenti ketika tidak ditemukan peningkatan objective.

Tabel 3.1 Variasi Parameter Pengujian Steepest Ascent

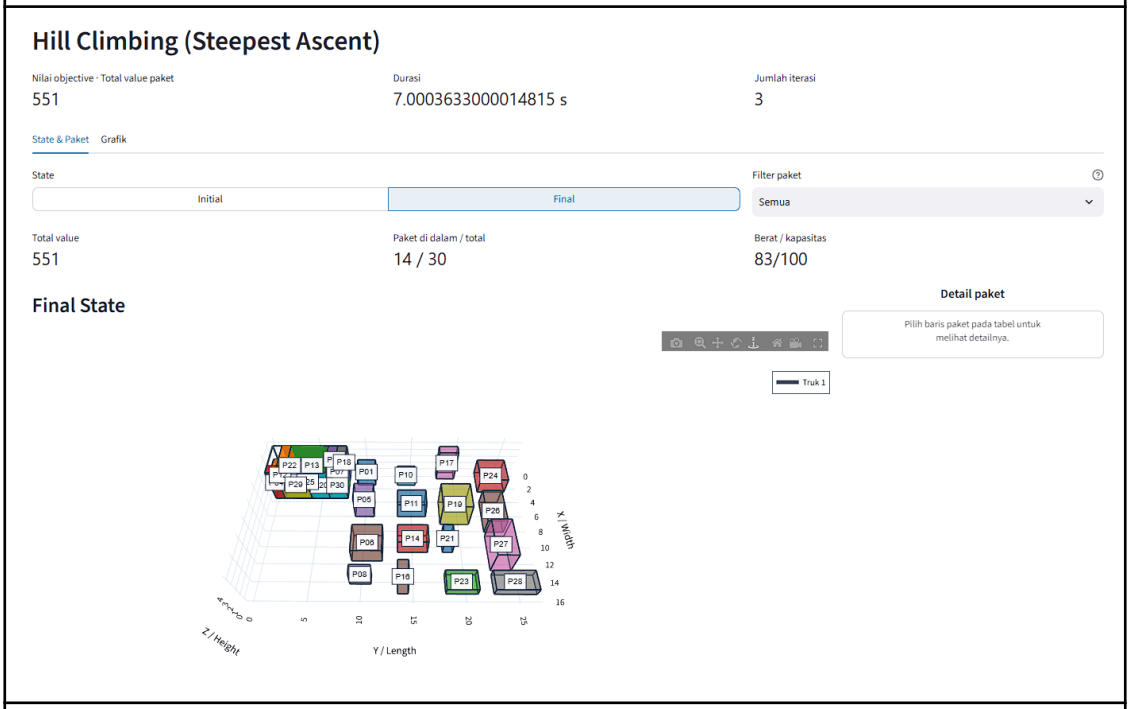
Parameter	Pengujian 1	Pengujian 2	Pengujian 3
Seed	42	43	44
Fungsi objektif	value	value	value
Jumlah truk	1	1	1

1. Pengujian HC-SA-01

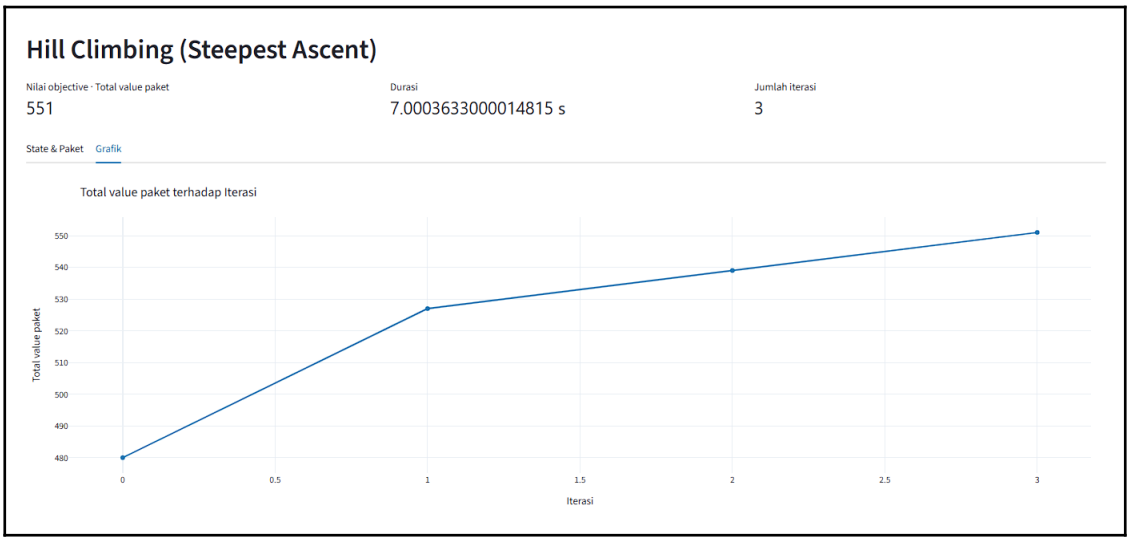
Initial State



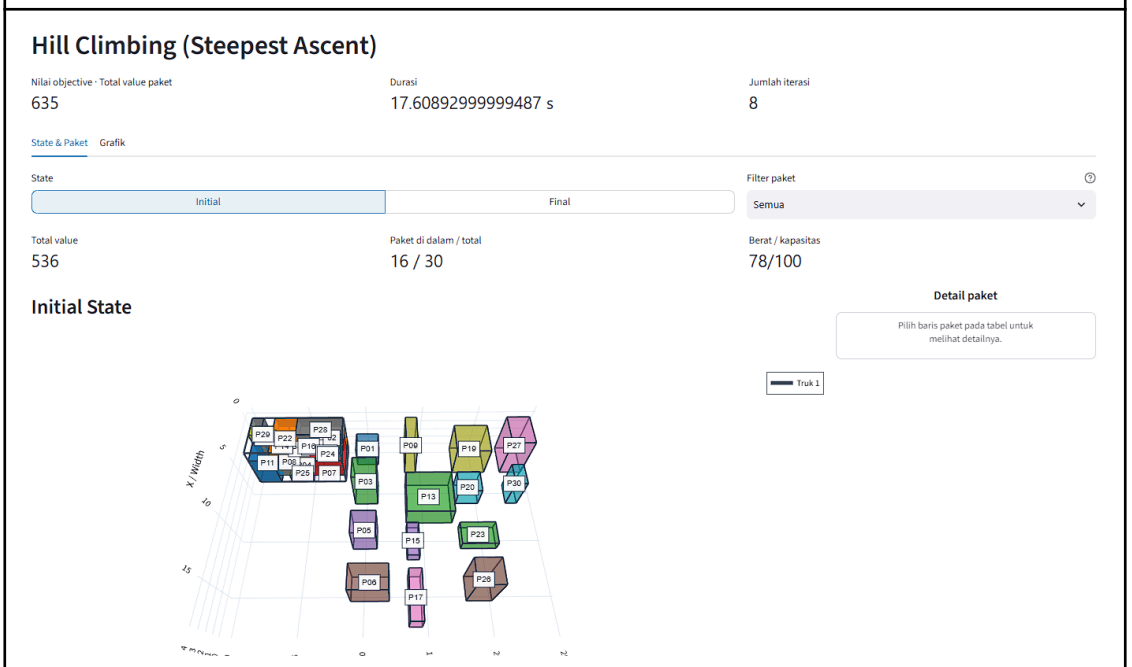
Final State

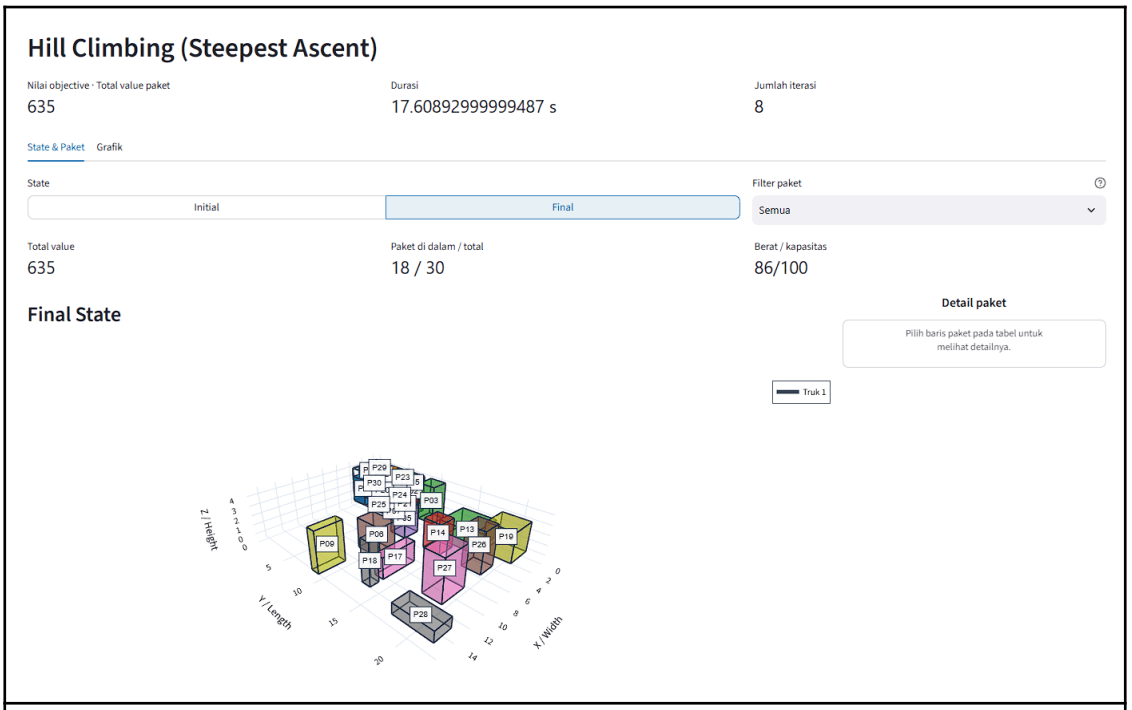


Grafik Objektif

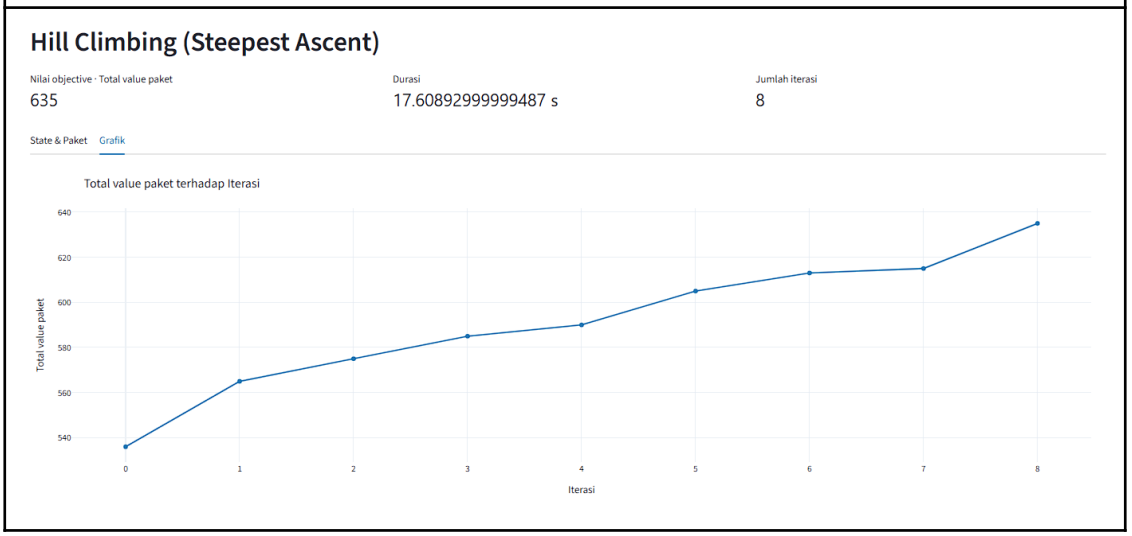


2. Pengujian HC-SA-02

Initial State

Final State



Grafik Objektif



3. Pengujian HC-SA-03

Initial State

Hill Climbing (Steepest Ascent)

Nilai objective · Total value paket
621

Durasi
12.558382499999425 s

Jumlah iterasi
6

State & Paket | Grafik

State: Initial | Final

Filter paket: Semua

Total value
513

Paket di dalam / total
17 / 30

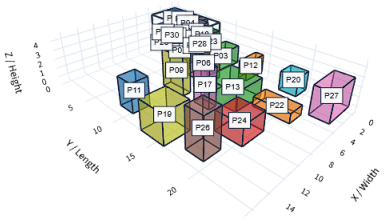
Berat / kapasitas
69/100

Initial State

Detail paket

Pilih baris paket pada tabel untuk melihat detailnya.

Truk 1



Final State

Hill Climbing (Steepest Ascent)

Nilai objective · Total value paket
621

Durasi
12.558382499999425 s

Jumlah iterasi
6

State & Paket | Grafik

State: Initial | Final

Filter paket: Semua

Total value
621

Paket di dalam / total
19 / 30

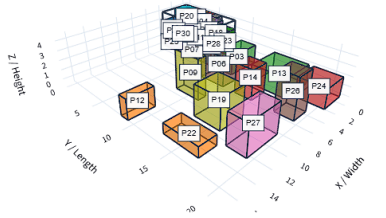
Berat / kapasitas
82/100

Final State

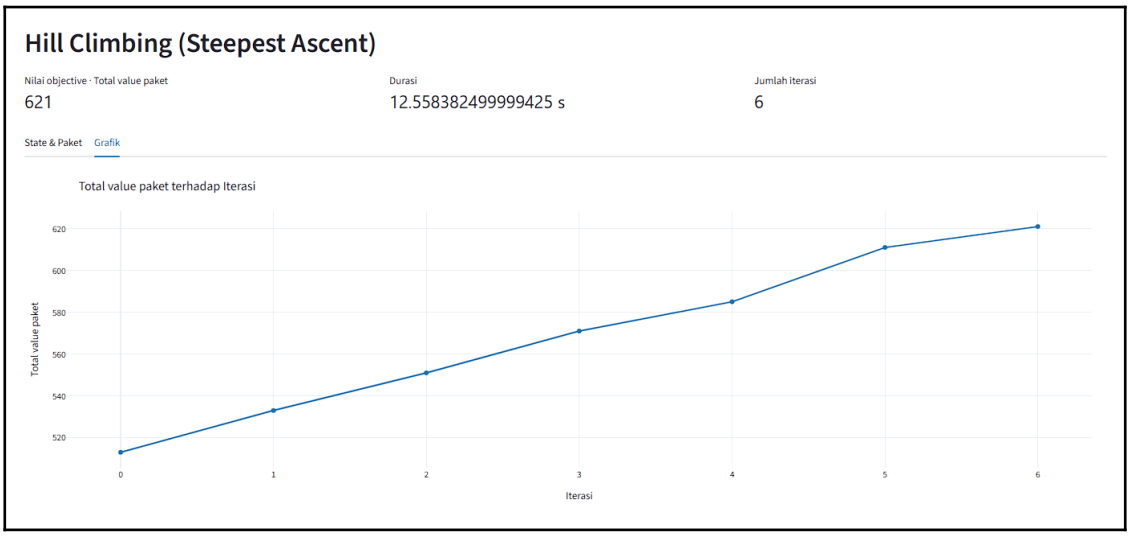
Detail paket

Pilih baris paket pada tabel untuk melihat detailnya.

Truk 1



Grafik Objektif



Hill Climbing with Sideways Move

Pengujian Sideways Move menggunakan batas total sideways move yang sama pada seluruh pengujian. Seed dibedakan untuk menghasilkan konfigurasi awal dan pemilihan kandidat bernilai sama yang dapat berbeda.

Tabel 3.2 Variasi Parameter Pengujian Sideways Move

Parameter	Pengujian 1	Pengujian 2	Pengujian 3
Seed	42	43	44
Maksimum sideways move	100	100	100
Fungsi objektif	value	value	value
Jumlah truk	1	1	1

4. Pengujian HC-SM-01

Initial State

Sideways

Nilai objective · Total value paket
607

Durasi
164.40569989999494 s

Jumlah iterasi
107

State & Paket | Grafik

State: Initial | Final

Filter paket: Semua

Total value
480

Paket di dalam / total
13 / 30

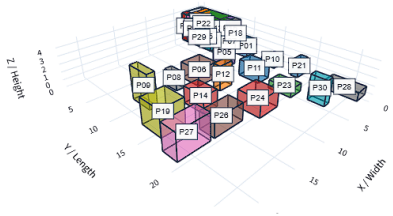
Berat / kapasitas
73/100

Initial State

Detail paket

Pilih baris paket pada tabel untuk melihat detailnya.

Truk 1



Final State

Sideways

Nilai objective · Total value paket
607

Durasi
164.40569989999494 s

Jumlah iterasi
107

State & Paket | Grafik

State: Initial | Final

Filter paket: Semua

Total value
607

Paket di dalam / total
16 / 30

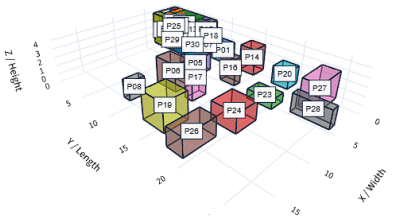
Berat / kapasitas
97/100

Final State

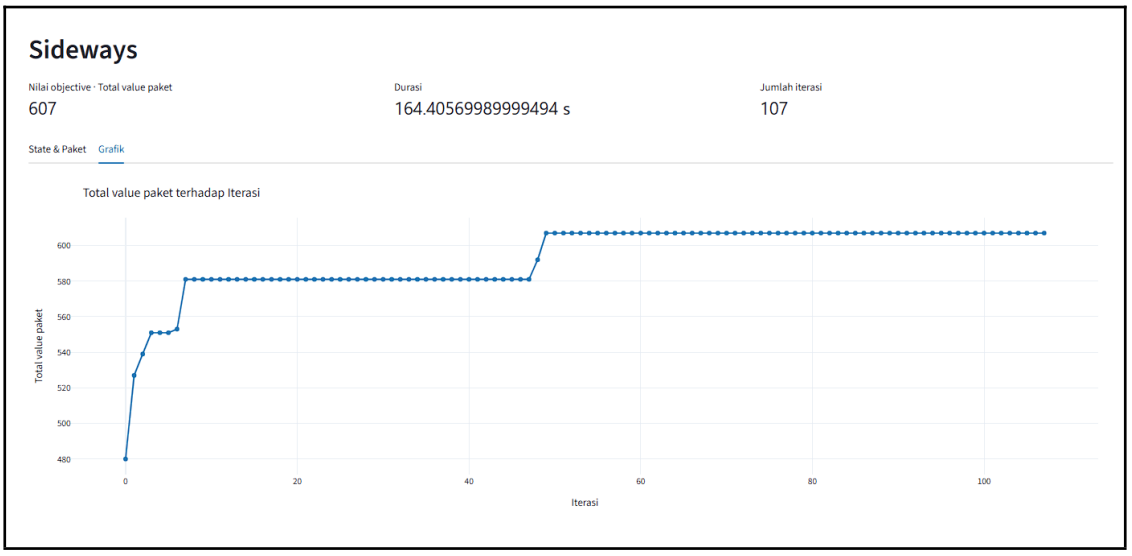
Detail paket

Pilih baris paket pada tabel untuk melihat detailnya.

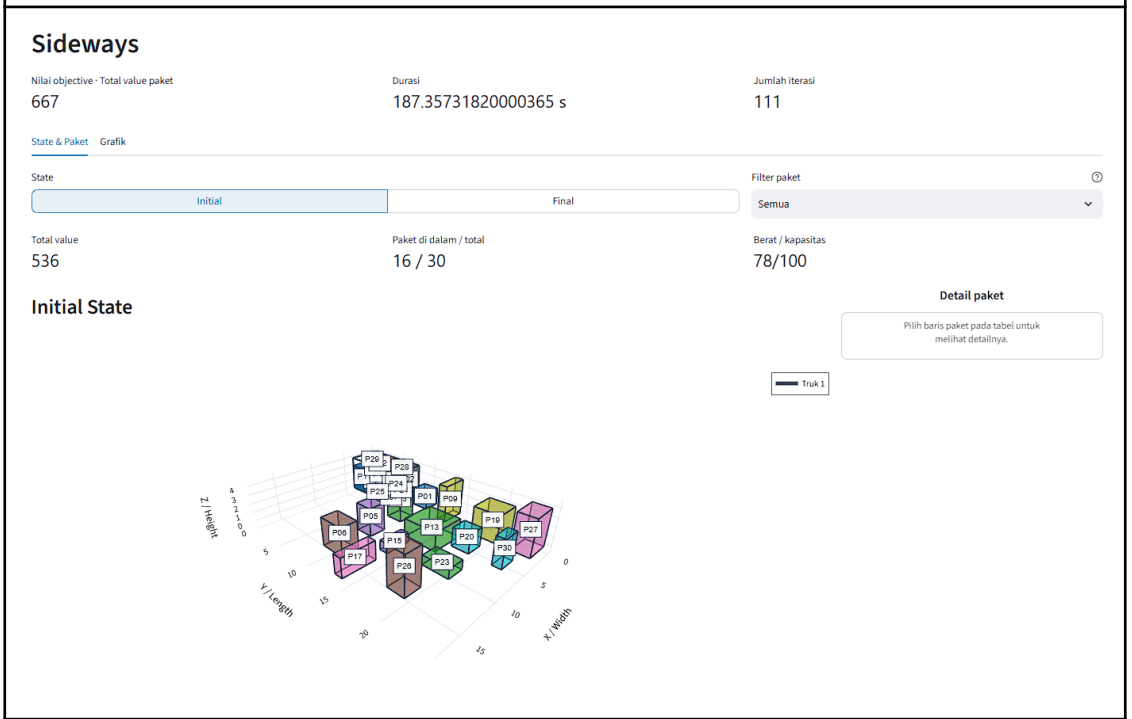
Truk 1

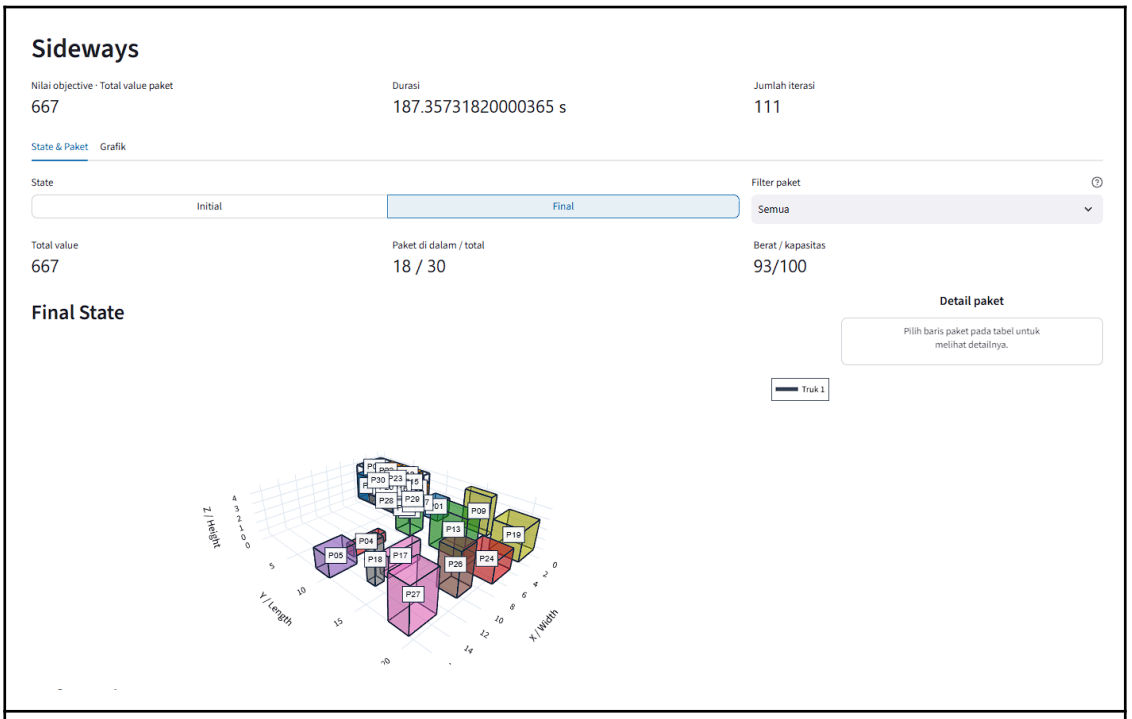


Grafik Objektif

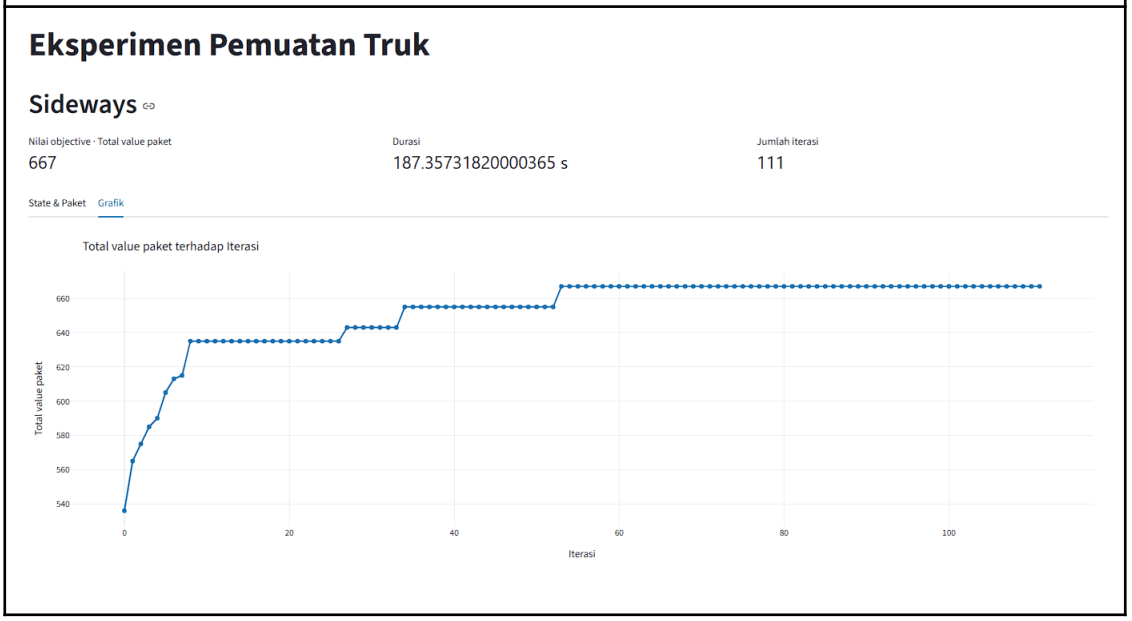


5. Pengujian HC-SM-02

Initial State

Final State

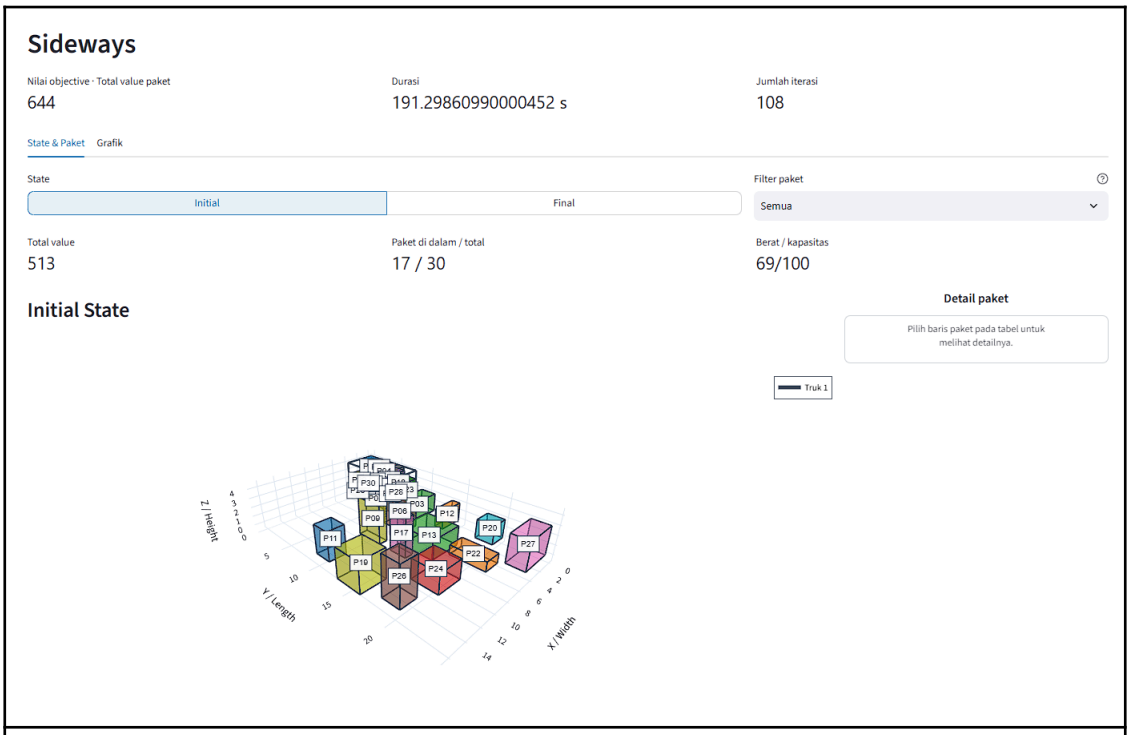


Grafik Objektif

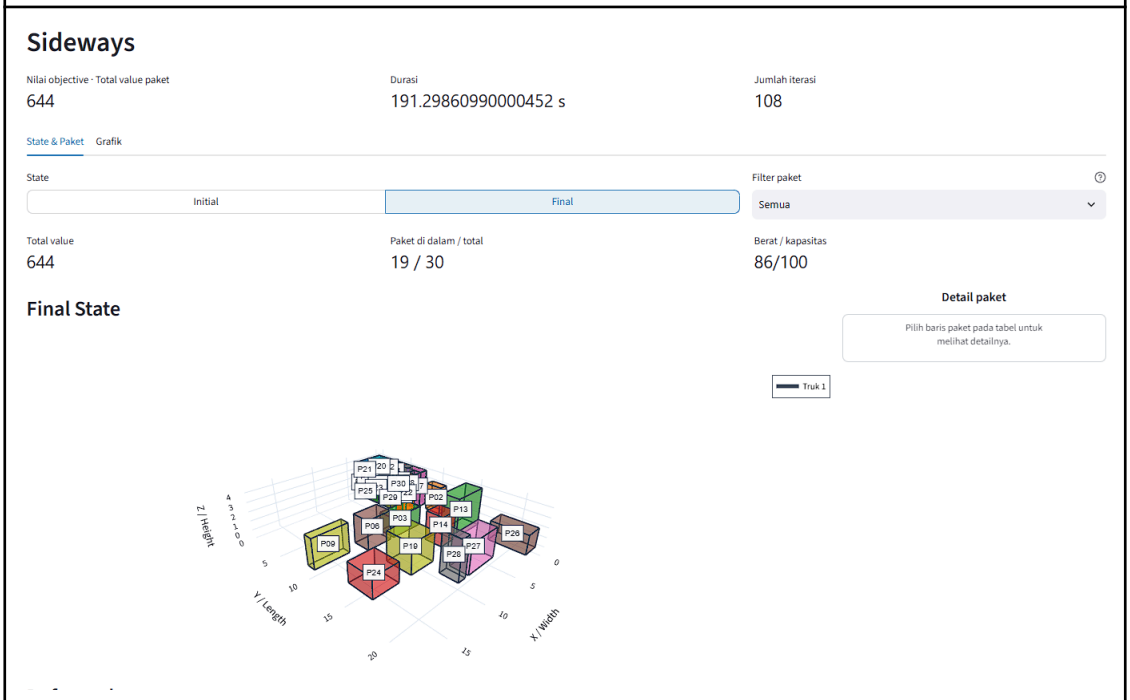


6. Pengujian HC-SM-03

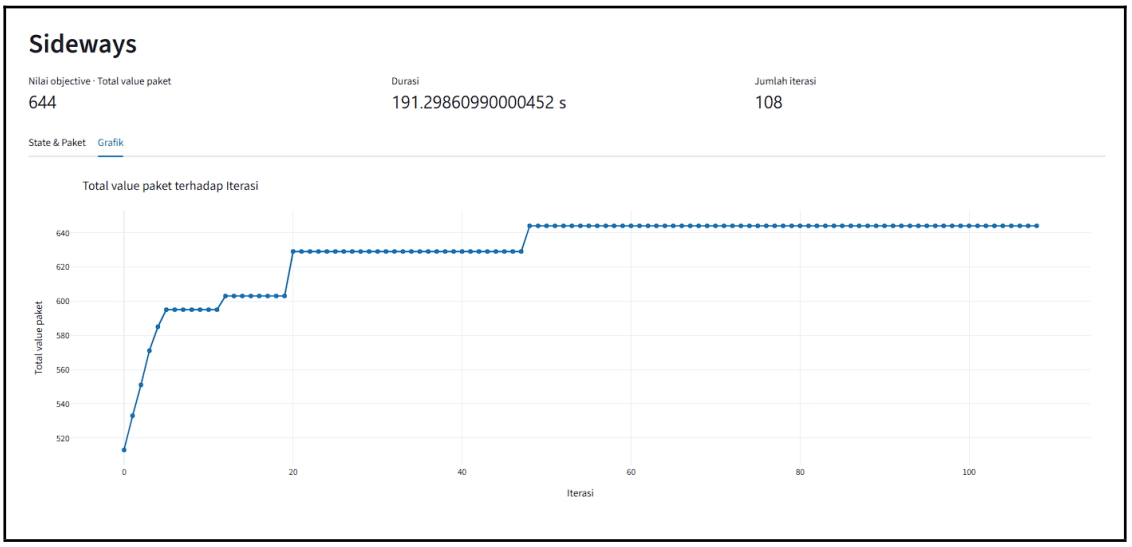
Initial State



Final State



Grafik Objektif



Random Restart Hill Climbing

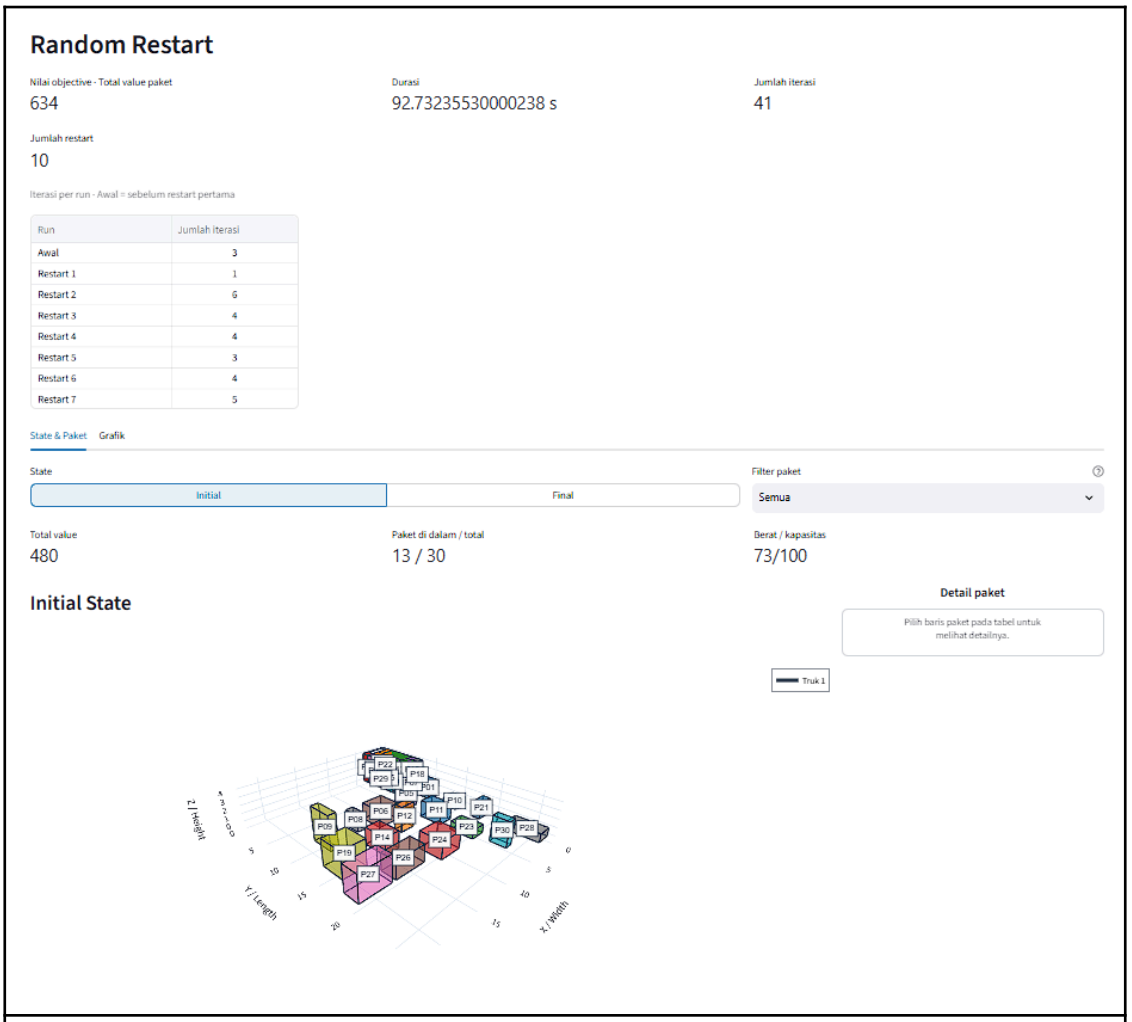
Pengujian Random Restart menggunakan batas restart dan batas percobaan penempatan paket yang sama. Seed dibedakan untuk menghasilkan konfigurasi awal serta rangkaian state restart yang berbeda. Dengan maksimum 10 restart, setiap pengujian menjalankan satu run awal dan sepuluh run restart.

Tabel 3.3 Variasi Parameter Pengujian Random Restart

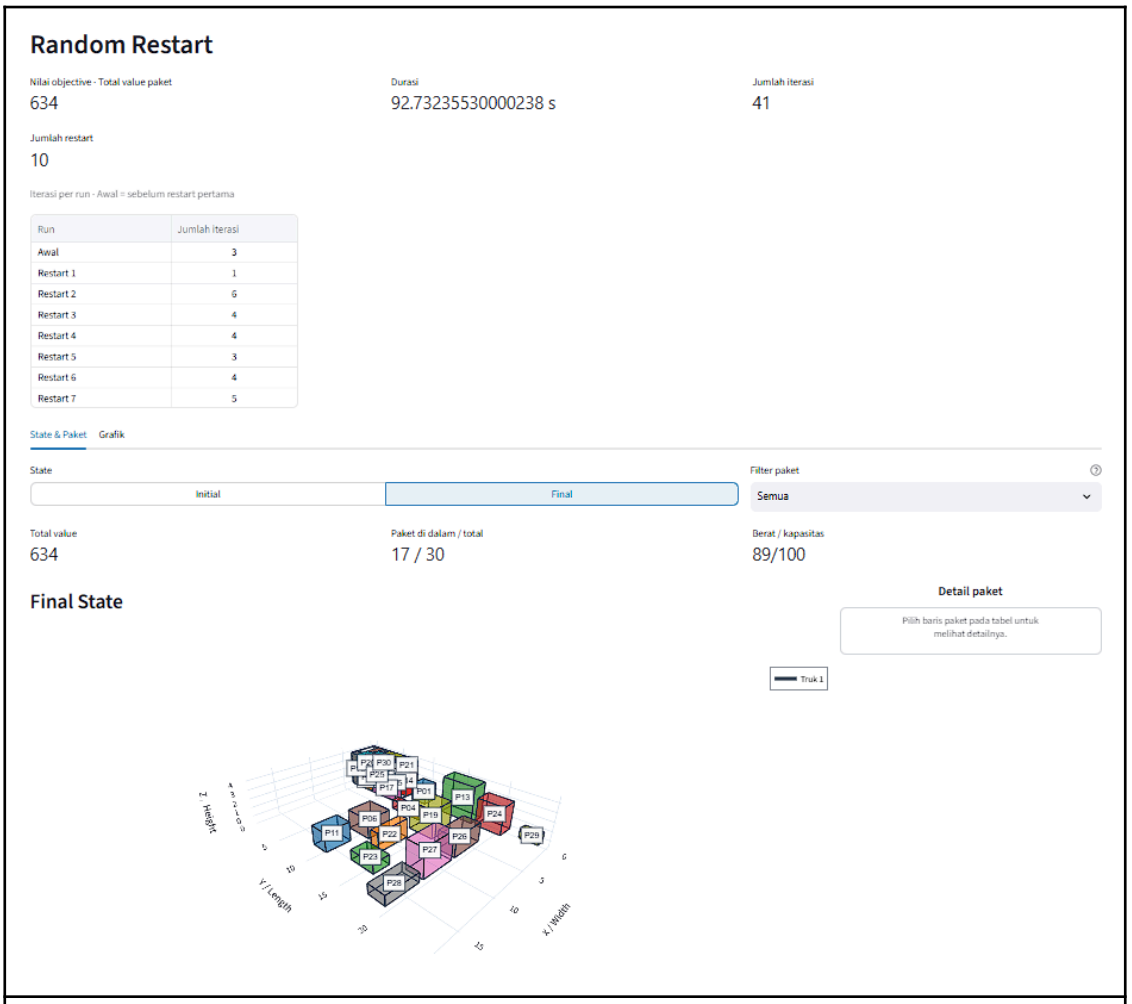
Parameter	Pengujian 1	Pengujian 2	Pengujian 3
Seed	42	43	44
Maksimum restart	10	10	10
Maksimum percobaan membentuk state awal restart	50	50	50
Fungsi objektif	value	value	value
Jumlah truk	1	1	1

7. Pengujian HC-RR-01

Initial State


Random Restart
Nilai objective - Total value paket
634
Durasi
92.73235530000238 s
Jumlah iterasi
41
Jumlah restart
10
Iterasi per run - Awal = sebelum restart pertama
Run
Jumlah iterasi
Awal
3
Restart 1
1
Restart 2
6
Restart 3
4
Restart 4
4
Restart 5
3
Restart 6
4
Restart 7
5
State & Paket
Grafik
State
Initial
Final
Filter paket
Semua
Total value
480
Paket di dalam / total
13 / 30
Berat / kapasitas
73/100
Initial State
Detail paket
Pilih baris paket pada tabel untuk melihat detailnya.
Truk 1
z / Height
x / Width
y / Length

Final State



Grafik Objektif

Random Restart

Nilai objective - Total value paket
634

Durasi
92.73235530000238 s

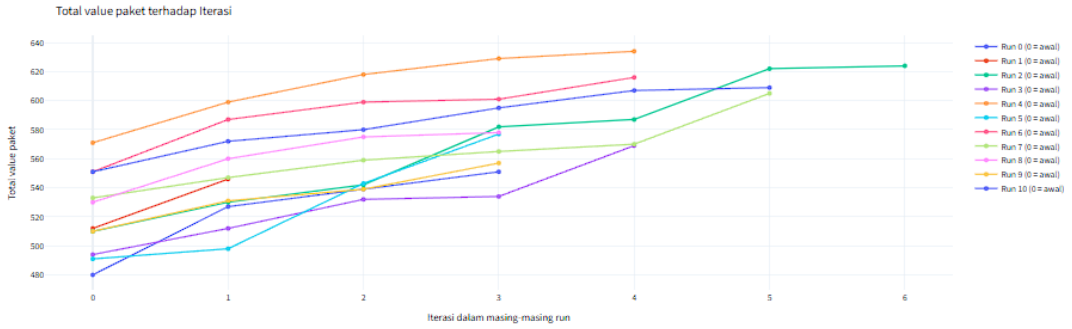
Jumlah iterasi
41

Jumlah restart
10

Iterasi per run - Awal = sebelum restart pertama

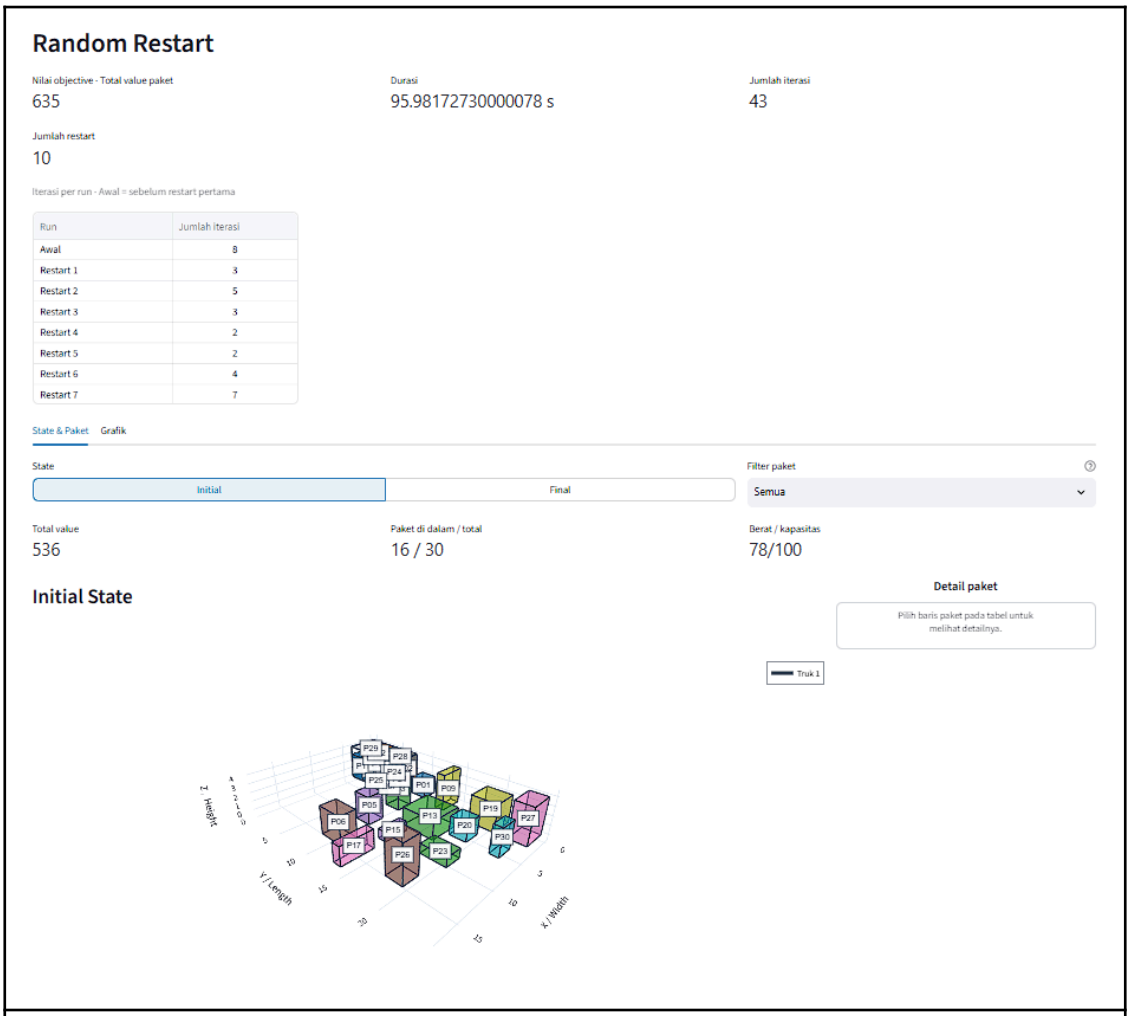
Run	Jumlah iterasi
Awal	3
Restart 1	1
Restart 2	6
Restart 3	4
Restart 4	4
Restart 5	3
Restart 6	4
Restart 7	5

State & Paket | Grafik

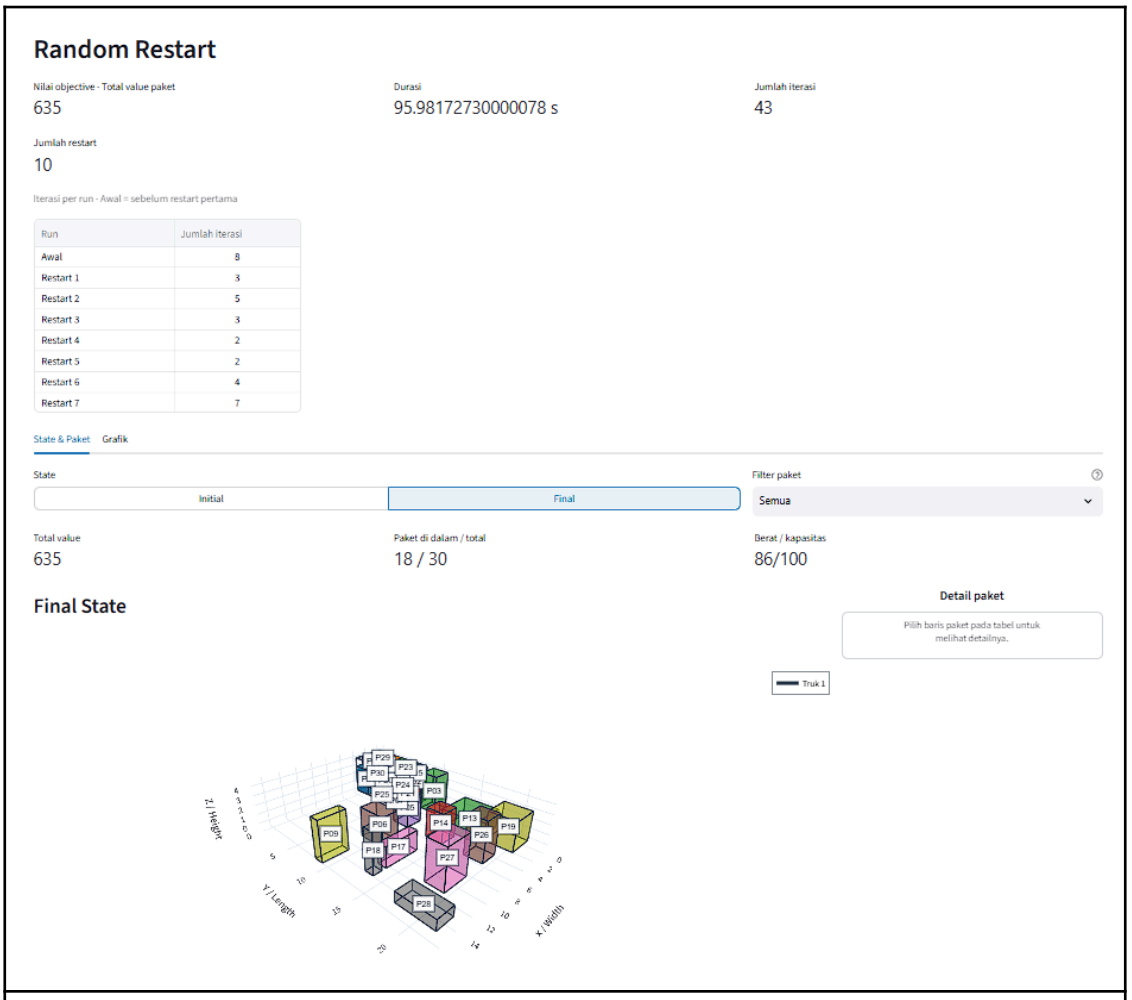


8. Pengujian HC-RR-02

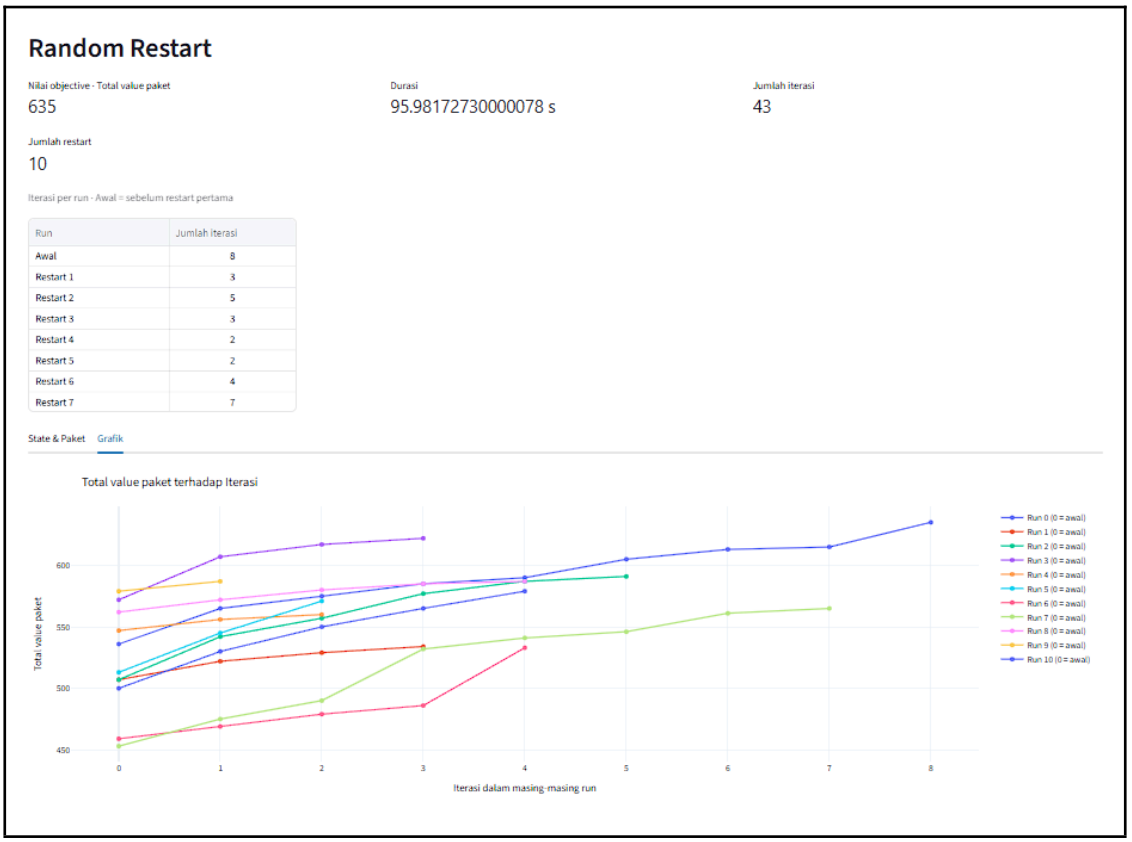
Initial State


Random Restart
Nilai objective - Total value paket
635
Durasi
95.98172730000078 s
Jumlah iterasi
43
Jumlah restart
10
Iterasi per run - Awal = sebelum restart pertama
Run
Jumlah iterasi
Awal
8
Restart 1
3
Restart 2
5
Restart 3
3
Restart 4
2
Restart 5
2
Restart 6
4
Restart 7
7
State & Paket
Grafik
State
Initial
Final
Filter paket
Semua
Total value
536
Paket di dalam / total
16 / 30
Berat / kapasitas
78/100
Initial State
Detail paket
Pilih baris paket pada tabel untuk melihat detailnya.
Truk 1
Z / Height
Y / Length
X / Width

Final State

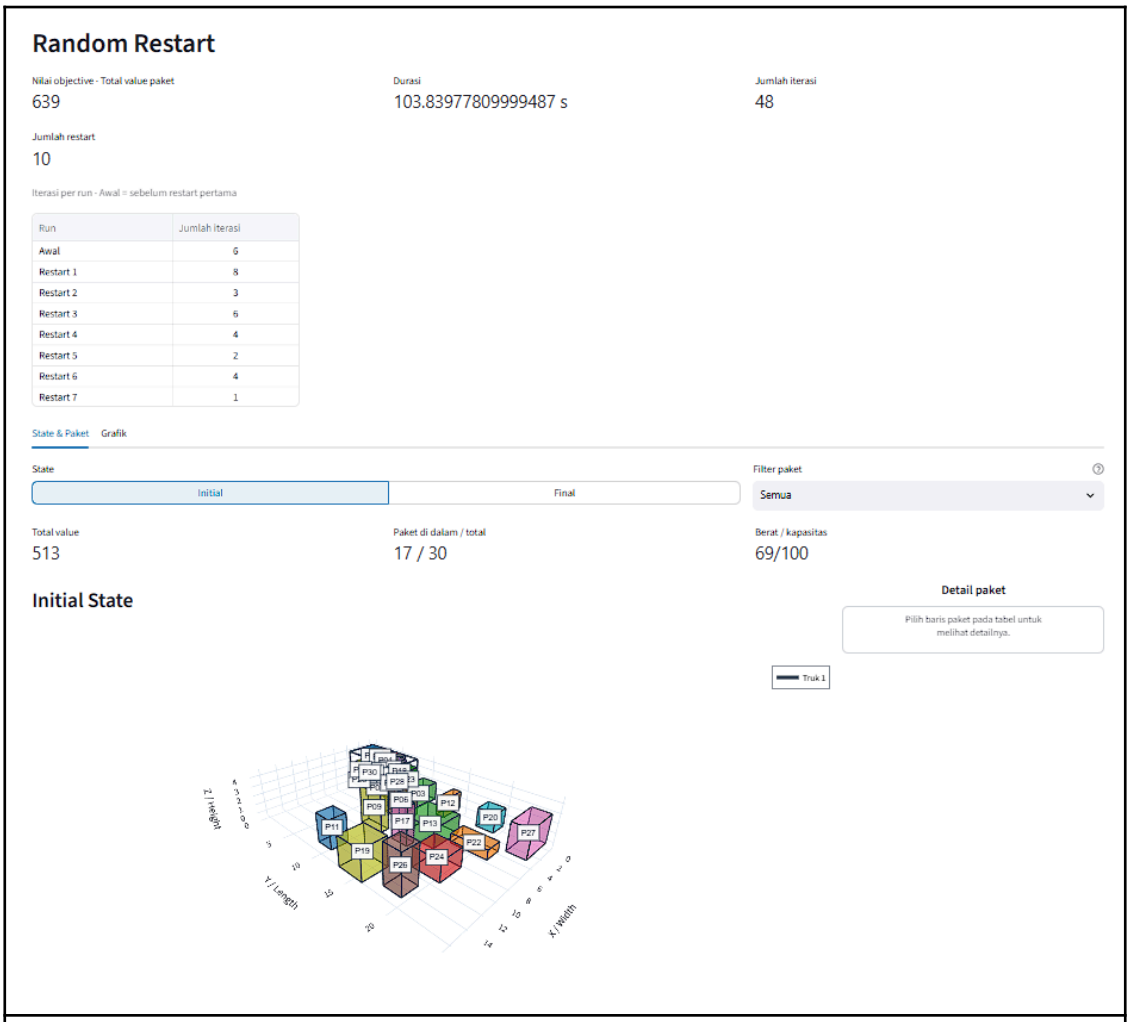


Grafik Objektif

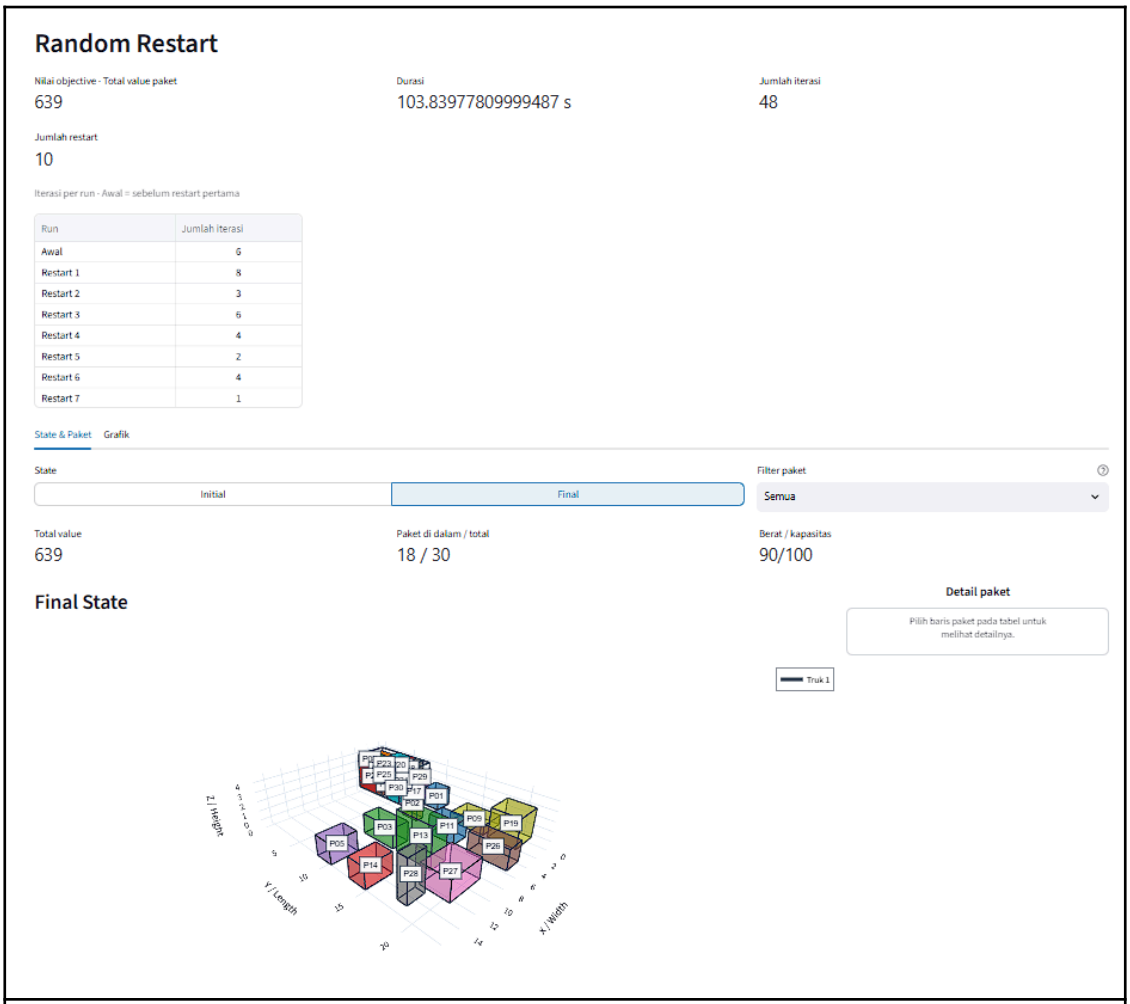


9. Pengujian HC-RR-03

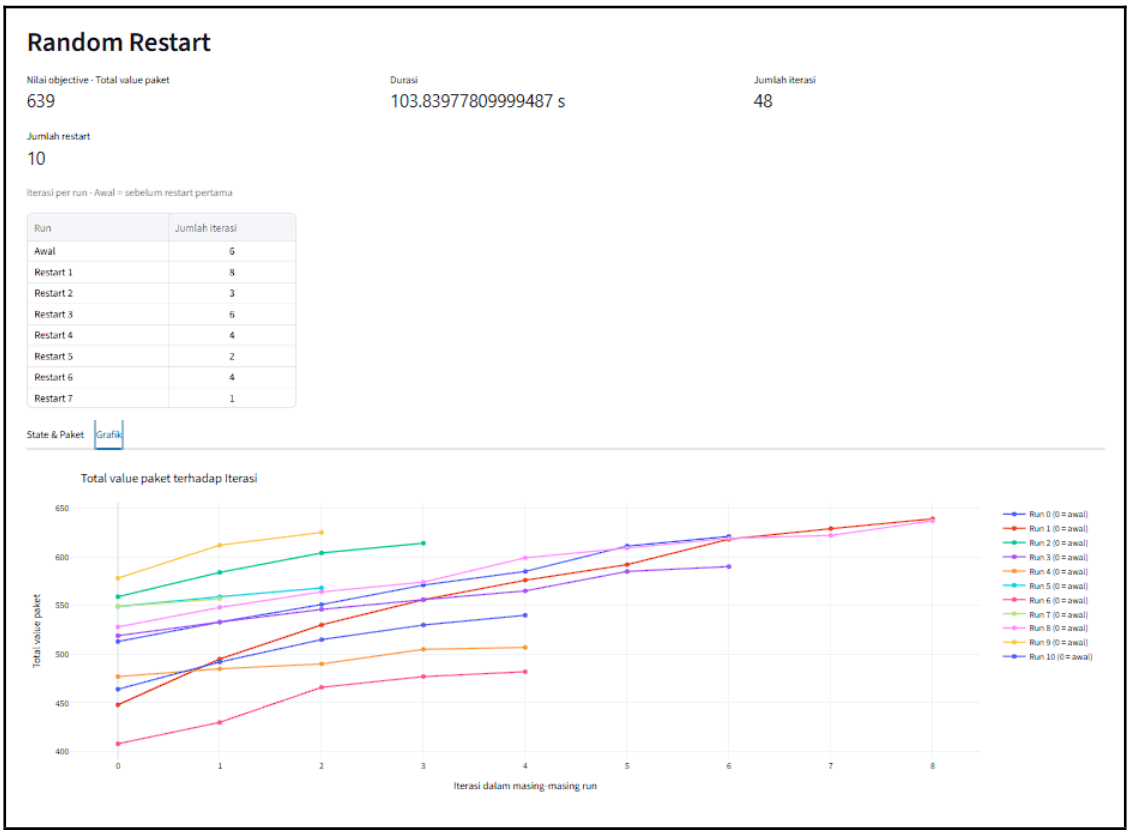
Initial State


Random Restart
Nilai objective - Total value paket
639
Durasi
103.83977809999487 s
Jumlah iterasi
48
Jumlah restart
10
Iterasi per run - Awal = sebelum restart pertama
Run
Jumlah iterasi
Awal
6
Restart 1
8
Restart 2
3
Restart 3
6
Restart 4
4
Restart 5
2
Restart 6
4
Restart 7
1
State & Paket
Grafik
State
Initial
Final
Filter paket
Semua
Total value
513
Paket di dalam / total
17 / 30
Berat / kapasitas
69/100
Initial State
Detail paket
Pilih baris paket pada tabel untuk melihat detailnya.
Truk 1
Z / Height
Y / Length
X / Width

Final State



Grafik Objektif



Stochastic Hill Climbing

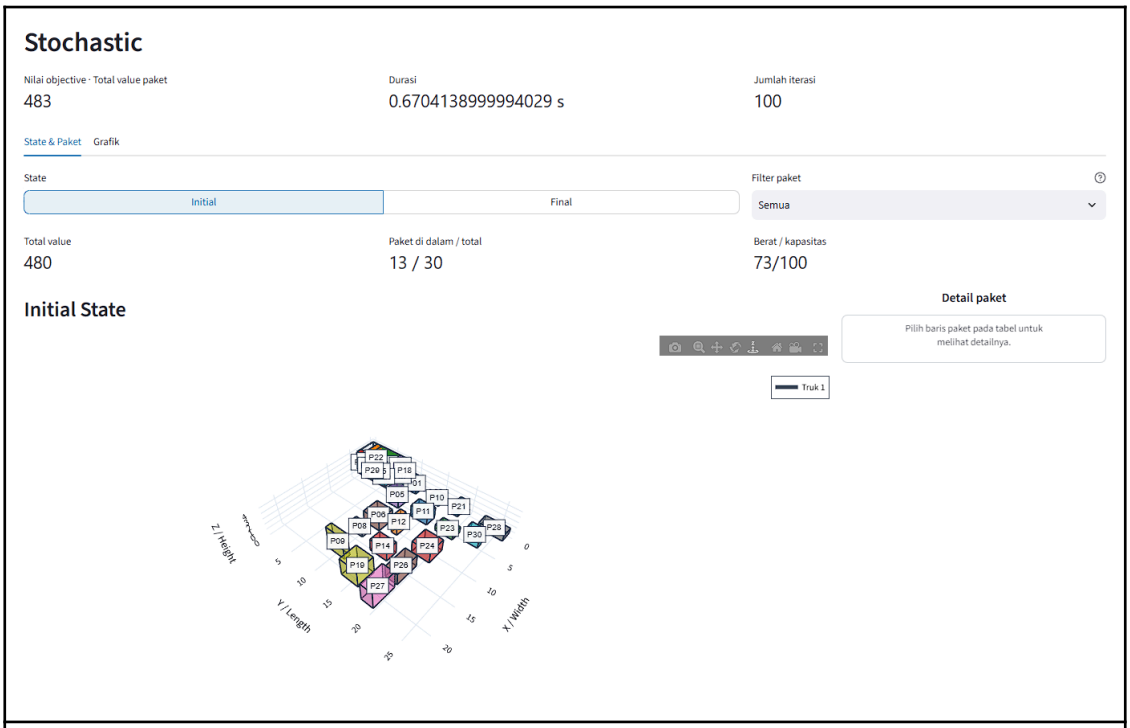
Pengujian Stochastic menggunakan batas maksimum percobaan yang sama. Seed dibedakan untuk menghasilkan konfigurasi awal dan urutan proposal acak yang berbeda. Setiap proposal dihitung sebagai iterasi, sedangkan jumlah perpindahan yang meningkatkan objective dicatat secara terpisah sebagai langkah diterima.

Tabel 3.4 Variasi Parameter Pengujian Stochastic HC

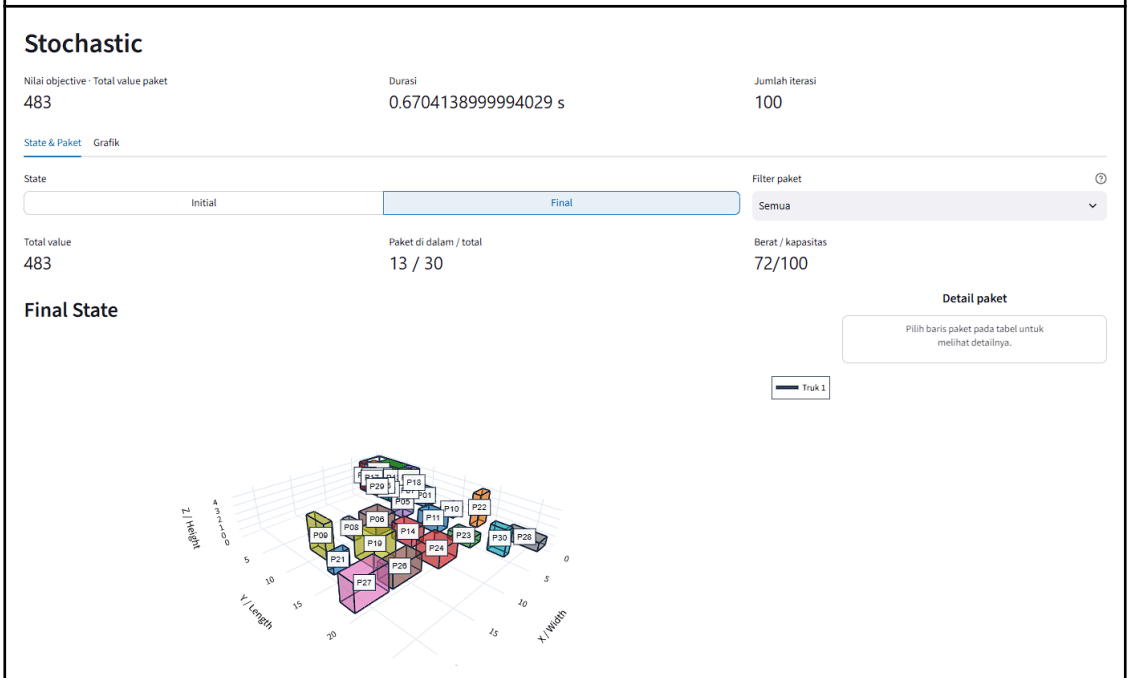
Parameter	Pengujian 1	Pengujian 2	Pengujian 3
Seed	42	43	44
Maksimum percobaan proposal (nmax)	100	100	100
Fungsi objektif	value	value	value
Jumlah truk	1	1	1

10. Pengujian HC-S-01

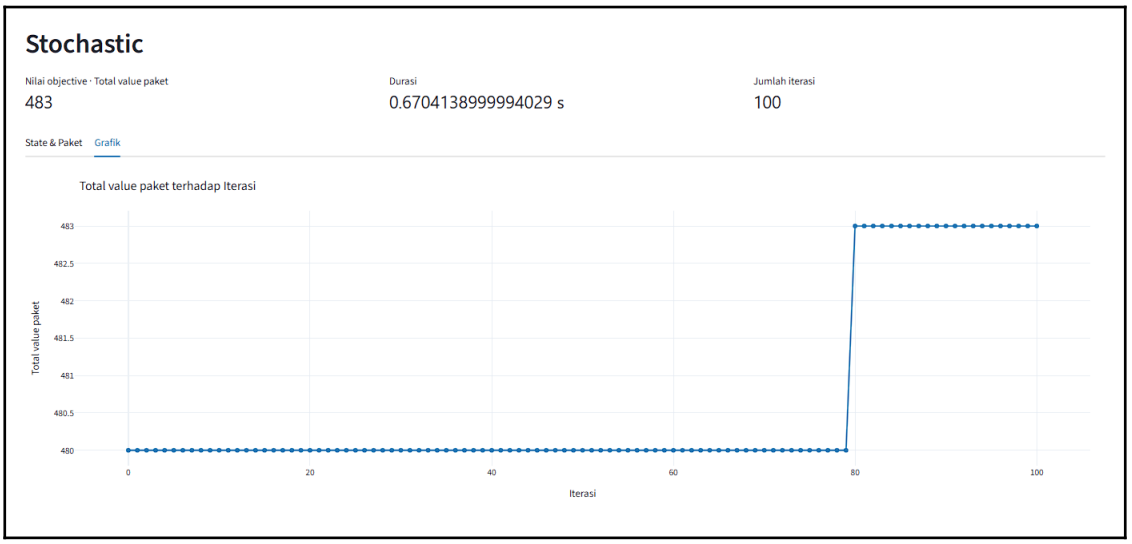
Initial State



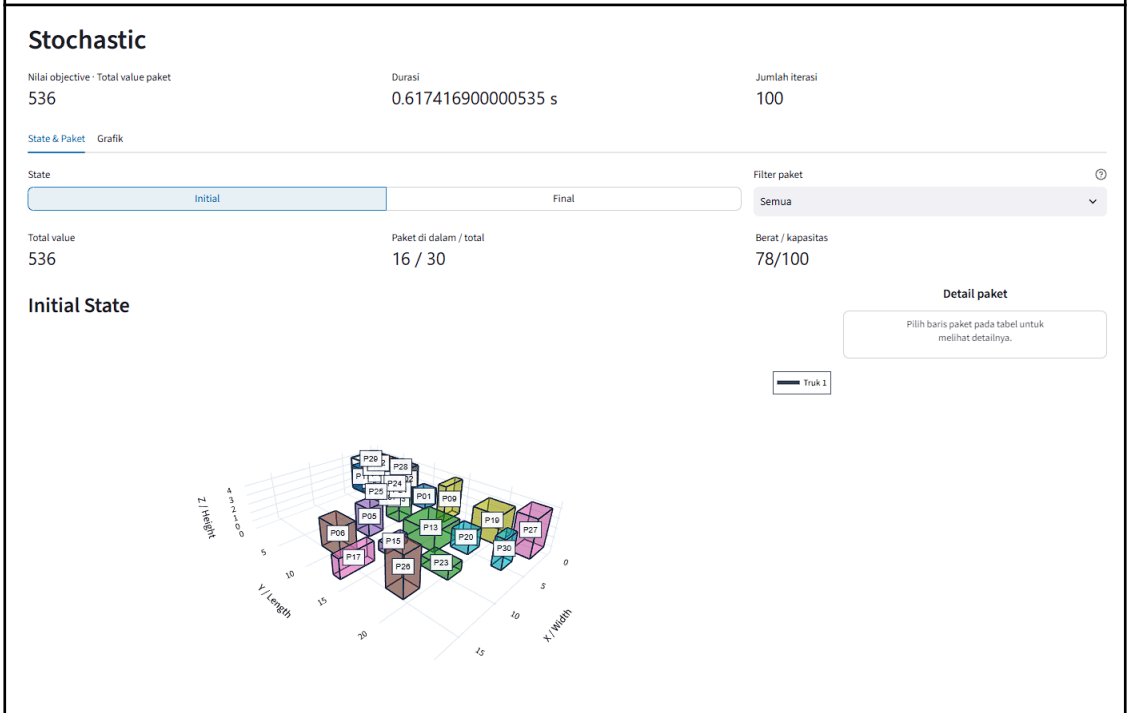
Final State

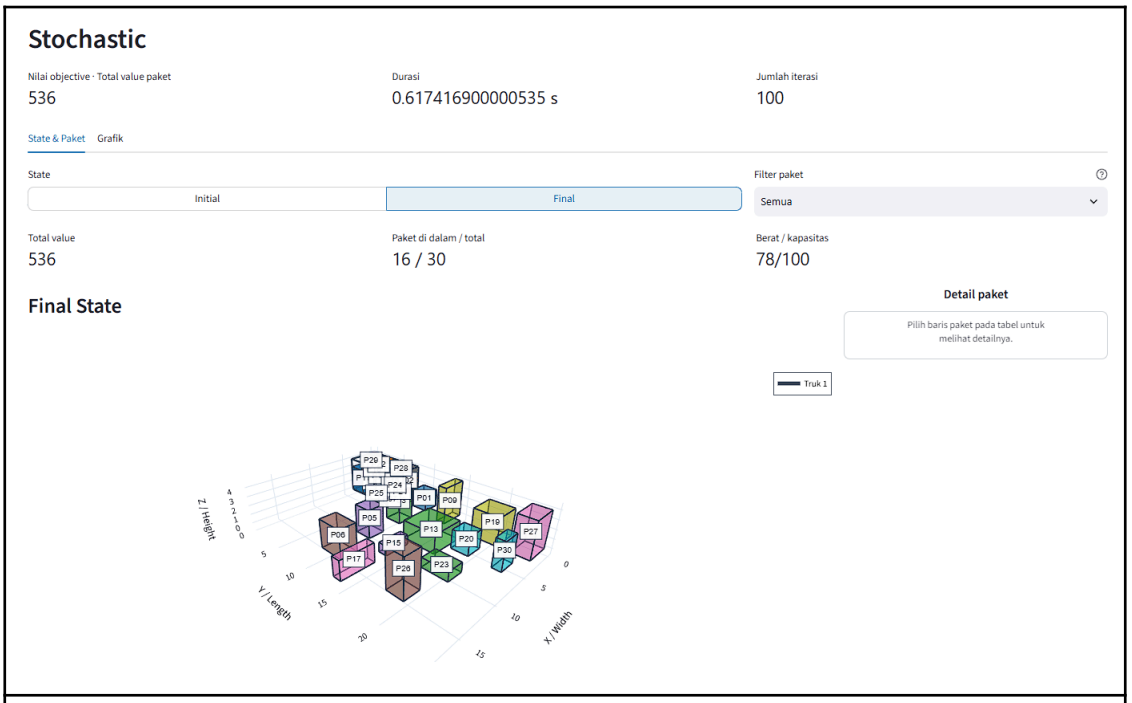


Grafik Objektif

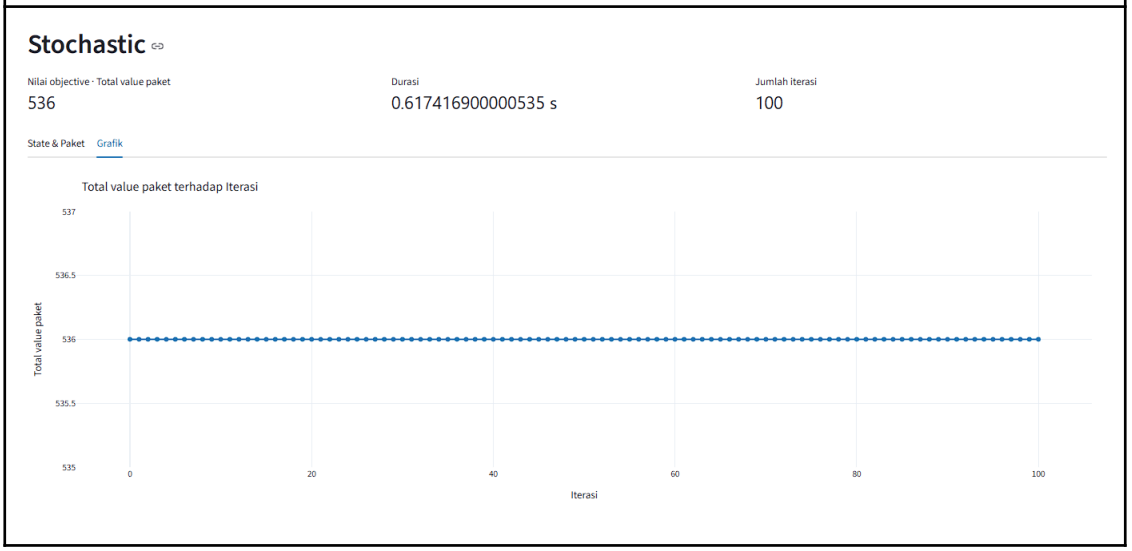


11. Pengujian HC-S-02

Initial State

Final State

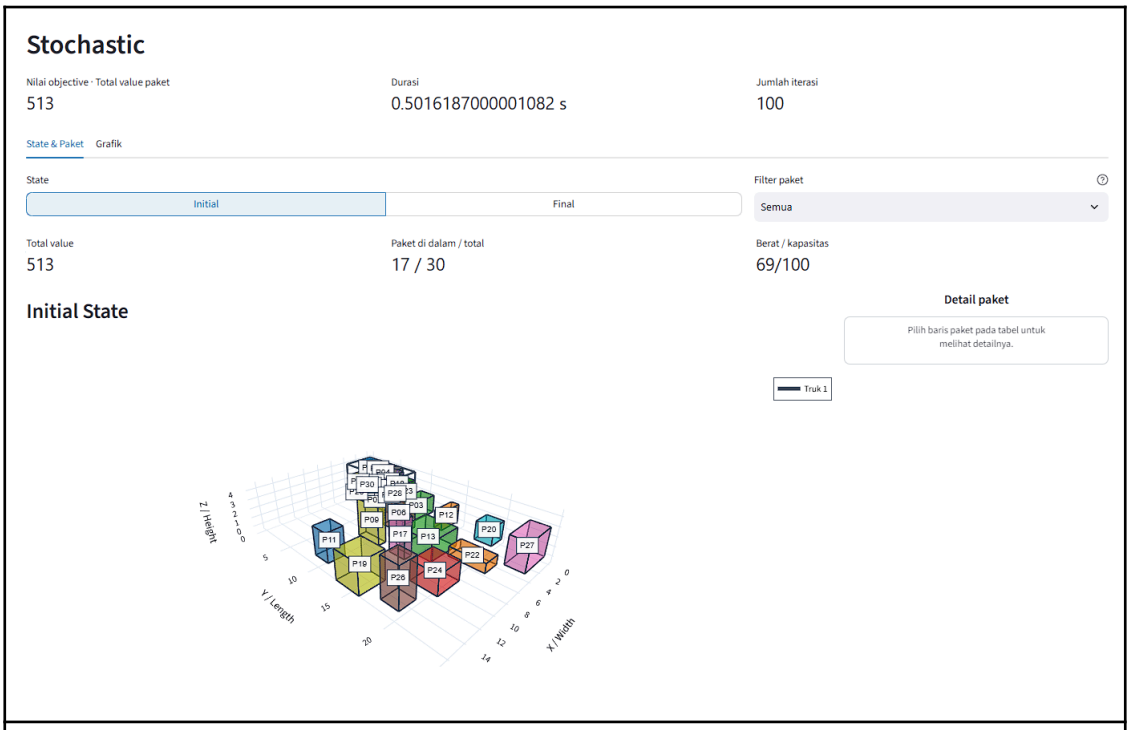


Grafik Objektif

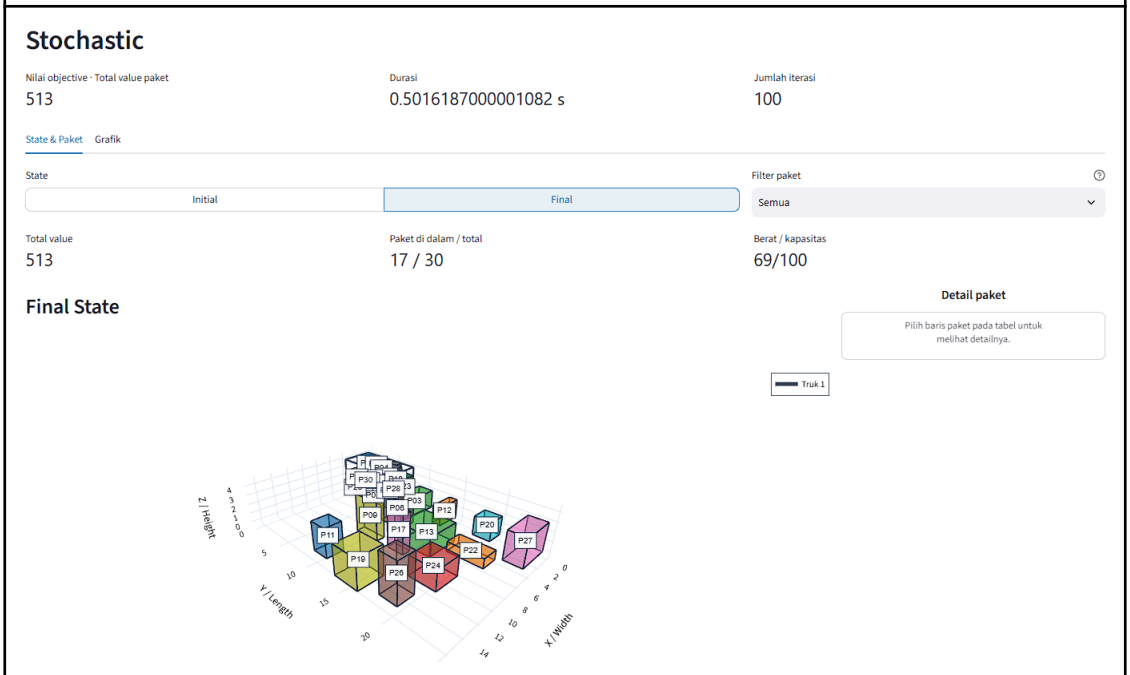


12. Pengujian HC-S-03

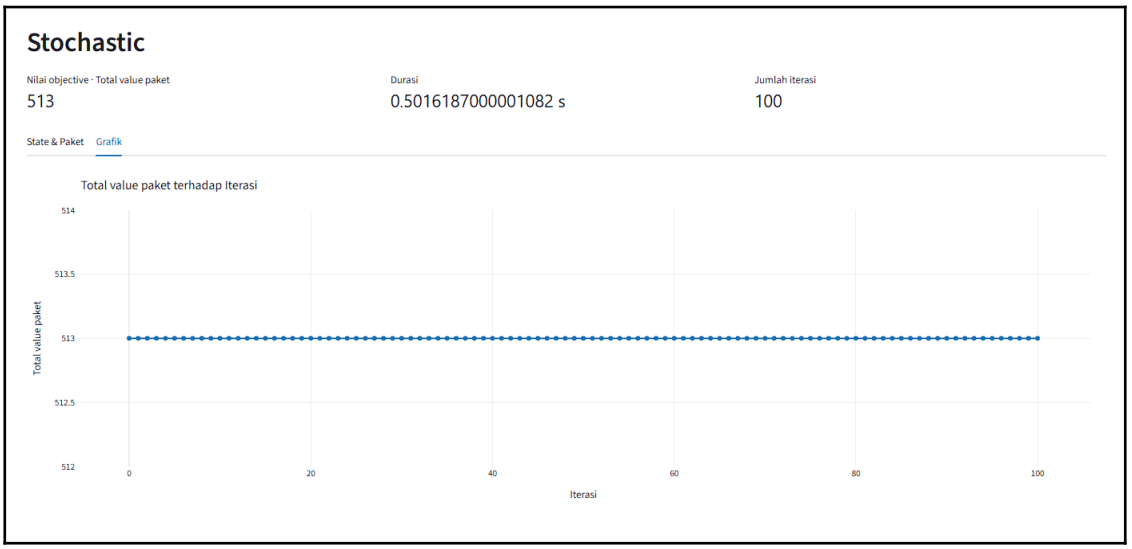
Initial State



Final State



Grafik Objektif



B. Simulated Annealing

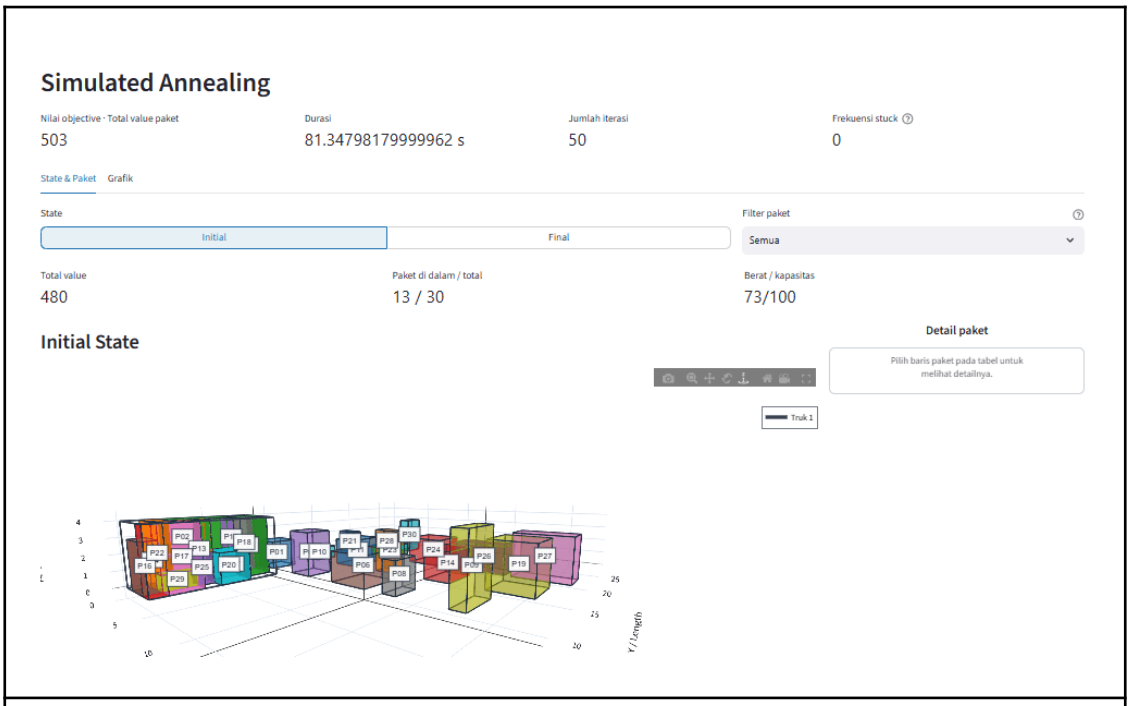
Pengujian algoritma Simulated Annealing dilakukan sebanyak tiga kali dengan konfigurasi parameter yang sama. Seed dibedakan pada setiap pengujian agar hasil dari beberapa proses pencarian dapat dibandingkan. Fungsi objektif yang digunakan adalah value, yaitu jumlah nilai paket yang berhasil dimuat ke dalam truk. Pengujian menggunakan satu truk.

Tabel 3.5 Variasi Parameter Pengujian Simulated Annealing

Parameter	Pengujian 1	Pengujian 2	Pengujian 3
Seed	42	43	44
Temperatur Awal	100	100	100
Cooling rate	0.95	0.95	0.95
Maksimum iterasi	50	50	50
Fungsi objektif	value	value	value
Jumlah truk	1	1	1

1. Pengujian SA-01

Initial State



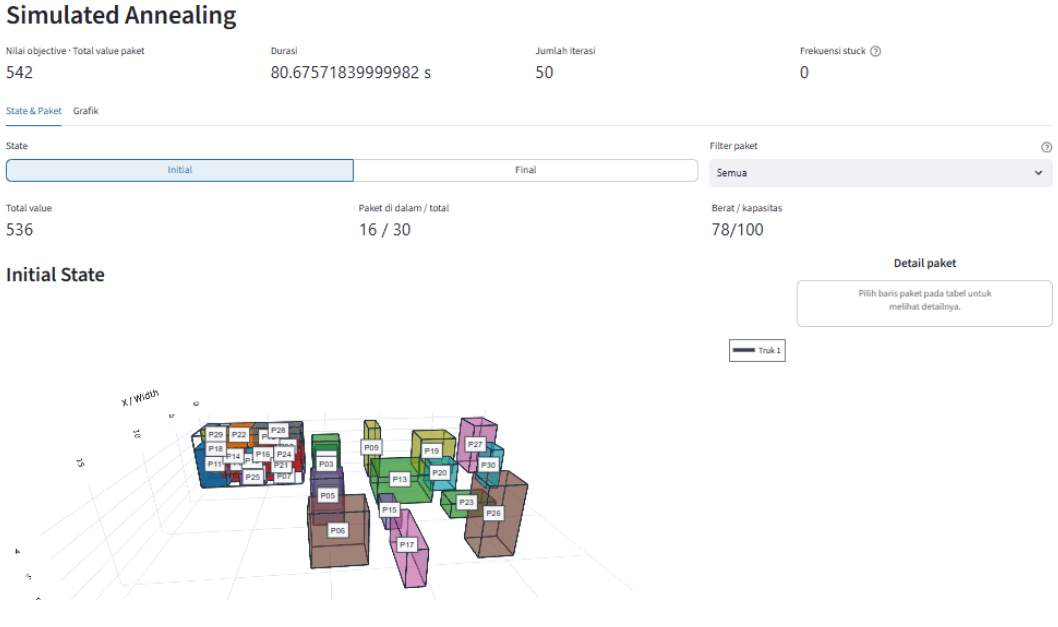
Final State



Grafik Objektif

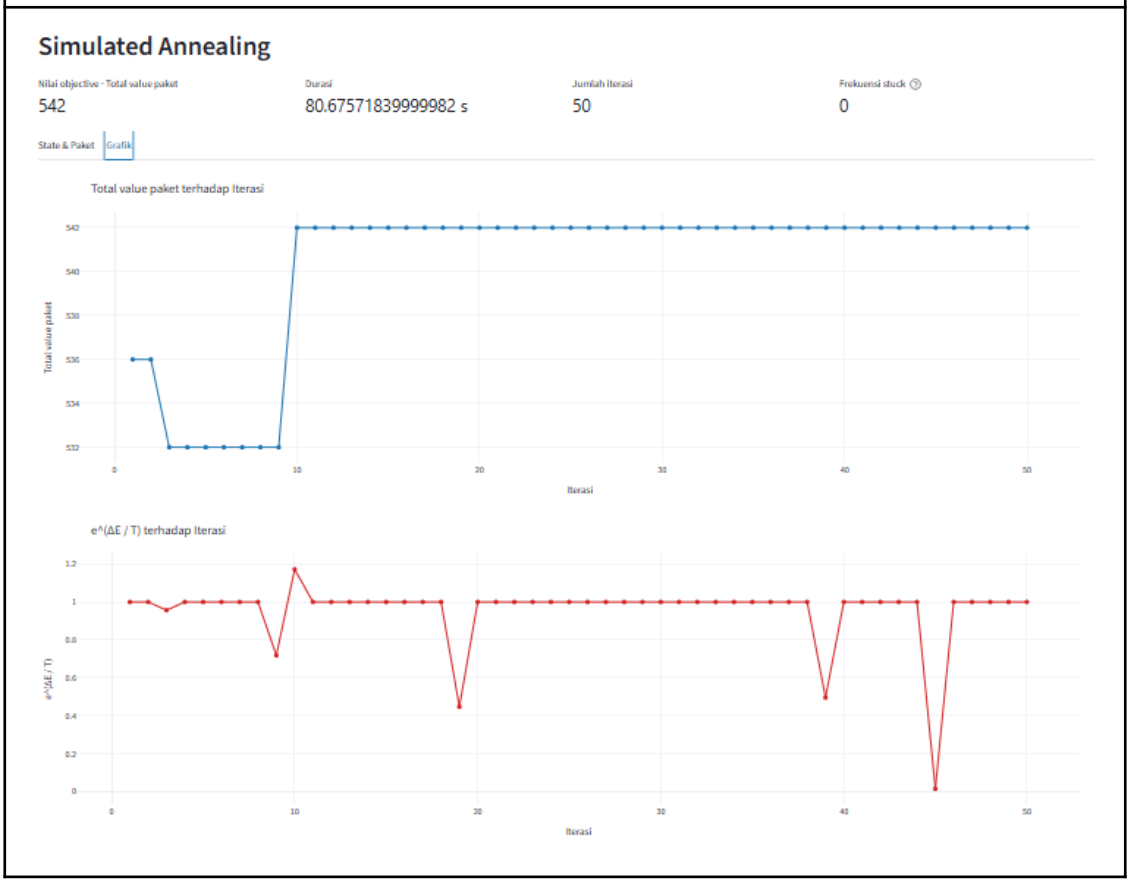


2. Pengujian SA-02

Initial State

Final State

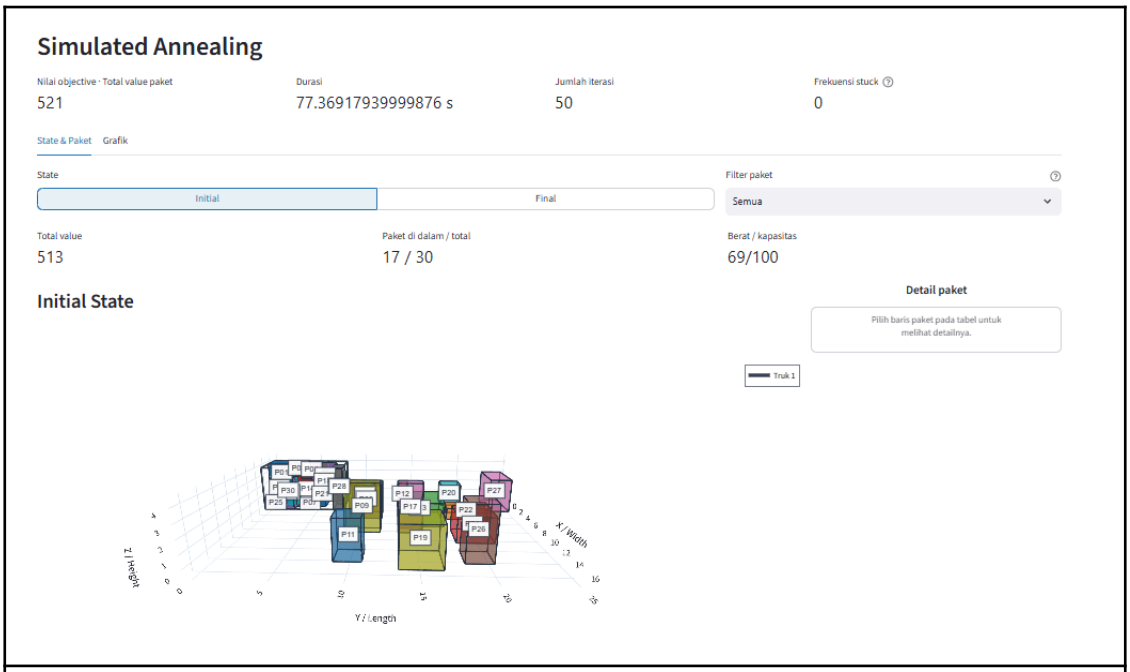


Grafik Objektif

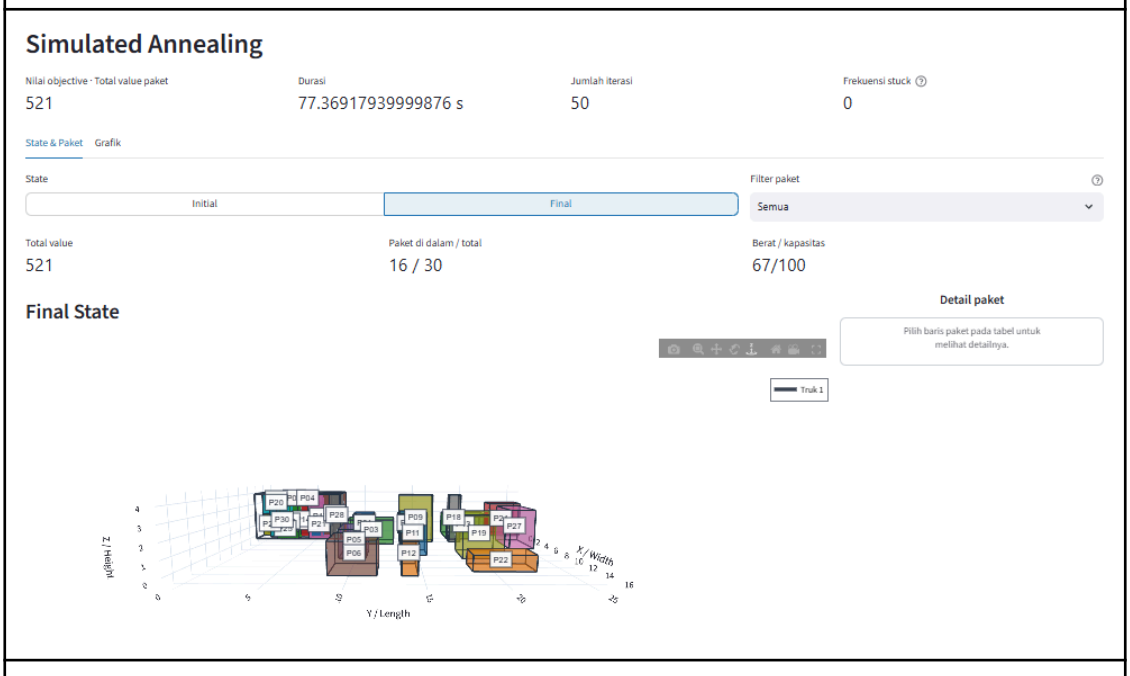


3. Pengujian SA-03

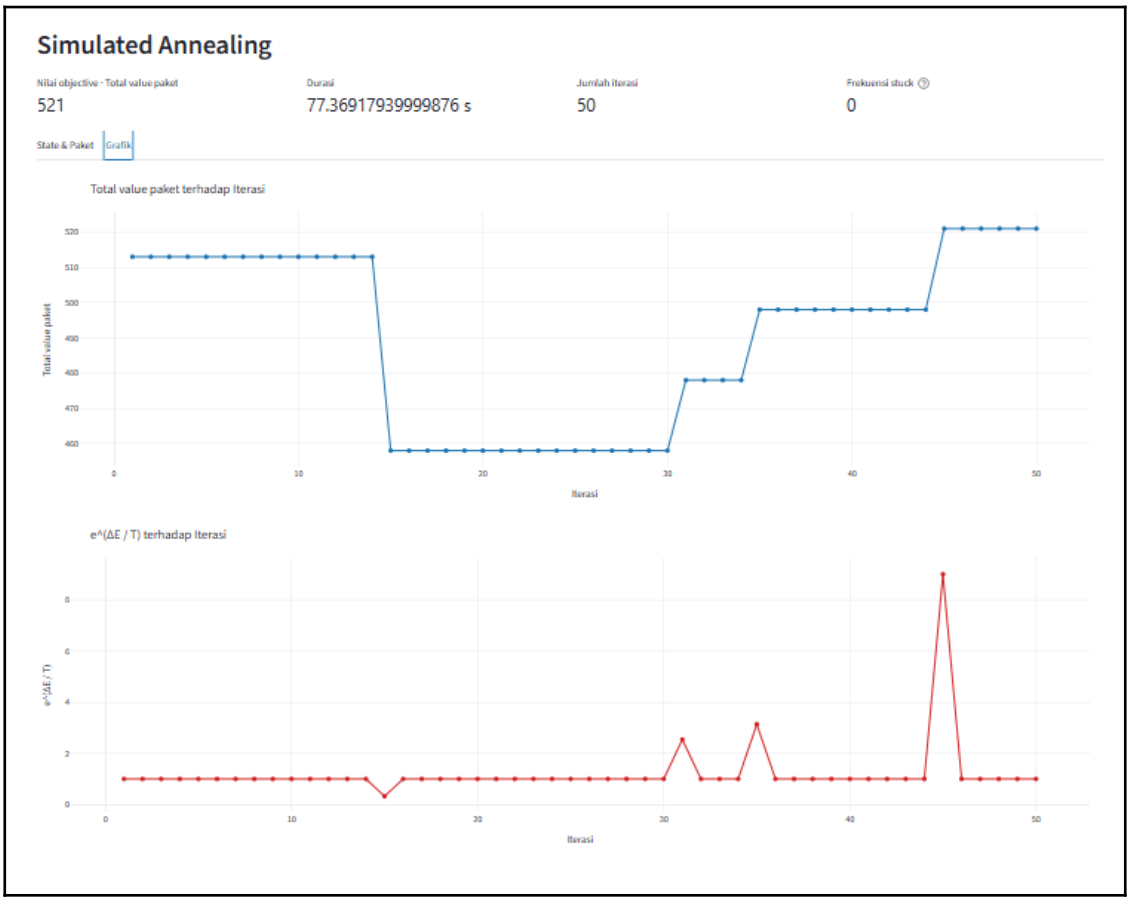
Initial State



Final State



Grafik Objektif



Pada tiga pengujian Simulated Annealing, nilai objective terbaik yang diperoleh berturut-turut adalah 503, 542, dan 521, dengan rata-rata 522. Setiap pengujian mencapai batas 50 iterasi dan frekuensi *stuck* tercatat 0. Durasi pencarian berkisar antara 77,37 hingga 81,35 detik, dengan rata-rata sekitar 79,80 detik. Nilai objective terbaik diperoleh pada pengujian kedua.

Grafik objective menunjukkan nilai state yang sedang dijelajahi, sedangkan nilai pada tabel merupakan state terbaik yang ditemukan. Pada pengujian pertama, misalnya, grafik berakhir di nilai 472, sementara solusi terbaiknya bernilai 503. Perbedaan ini terjadi karena Simulated Annealing dapat menerima perpindahan ke state yang lebih buruk selama pencarian.

C. Genetic Algorithm

Pengujian Genetic Algorithm dilakukan sebanyak tiga kali untuk setiap variasi dengan konfigurasi parameter yang sama. Seed dibedakan pada setiap pengujian agar hasil dari beberapa proses pencarian dapat dibandingkan. Fungsi objektif yang digunakan

adalah value, yaitu jumlah nilai paket yang berhasil dimuat ke dalam truk. Pengujian menggunakan satu truk.

Tabel 3.6 Parameter Pengujian GA dengan Kontrol Populasi

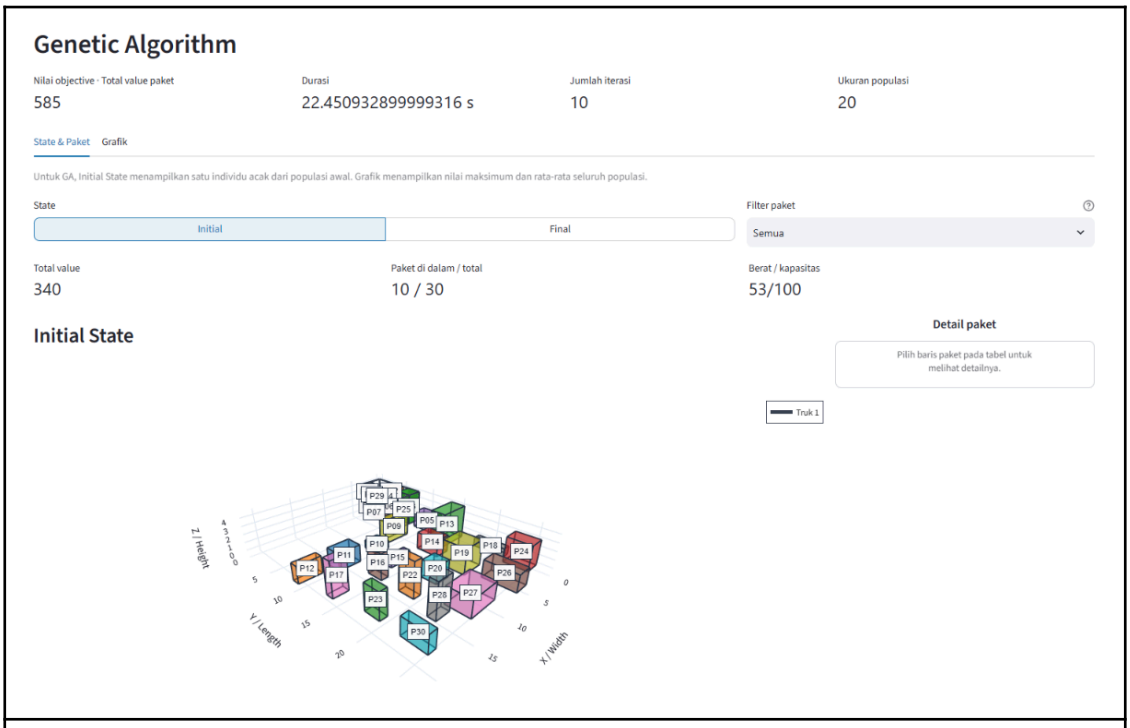
ID Variasi	Jumlah Populasi	Jumlah Iterasi
GA-01	20	10
GA-02	20	20
GA-03	20	40

Tabel 3.7 Parameter Pengujian GA dengan Kontrol Iterasi

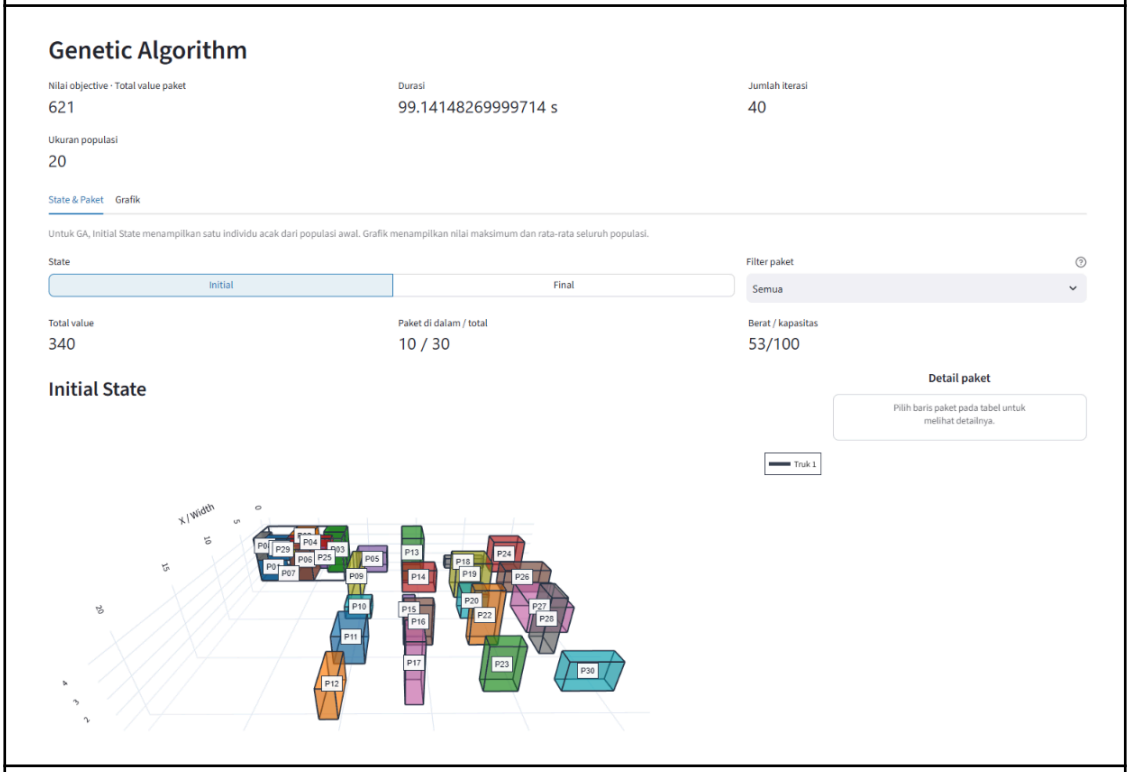
ID Variasi	Jumlah Populasi	Jumlah Iterasi
GA-04	8	20
GA-05	32	20
GA-06	64	20

1. Pengujian GA-01

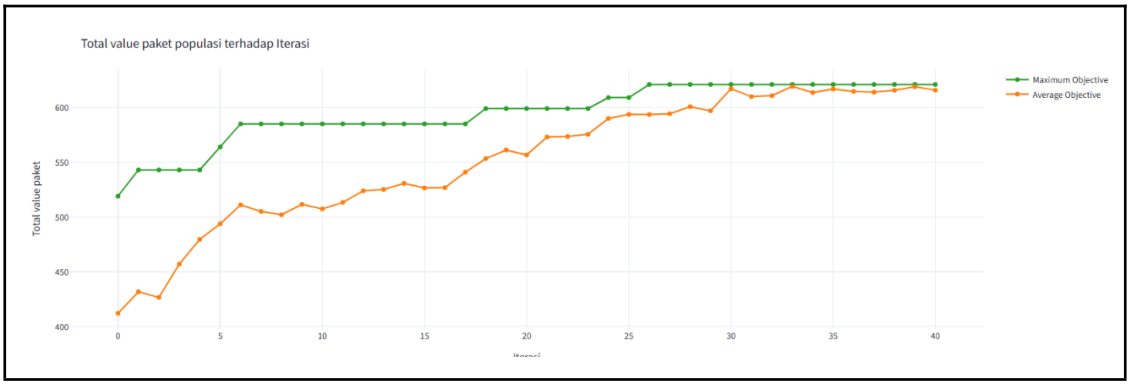
Iterasi Ke-1
Seed: 42
Initial State

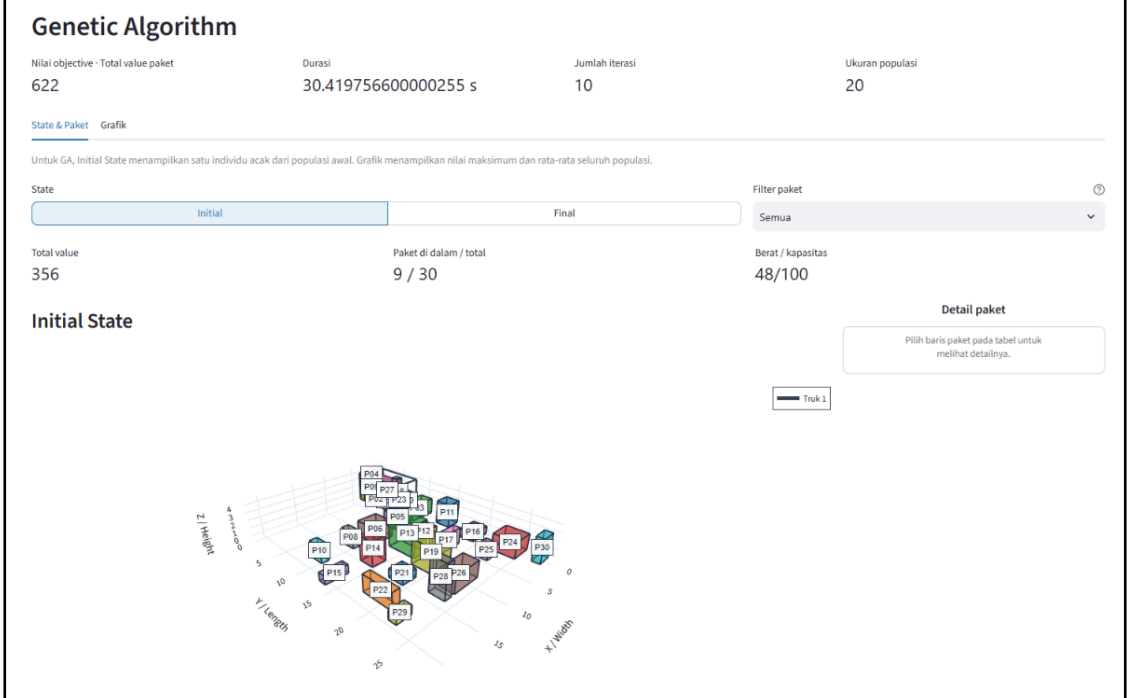


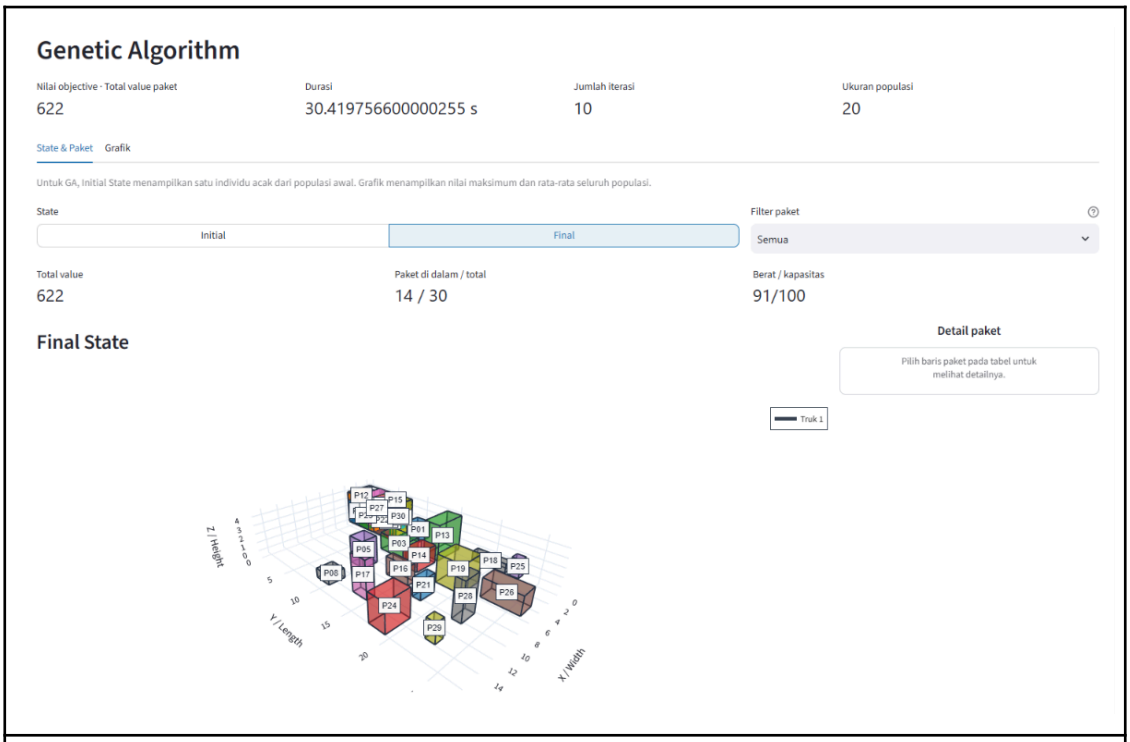
Final State



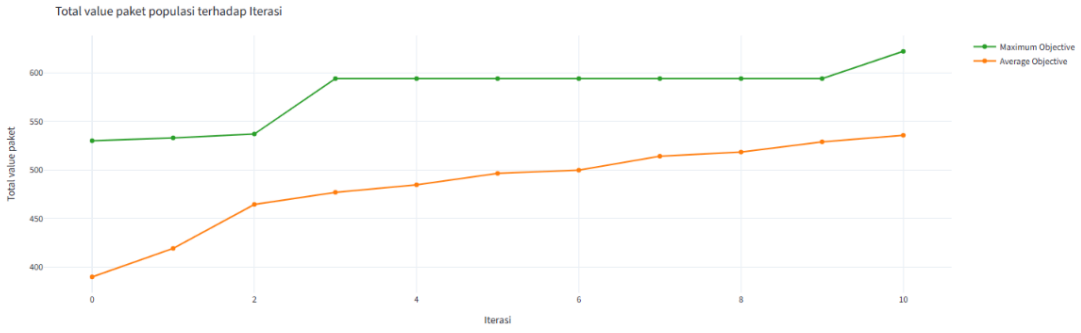
Grafik Objektif



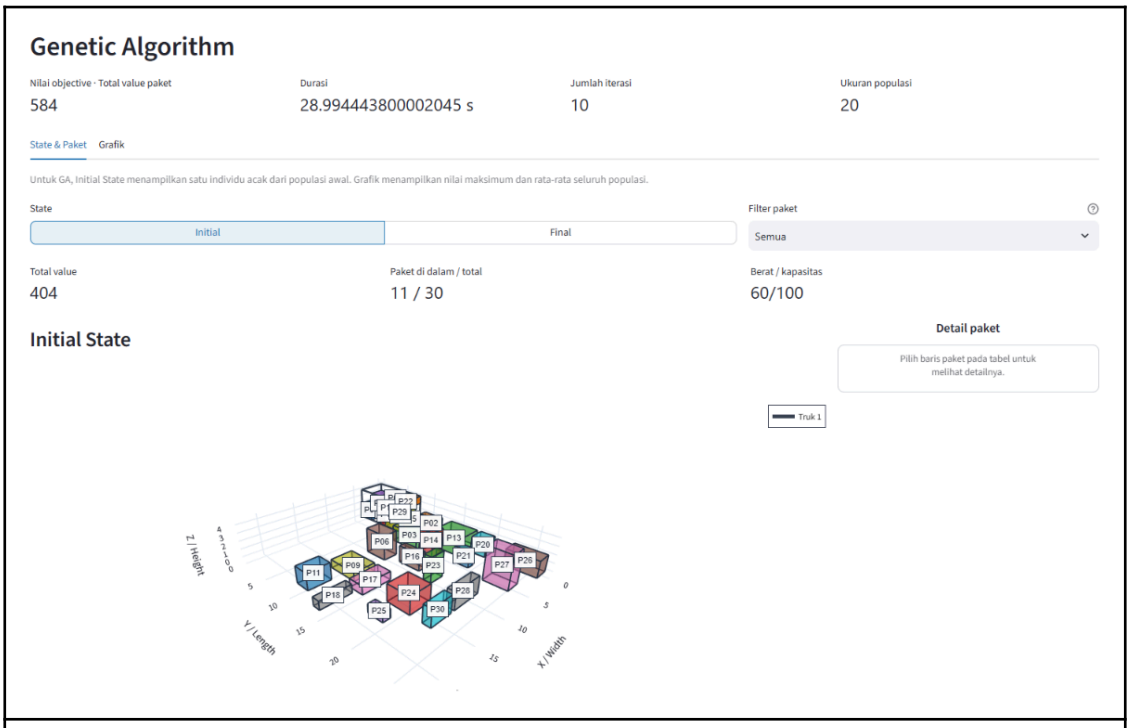
Iterasi Ke-2
Seed: 67
Initial State

Final State



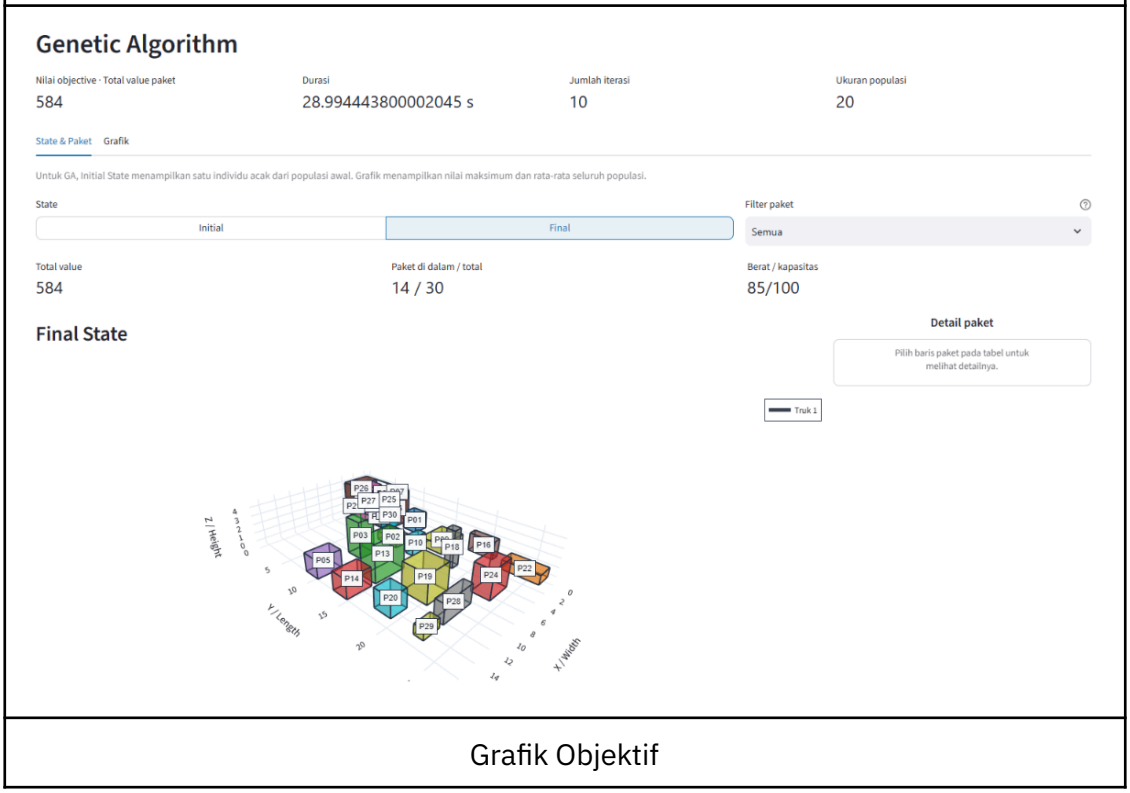
Grafik Objektif



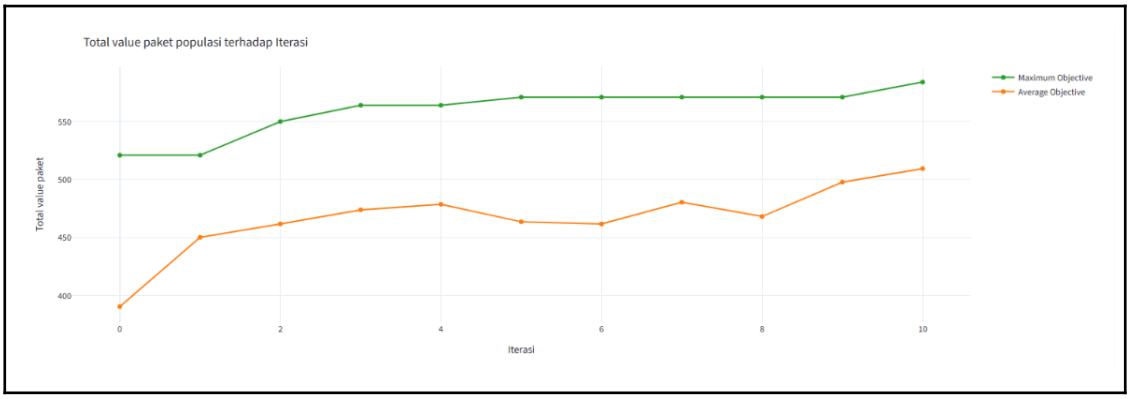
Iterasi Ke-3
Seed: 999
Initial State



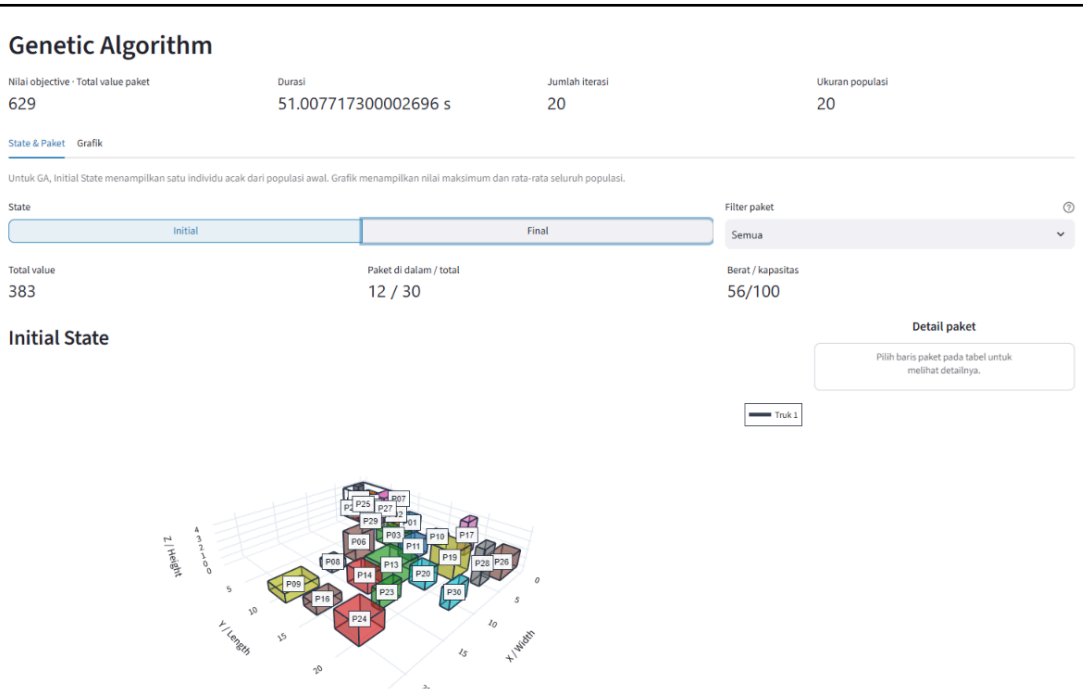
Final State

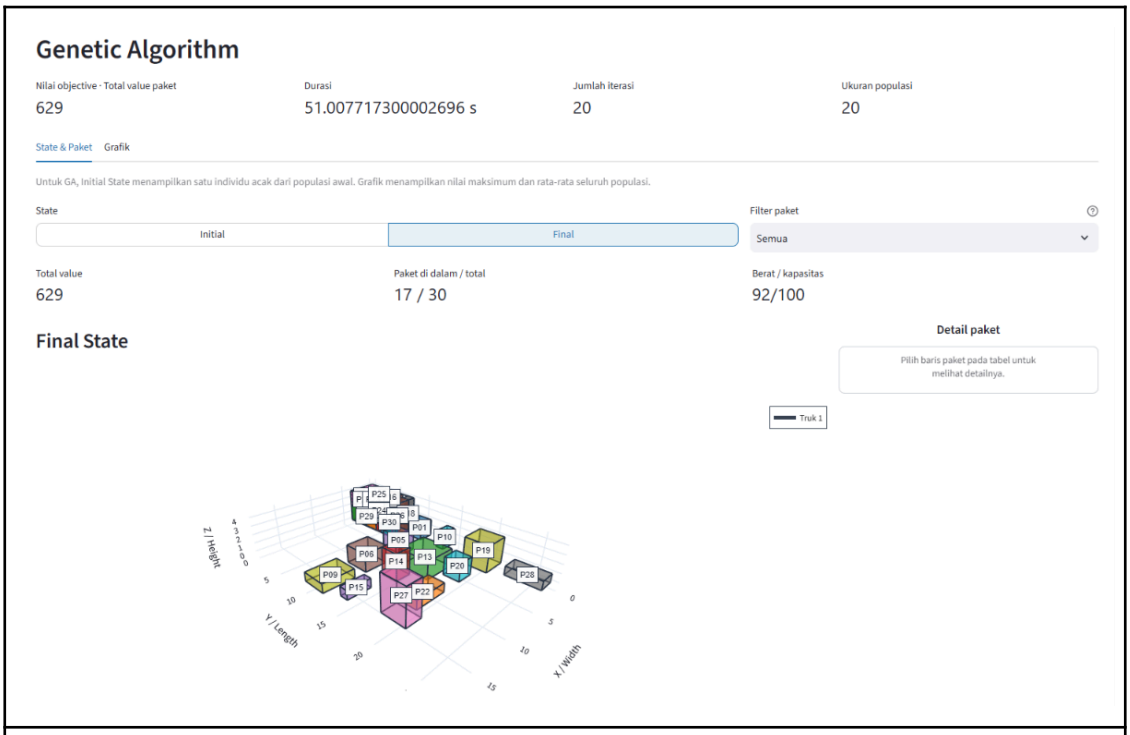


Grafik Objektif

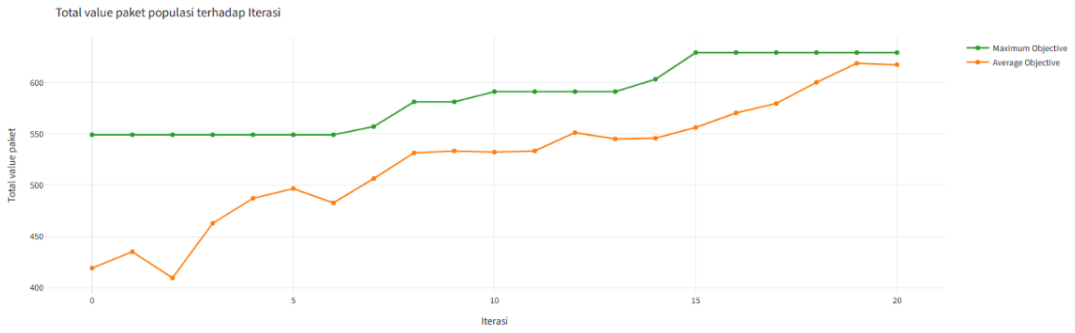


2. Pengujian GA-02

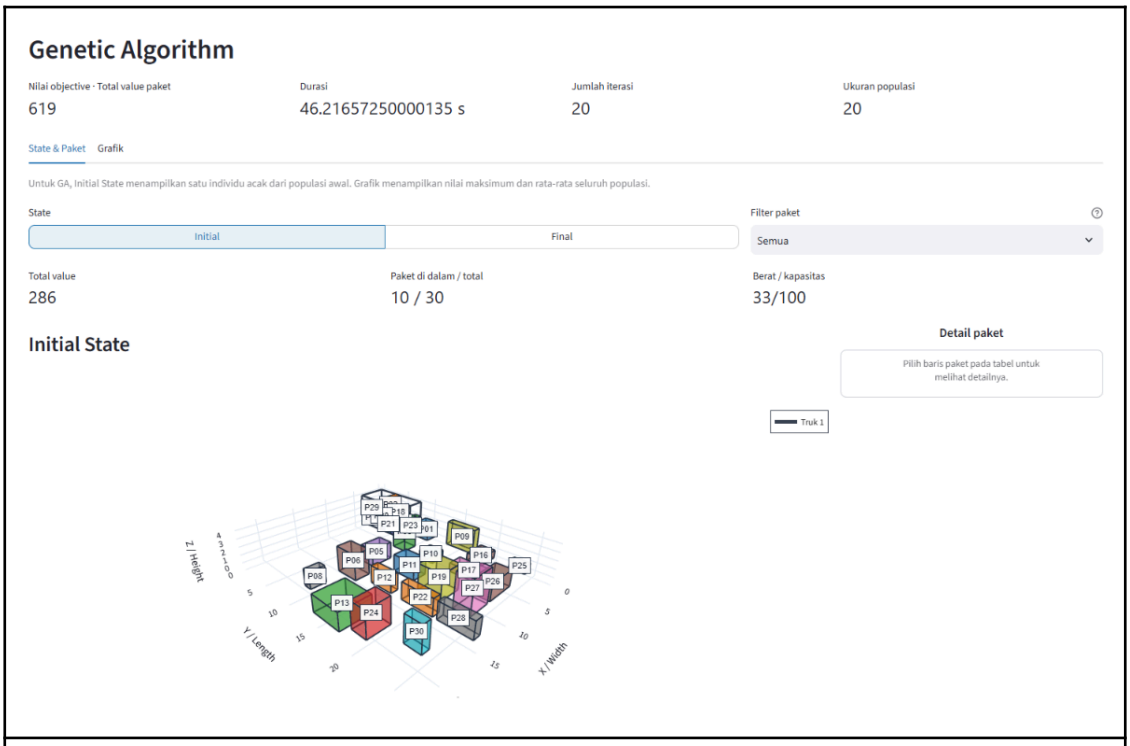
Iterasi Ke-1
Seed: 20
Initial State

Final State



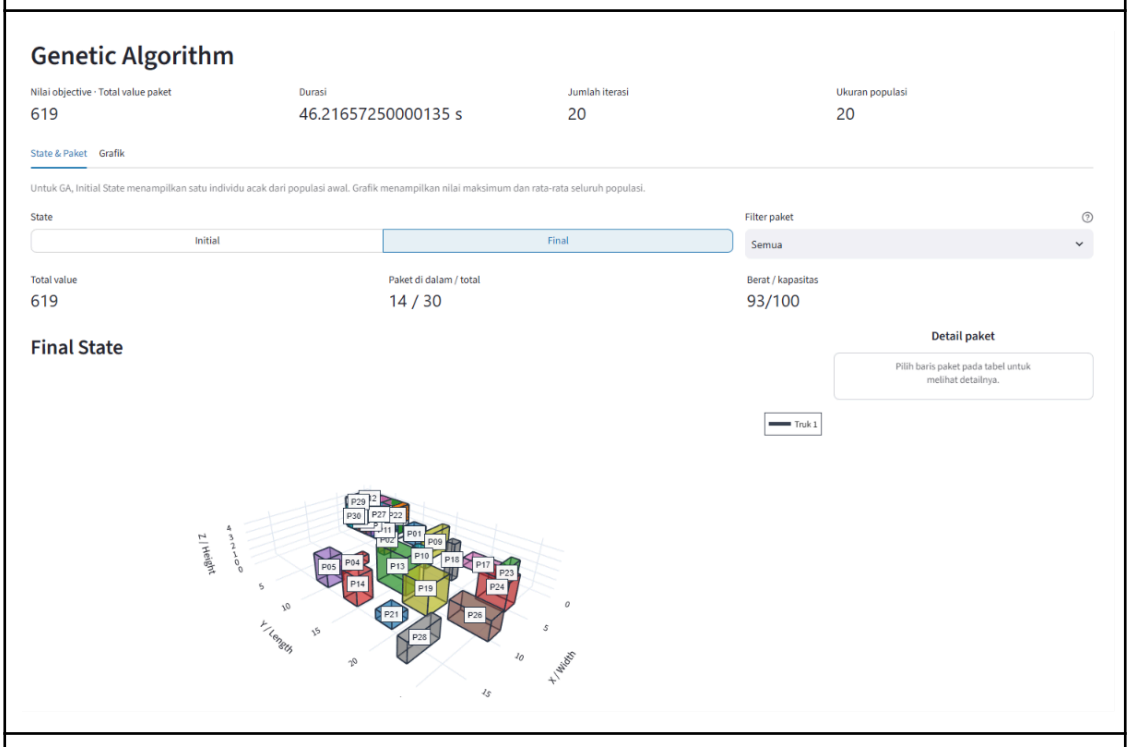
Grafik Objektif



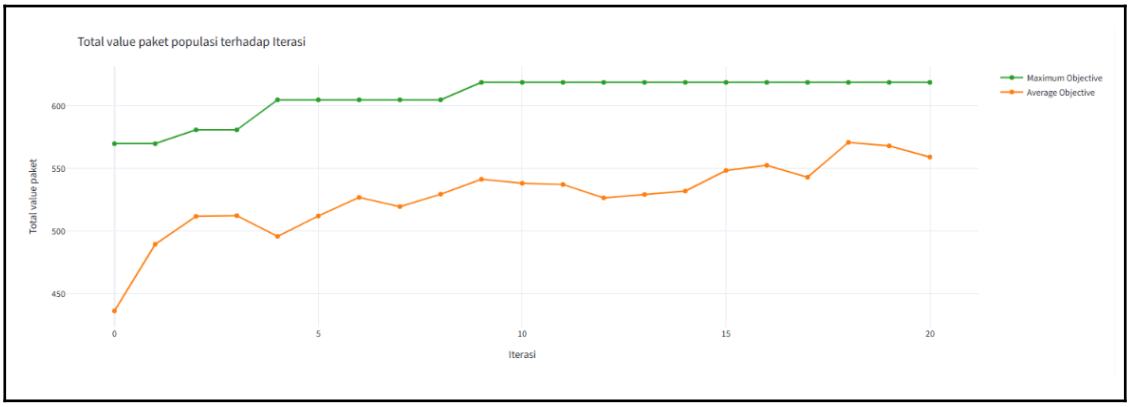
Iterasi Ke-2
Seed: 400
Initial State

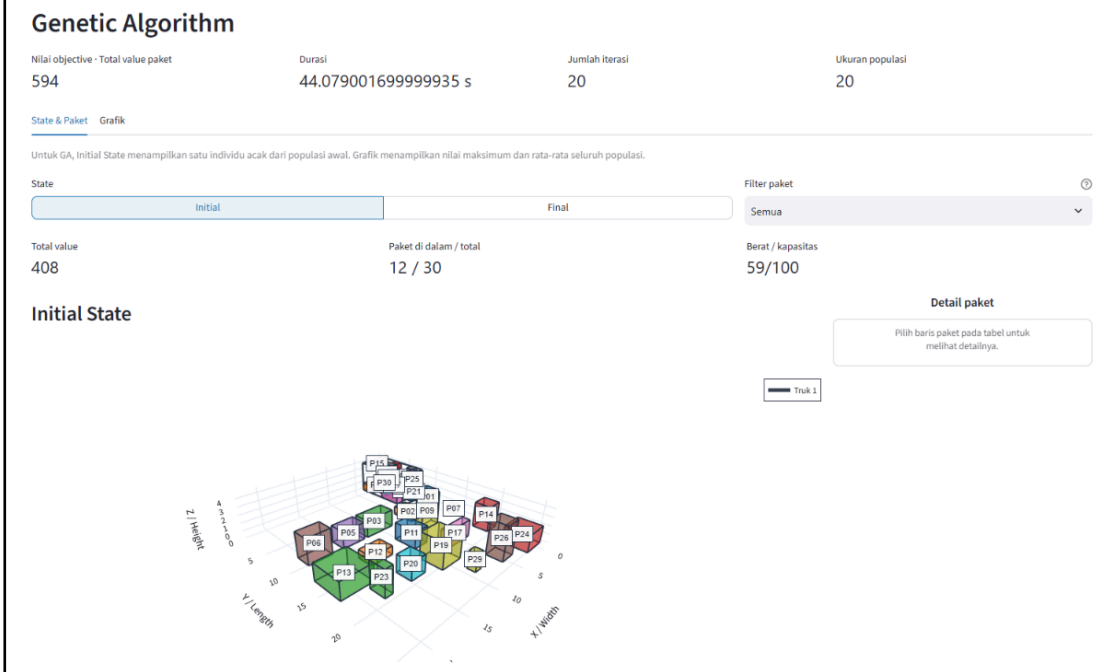


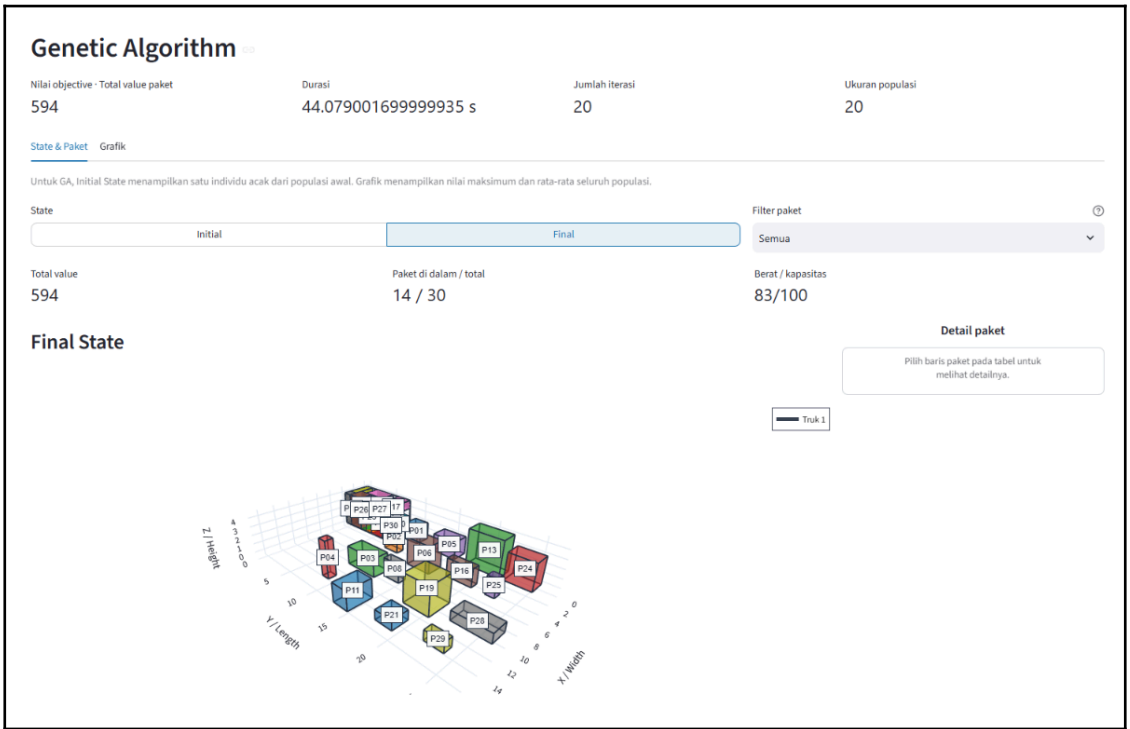
Final State



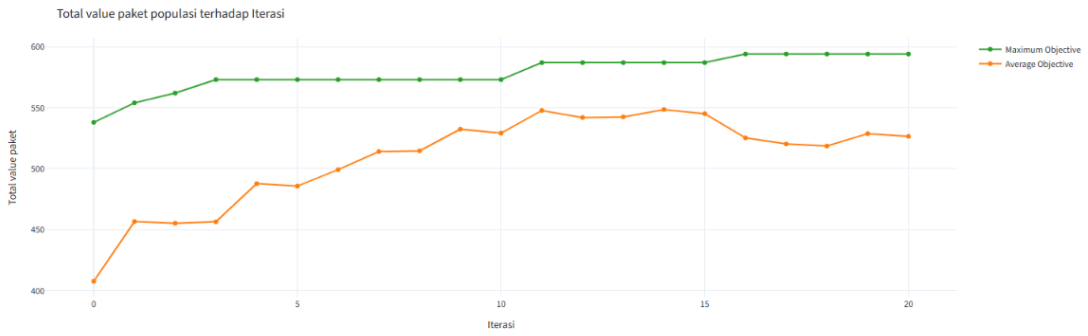
Grafik Objektif



Iterasi Ke-3
Seed: 313
Initial State

Final State

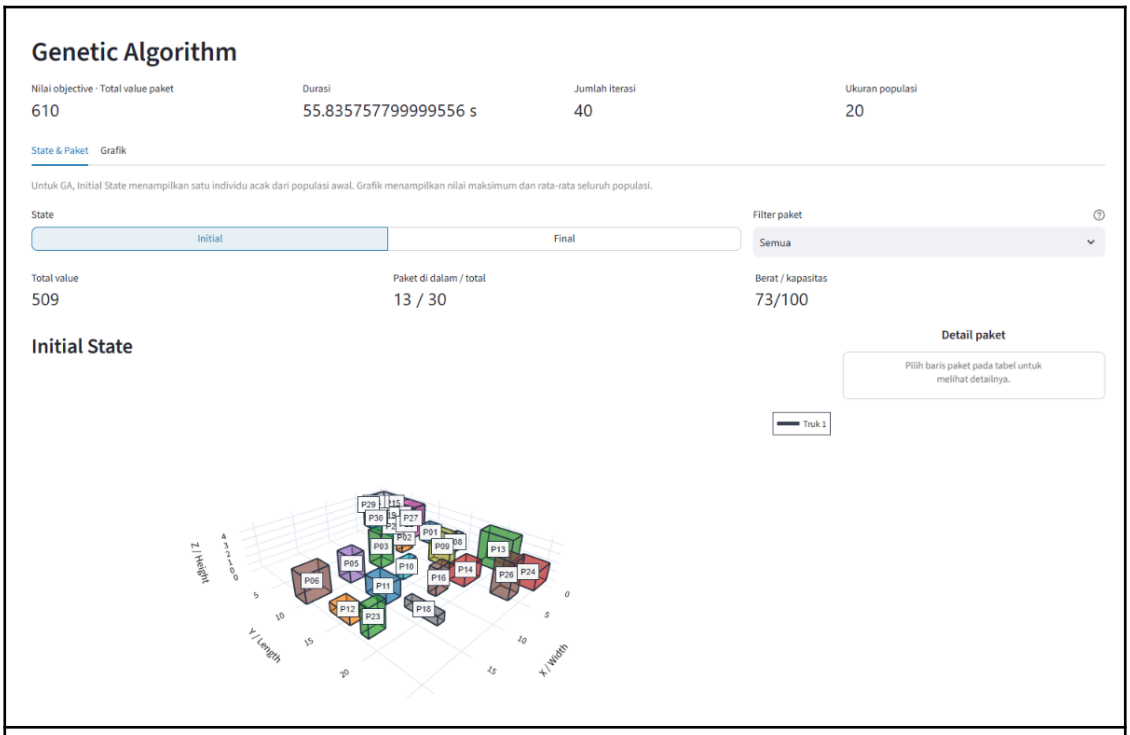


Grafik Objektif

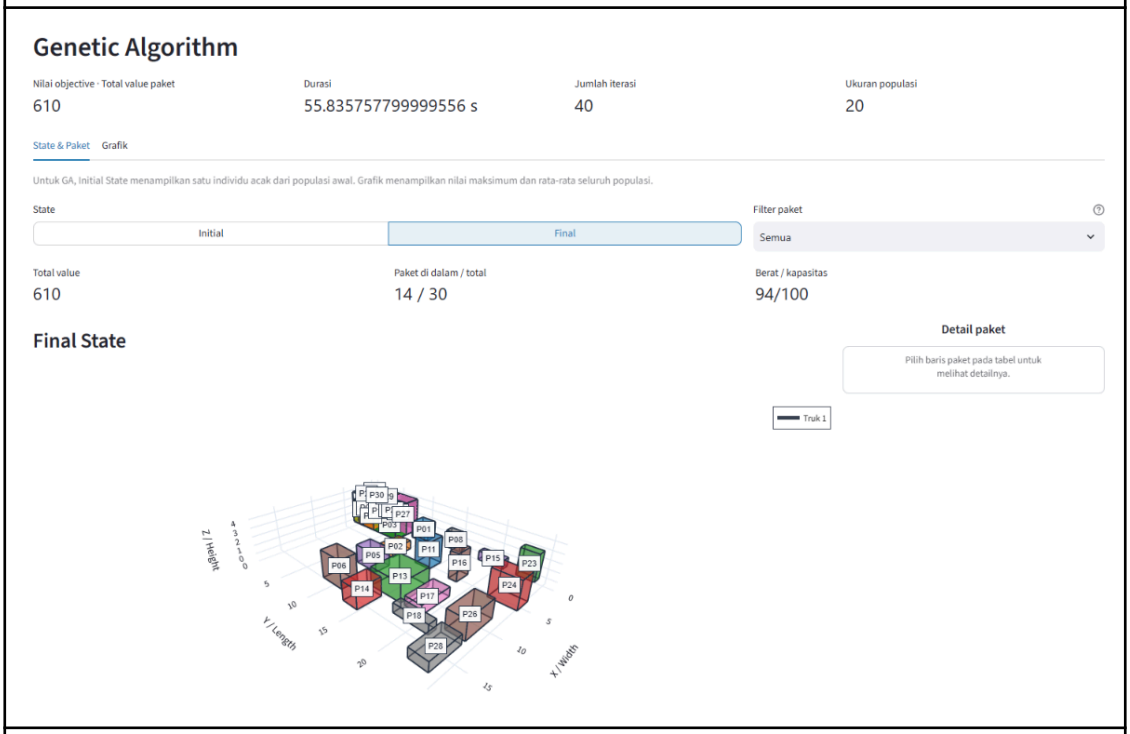


3. Pengujian GA-03

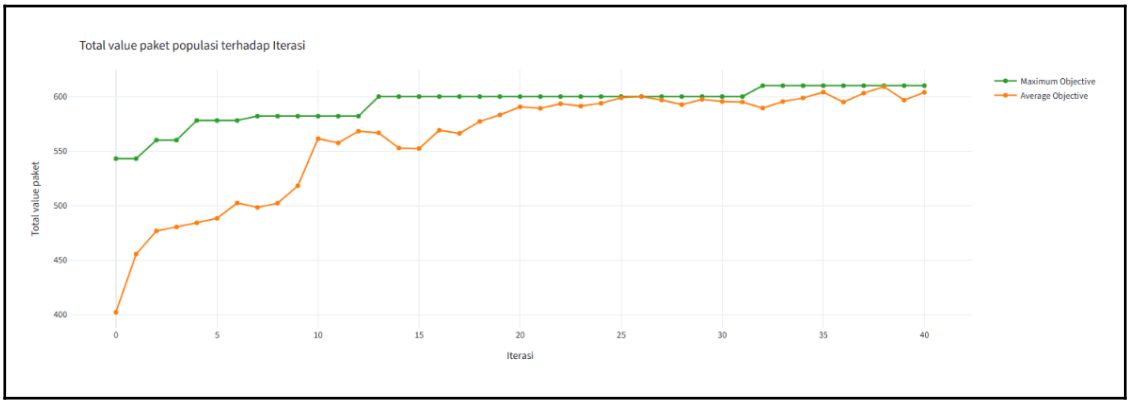
Iterasi Ke-1
Seed: 39
Initial State

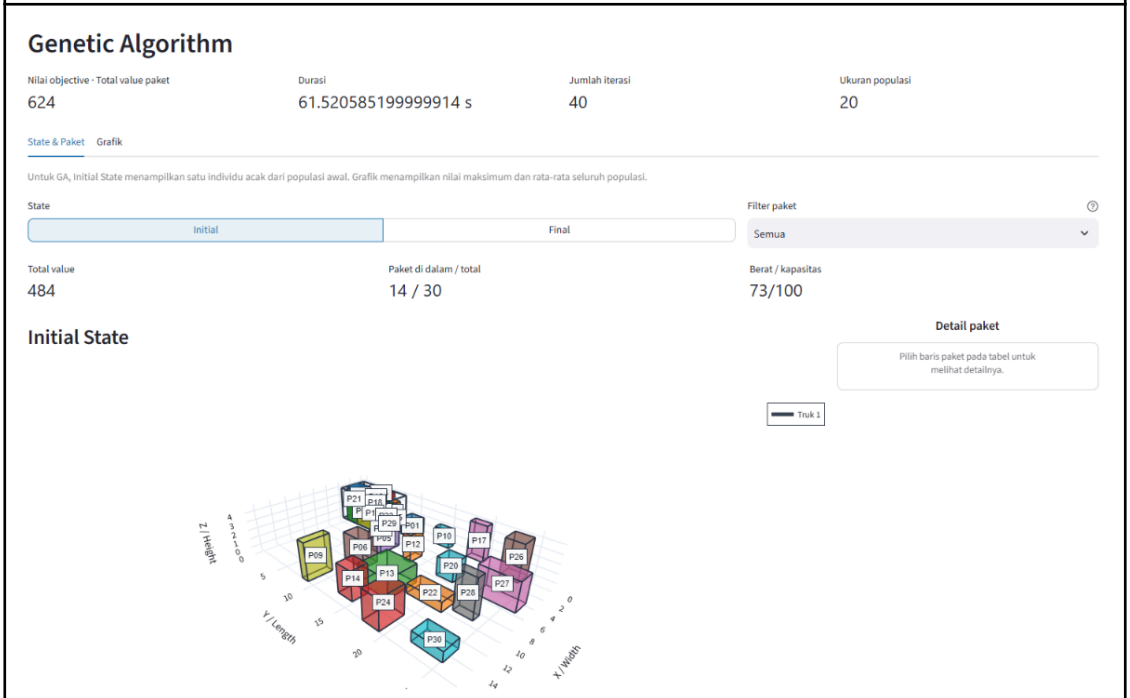


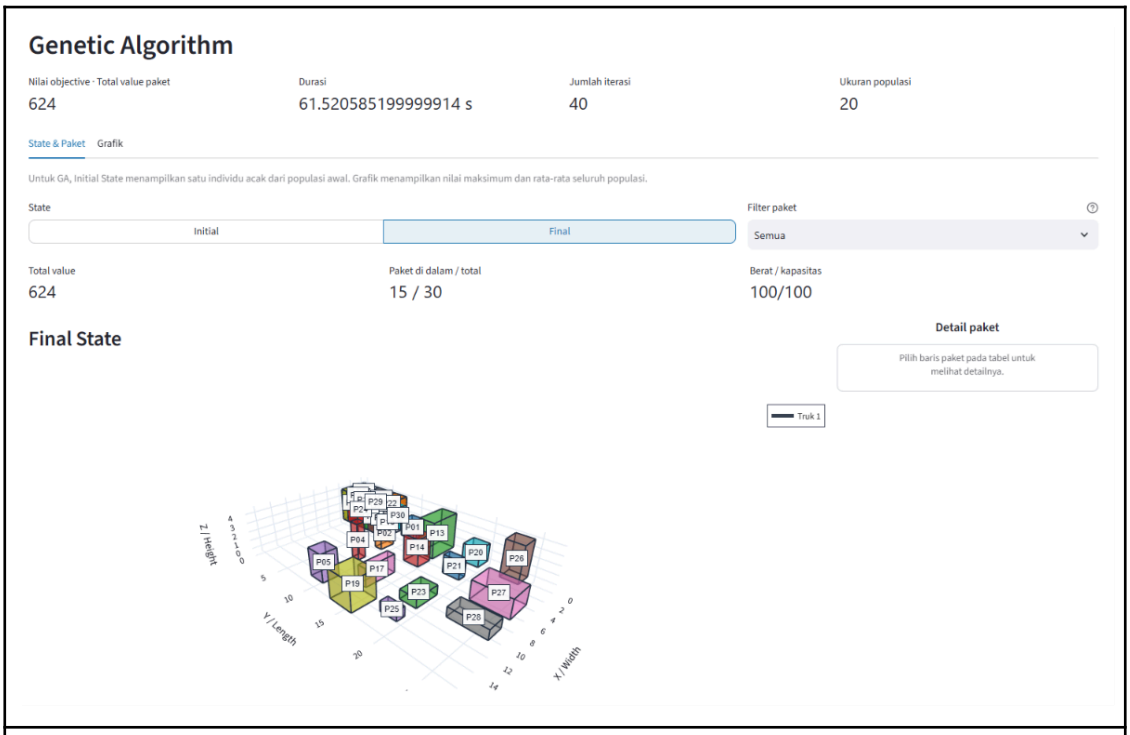
Final State



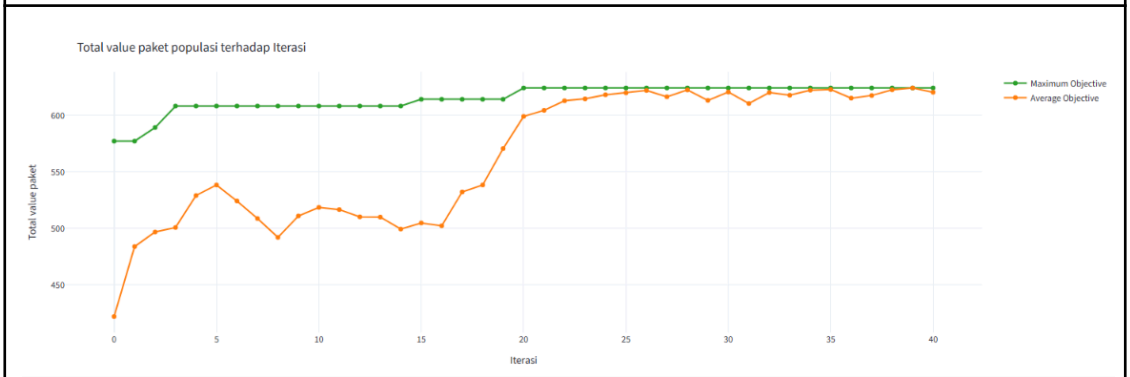
Grafik Objektif



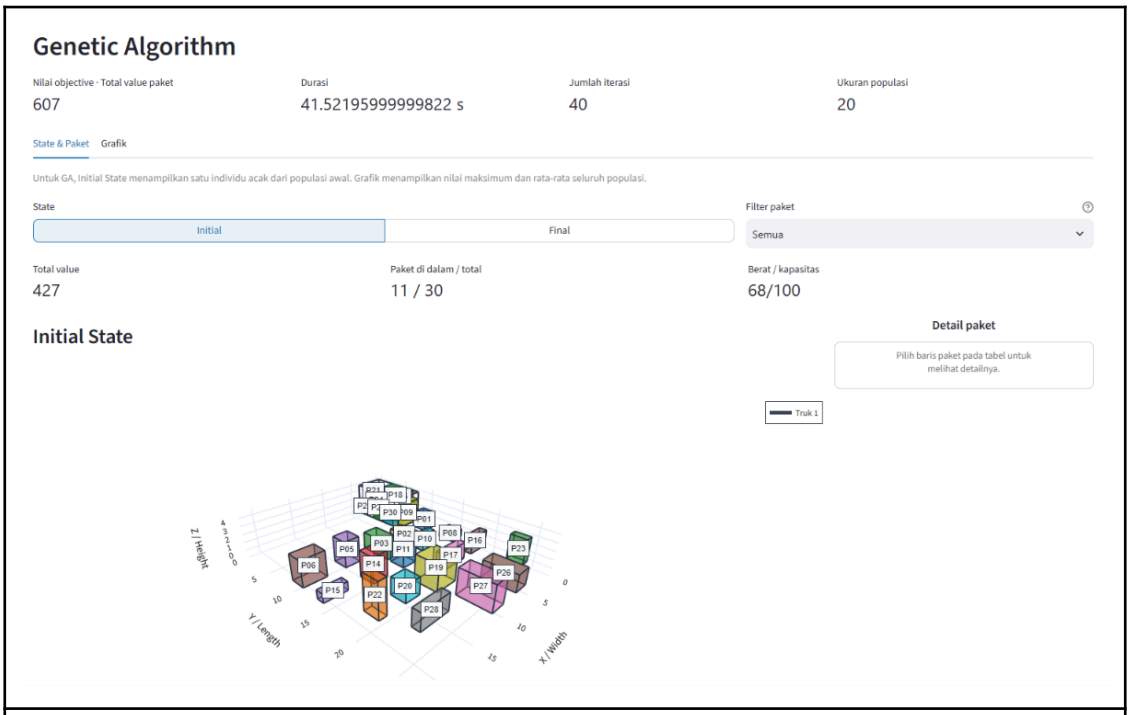
Iterasi Ke-2
Seed: 320
Initial State

Final State



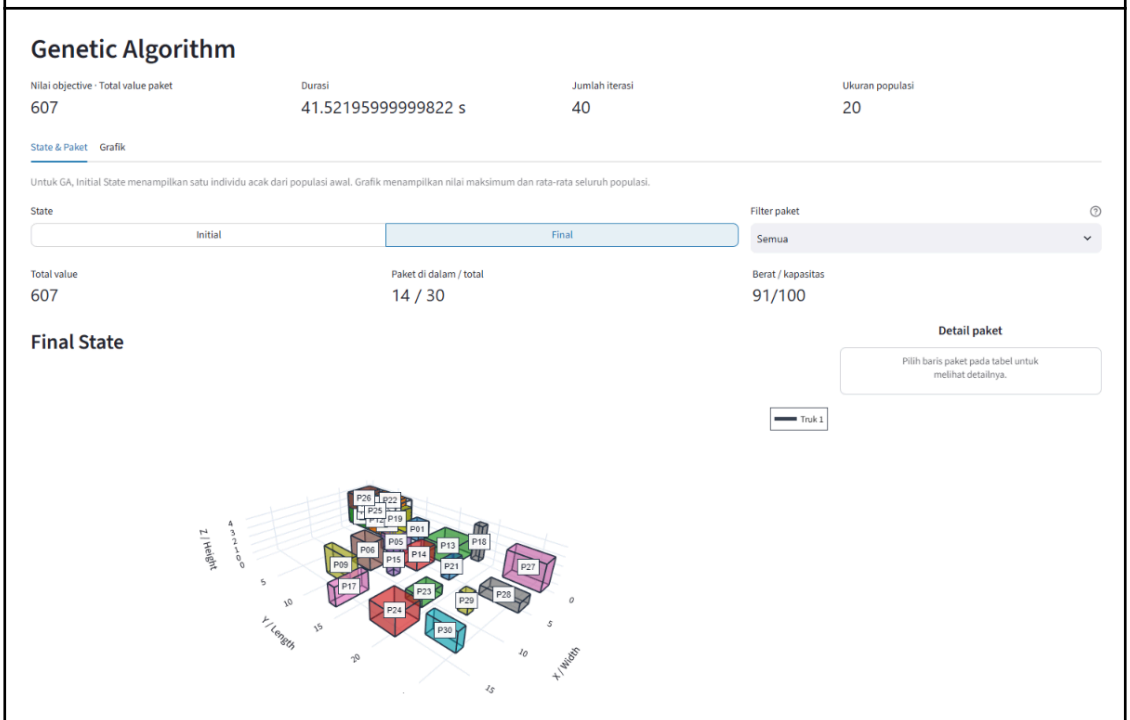
Grafik Objektif



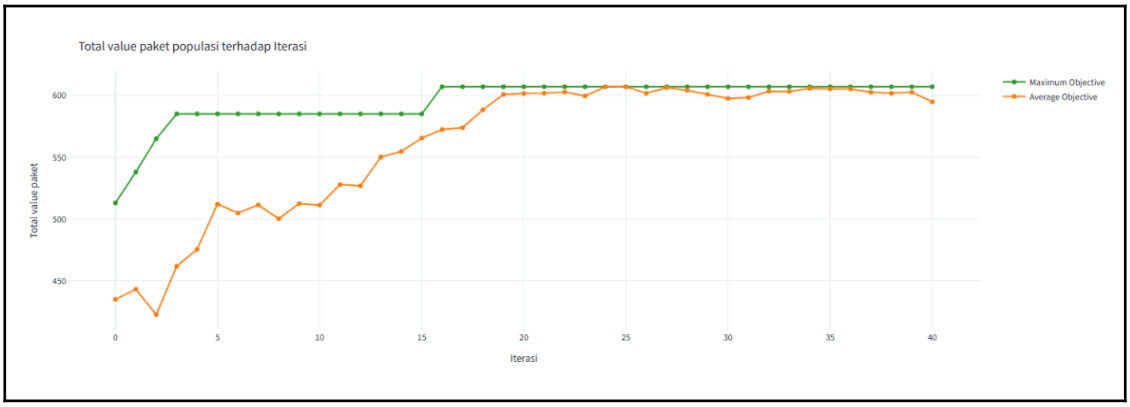
Iterasi Ke-3
Seed: 3840
Initial State



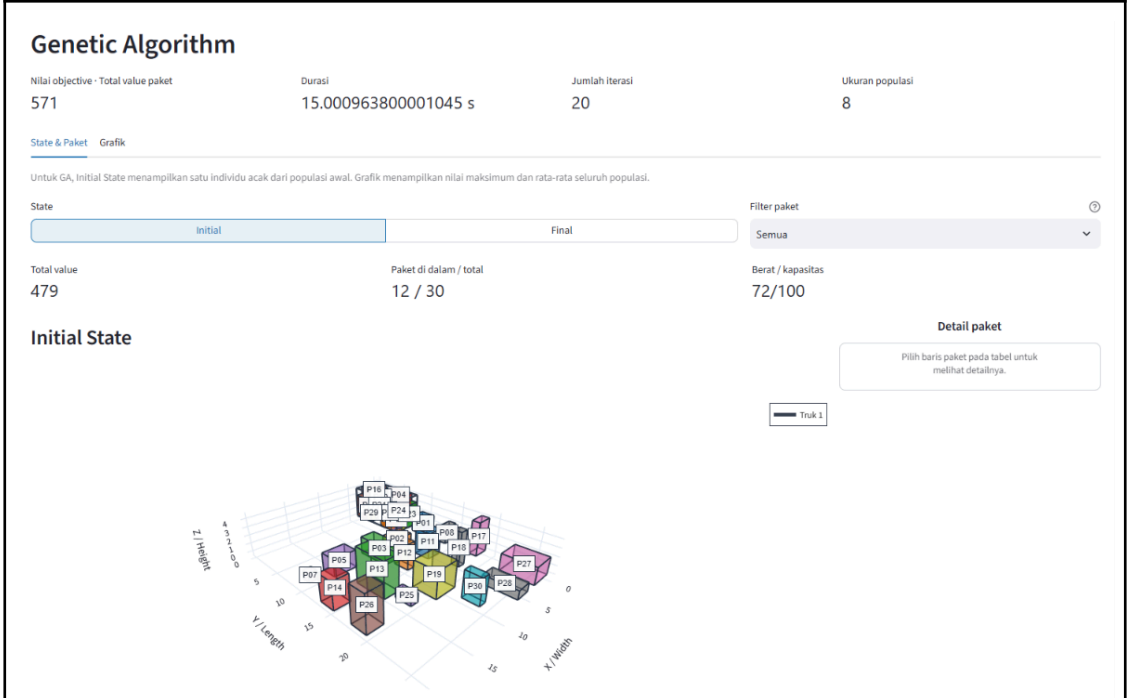
Final State

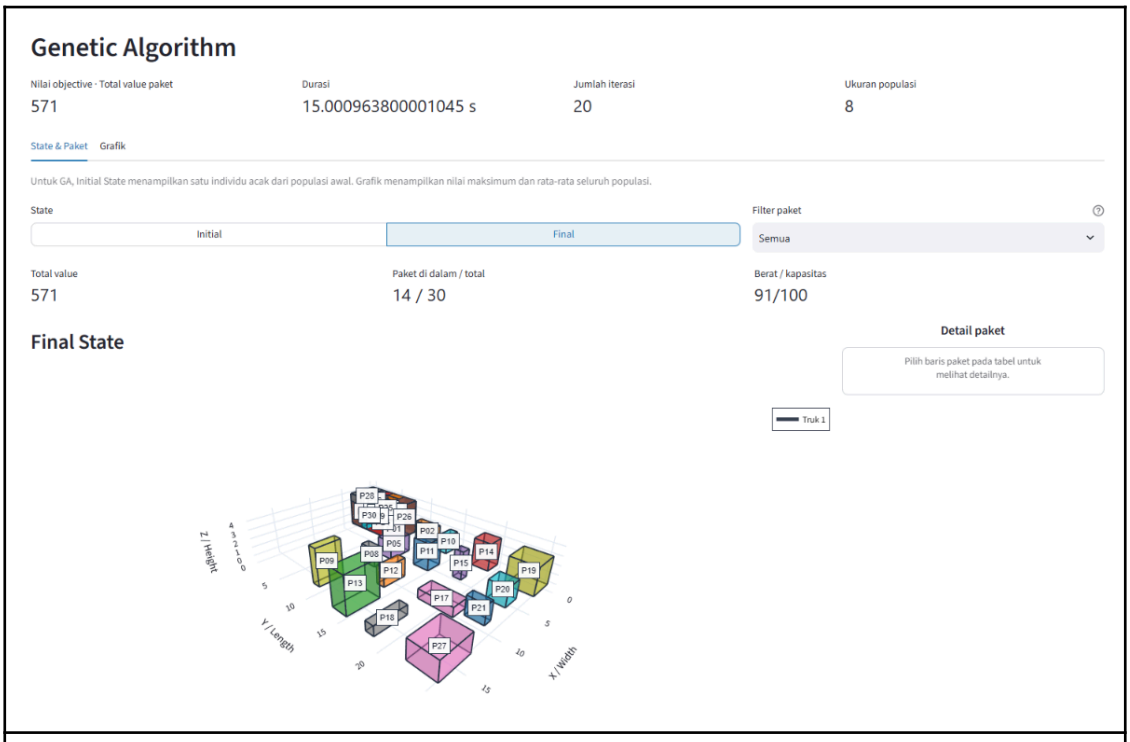


Grafik Objektif

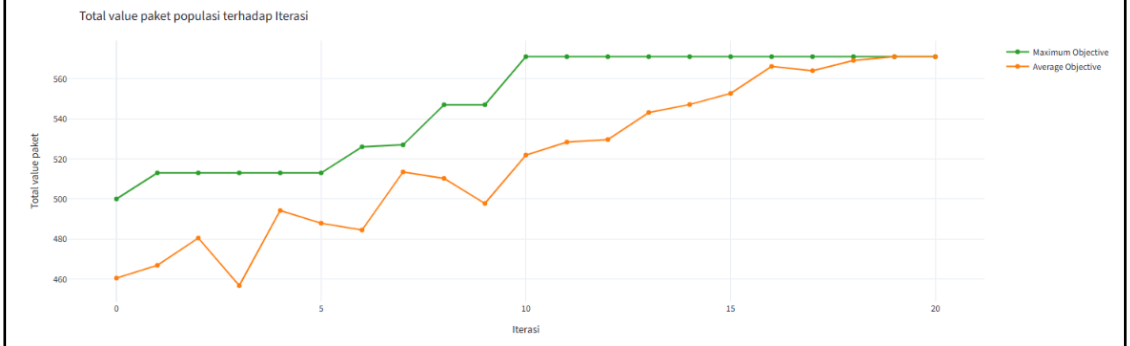


4. Pengujian GA-04

Iterasi Ke-1
Seed: 128
Initial State

Final State



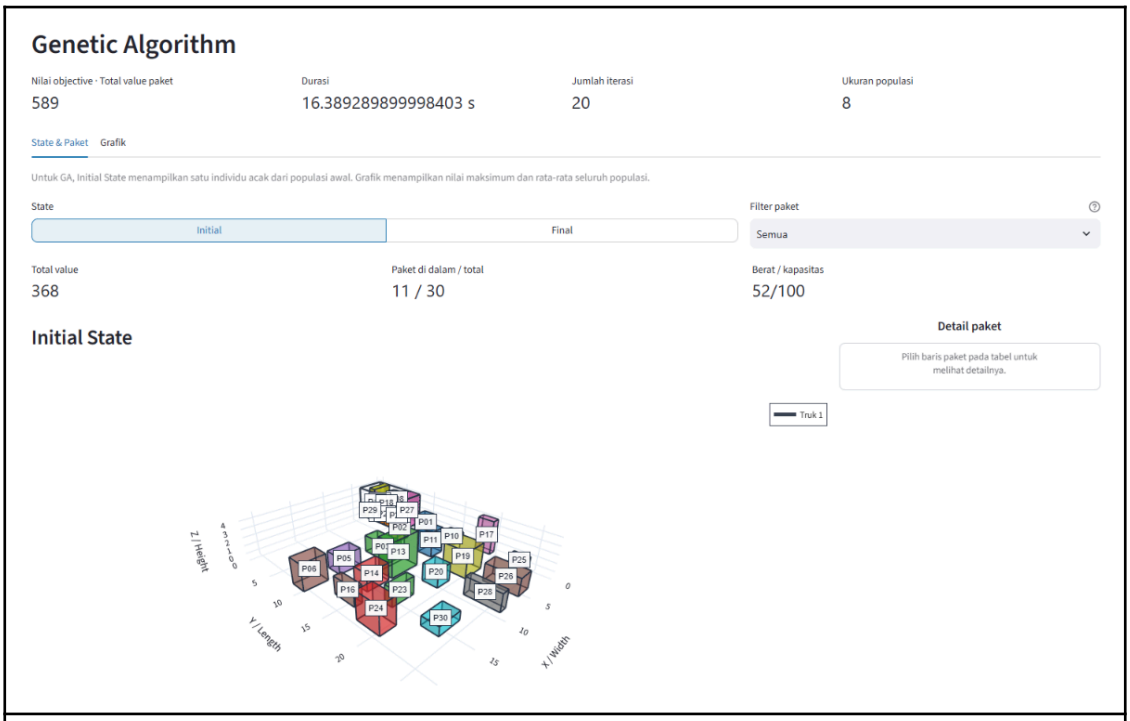
Grafik Objektif



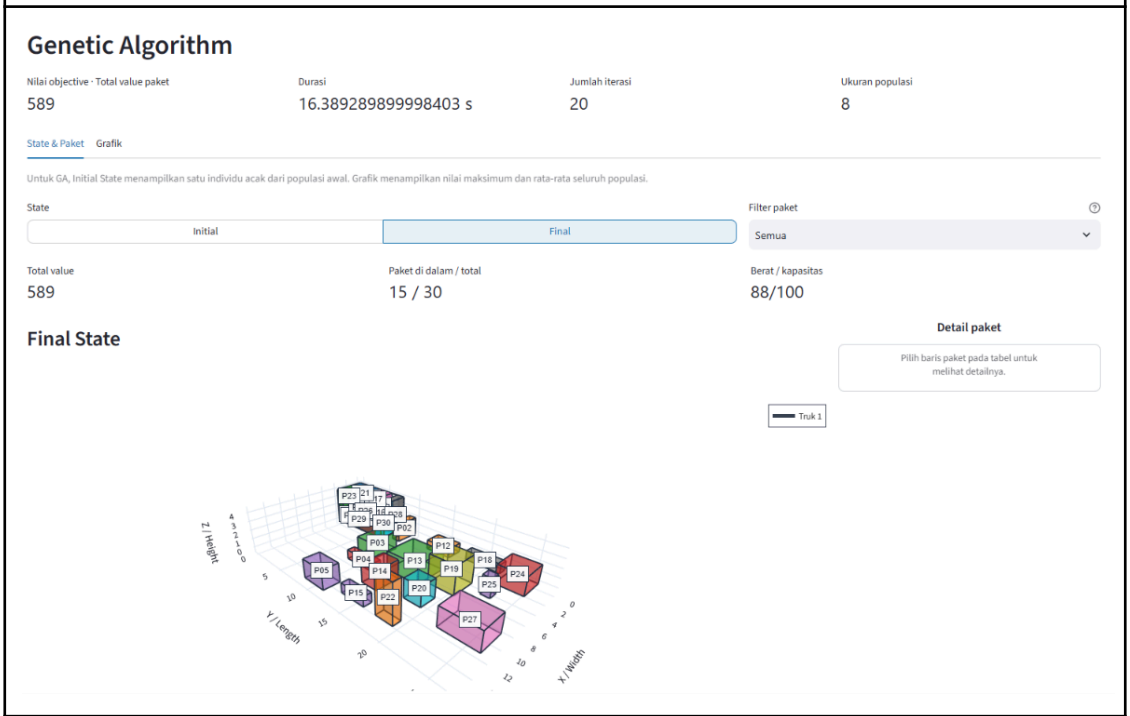
Iterasi Ke-2

Seed: 1280

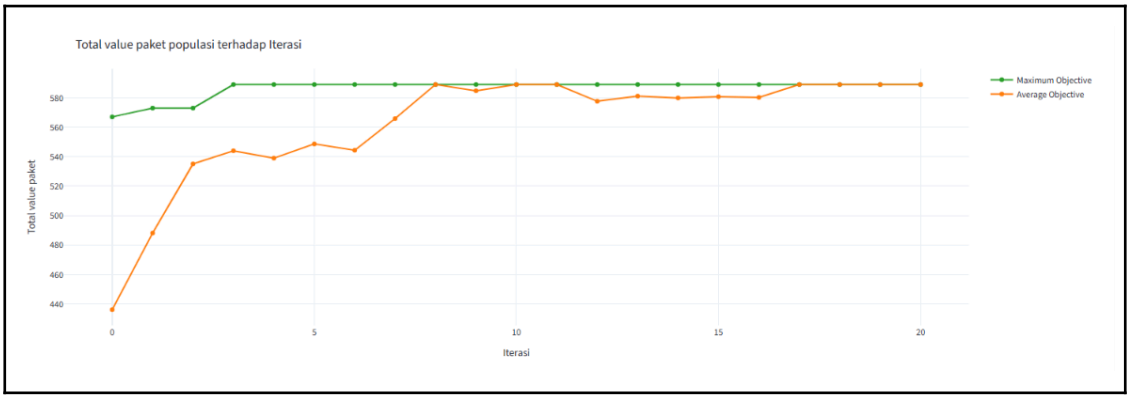
Initial State



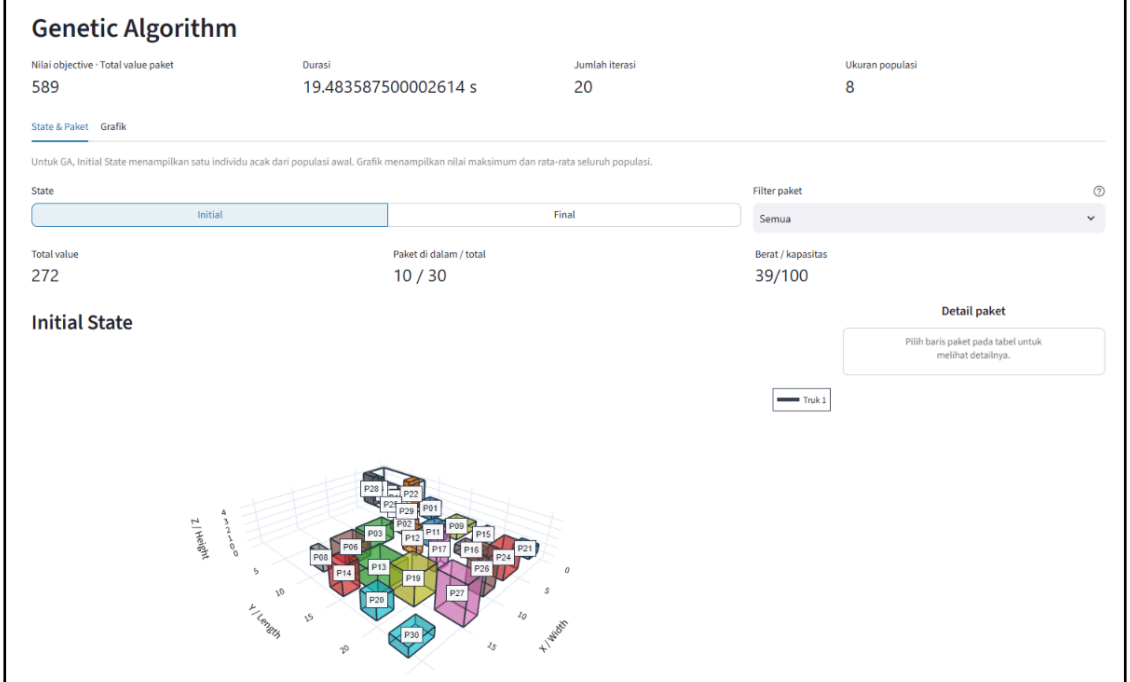
Final State



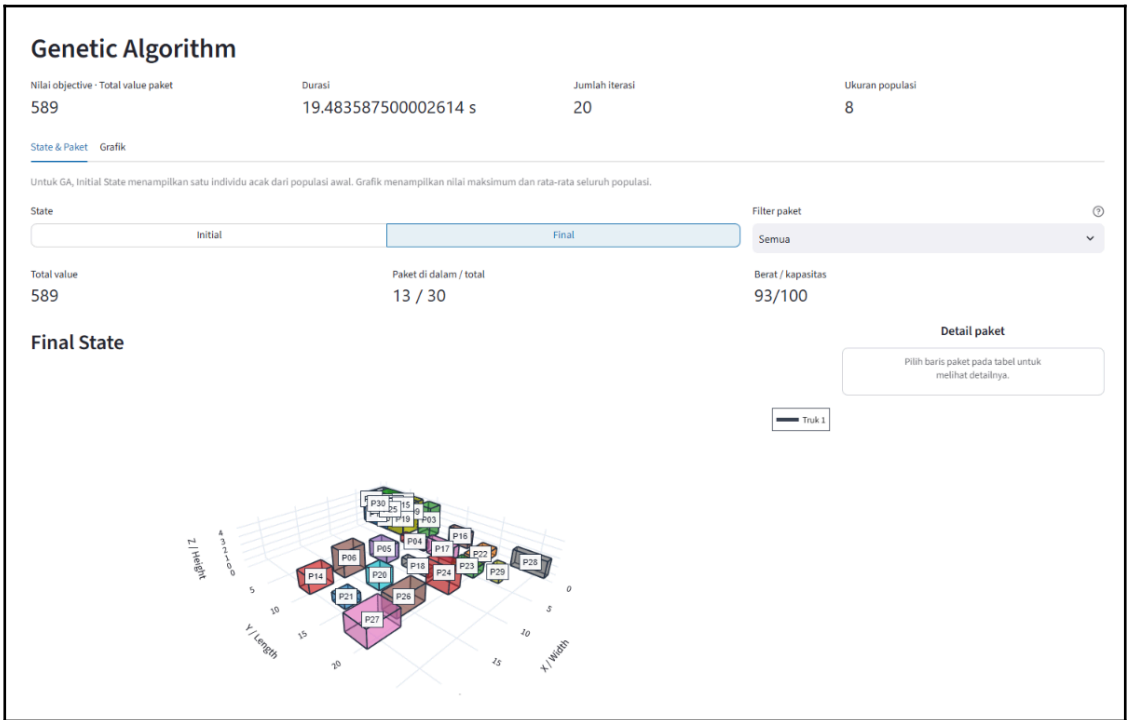
Grafik Objektif



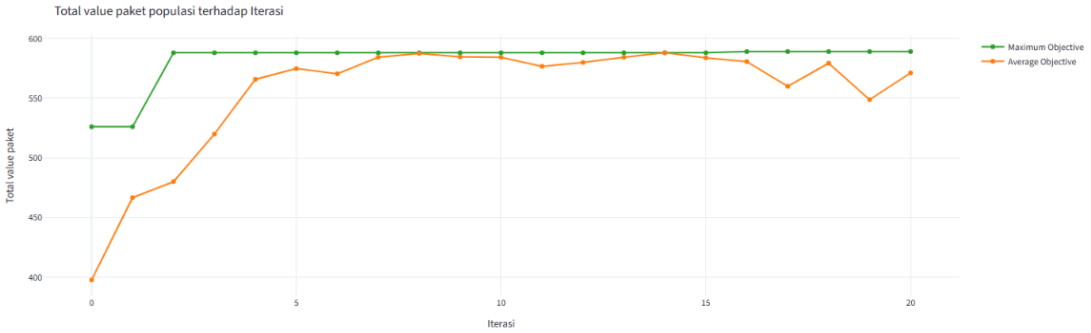
Iterasi Ke-3
Seed: 12800
Initial State



Final State

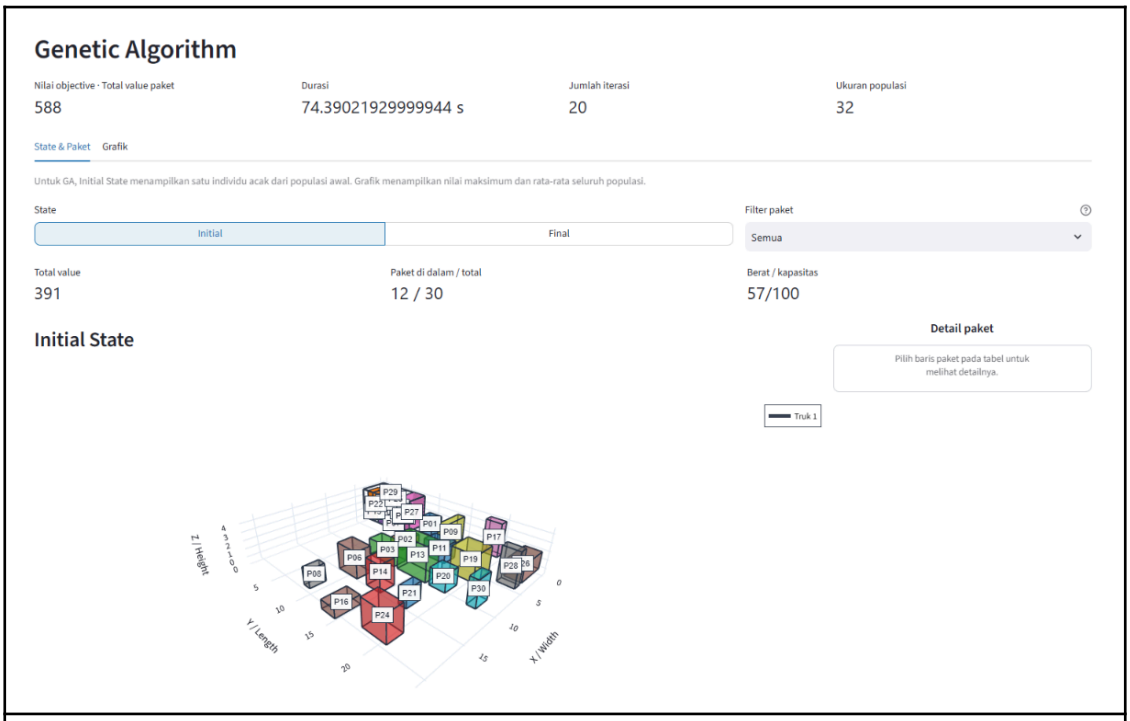


Grafik Objektif

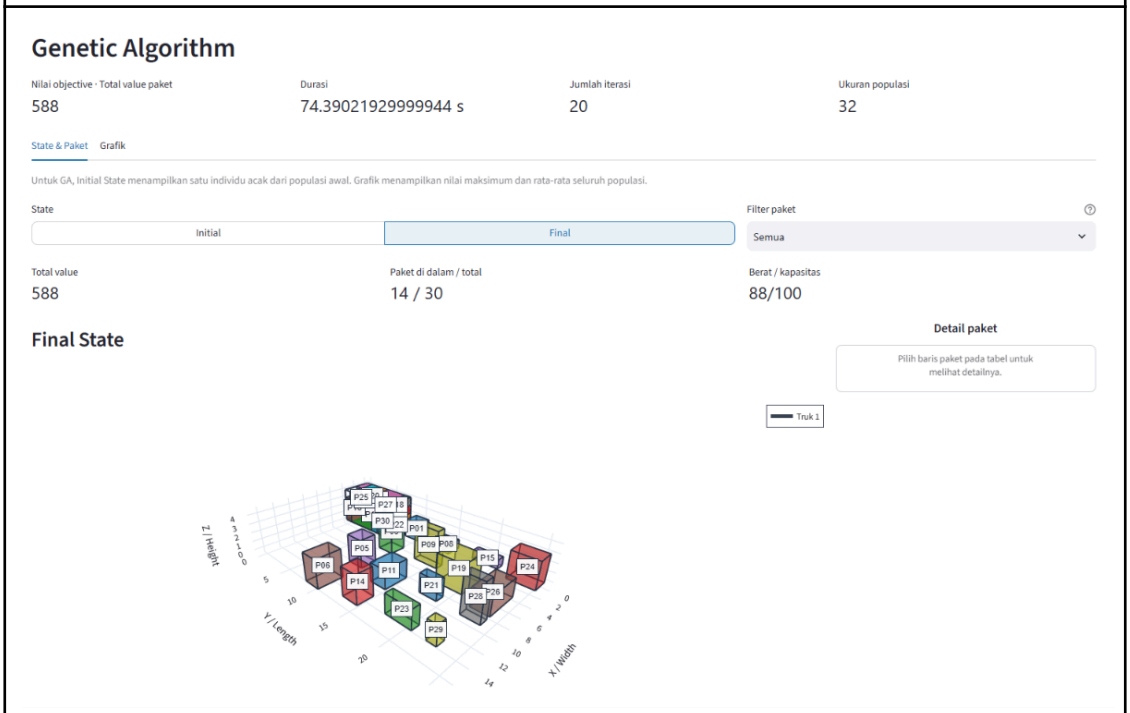


5. Pengujian GA-05

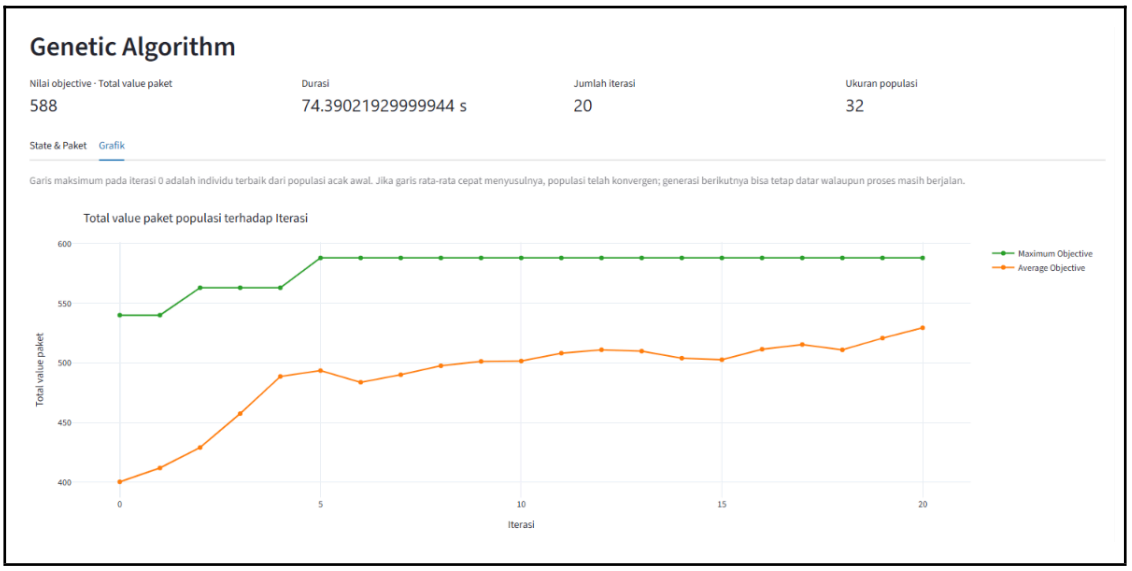
Iterasi Ke-1
Seed: 11911
Initial State



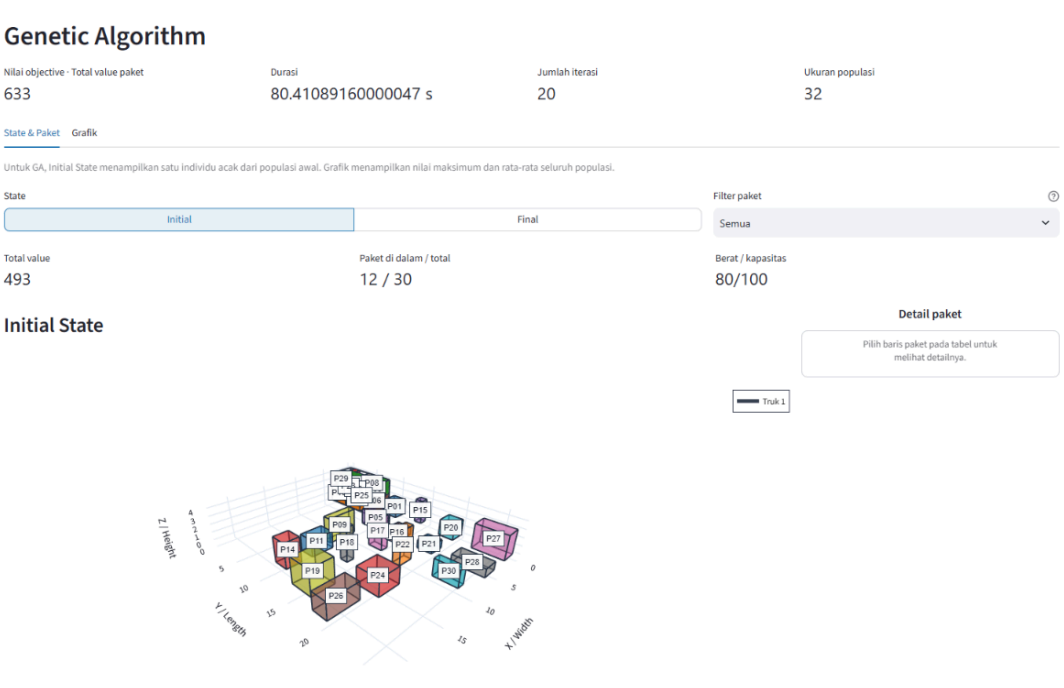
Final State



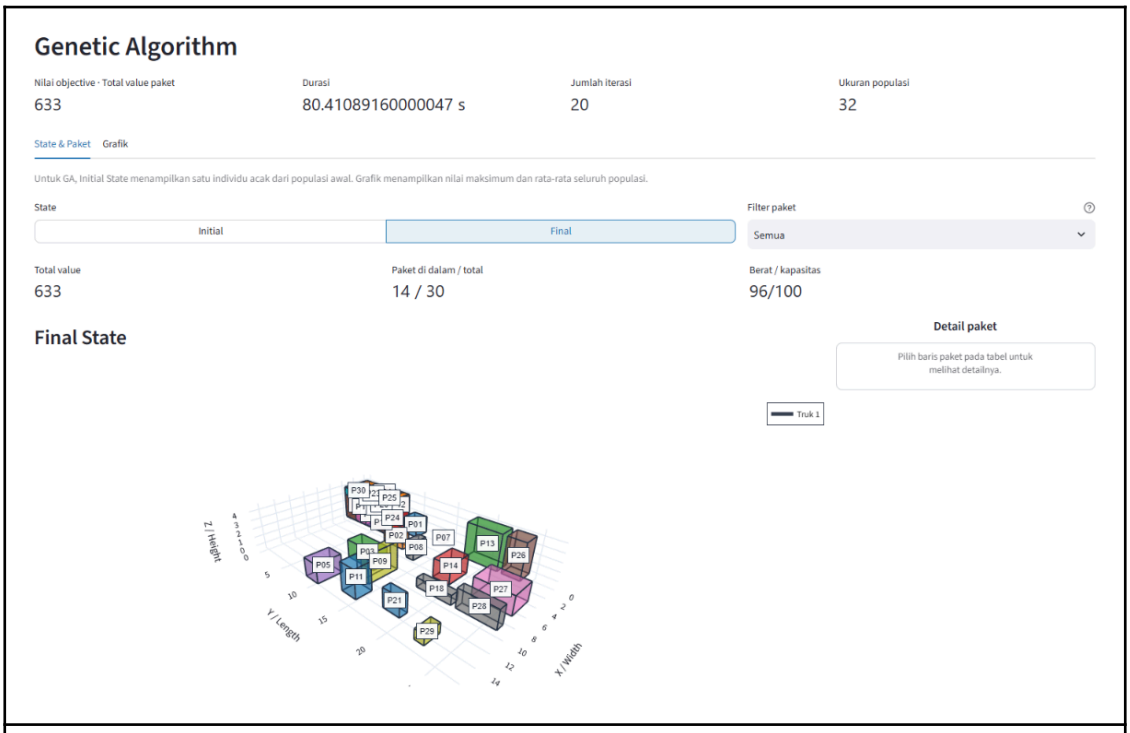
Grafik Objektif



Iterasi Ke-2
Seed: 33933
Initial State



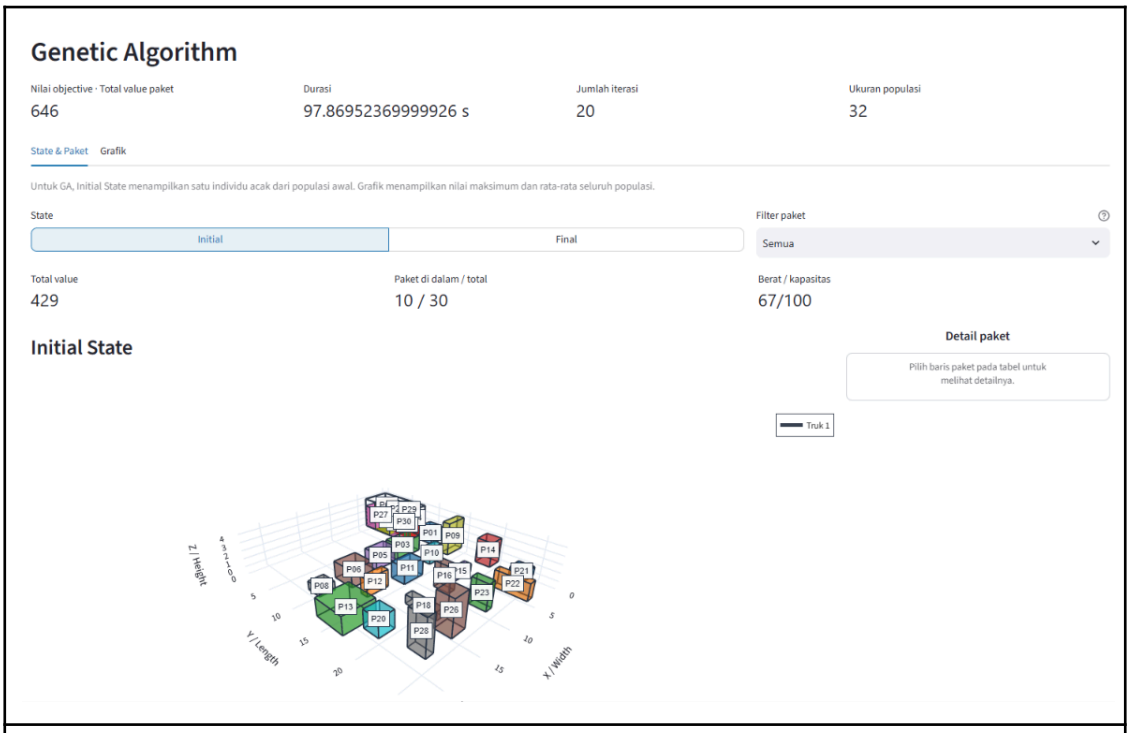
Final State



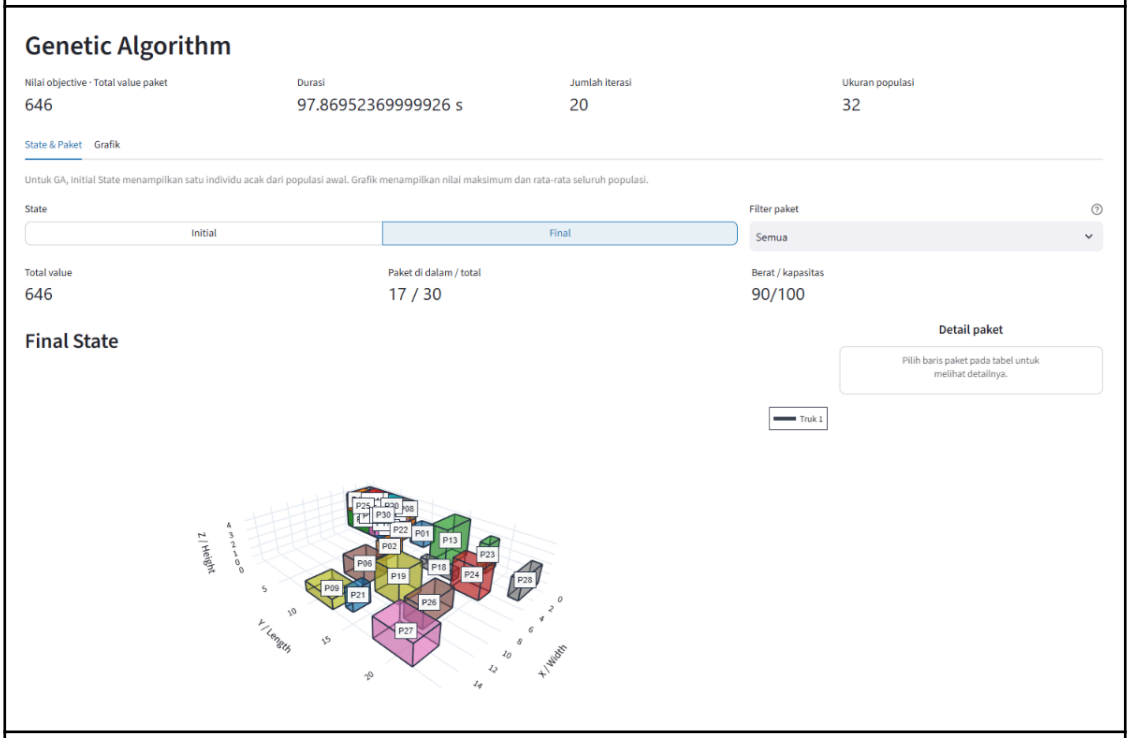
Grafik Objektif



Iterasi Ke-3
Seed: 99199
Initial State



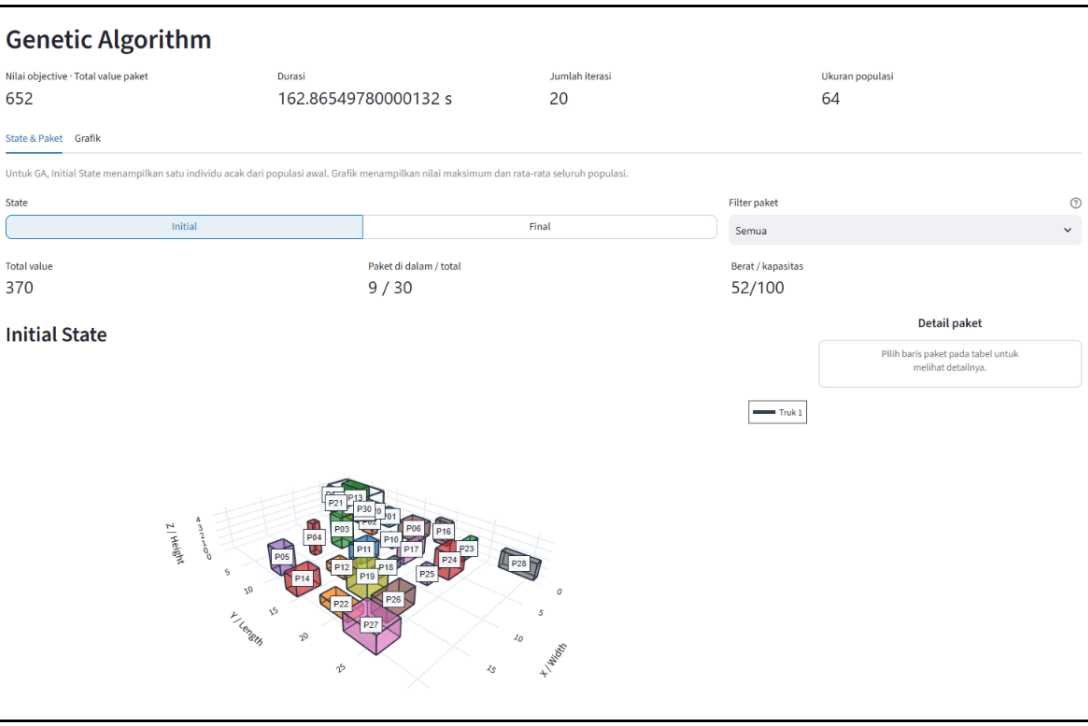
Final State

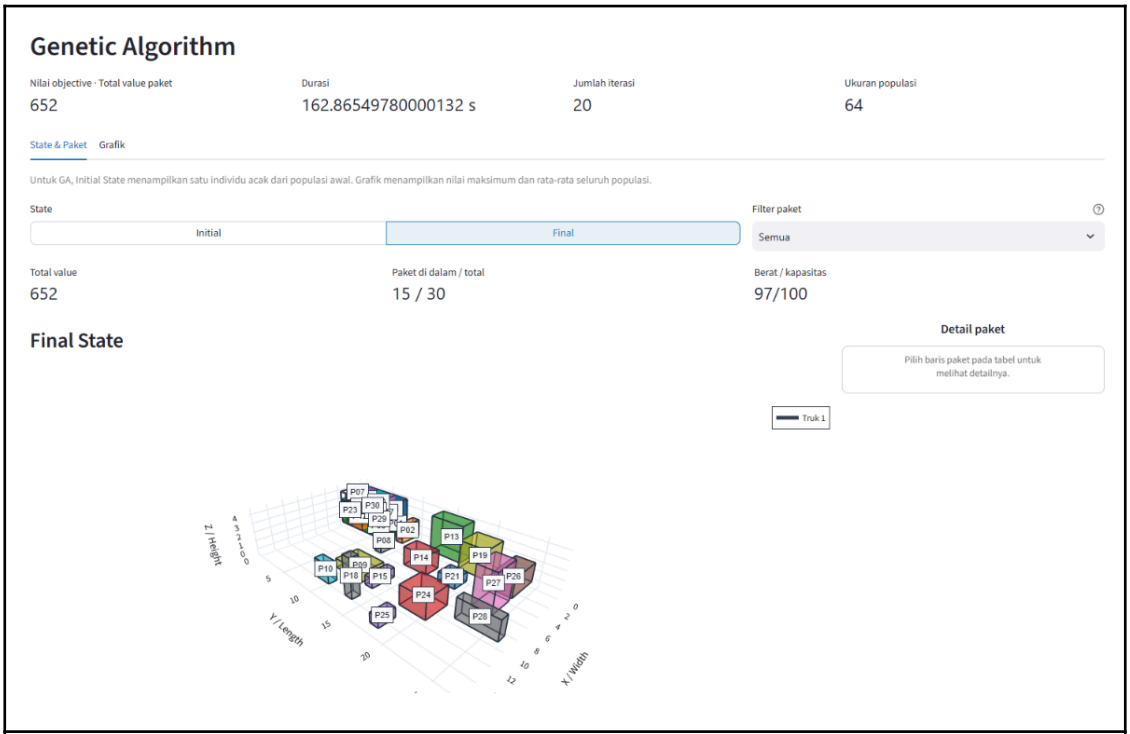


Grafik Objektif



6. Pengujian GA-06

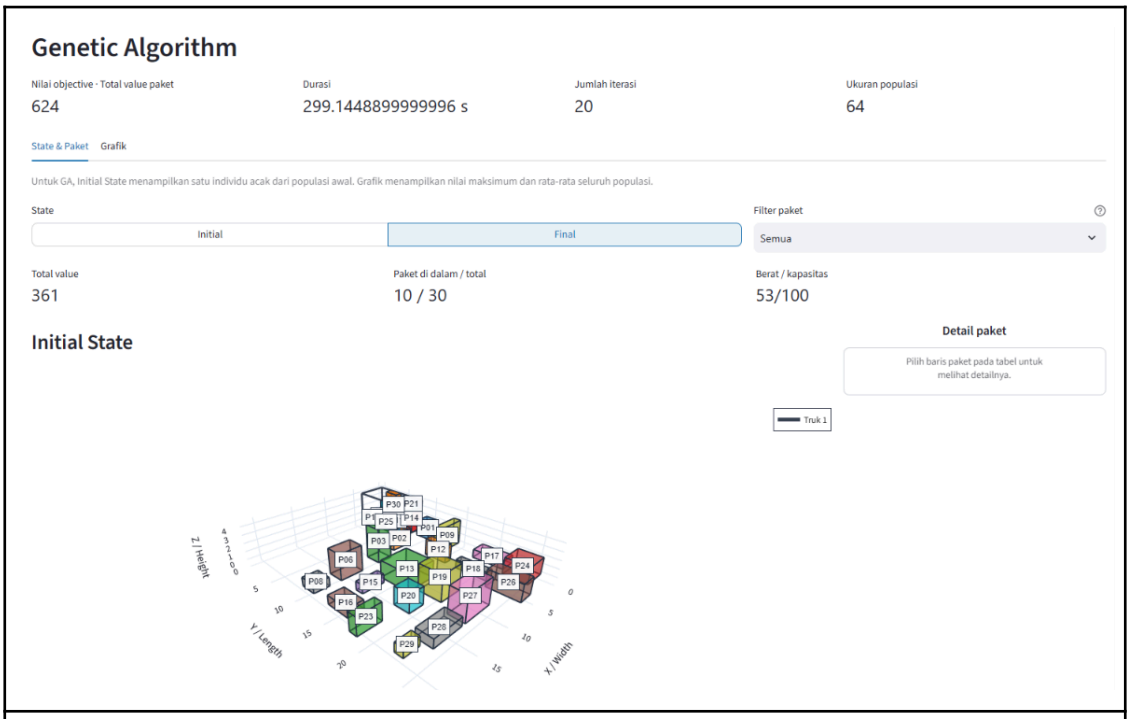
Iterasi Ke-1
Seed: 6285282061595
Initial State

Final State



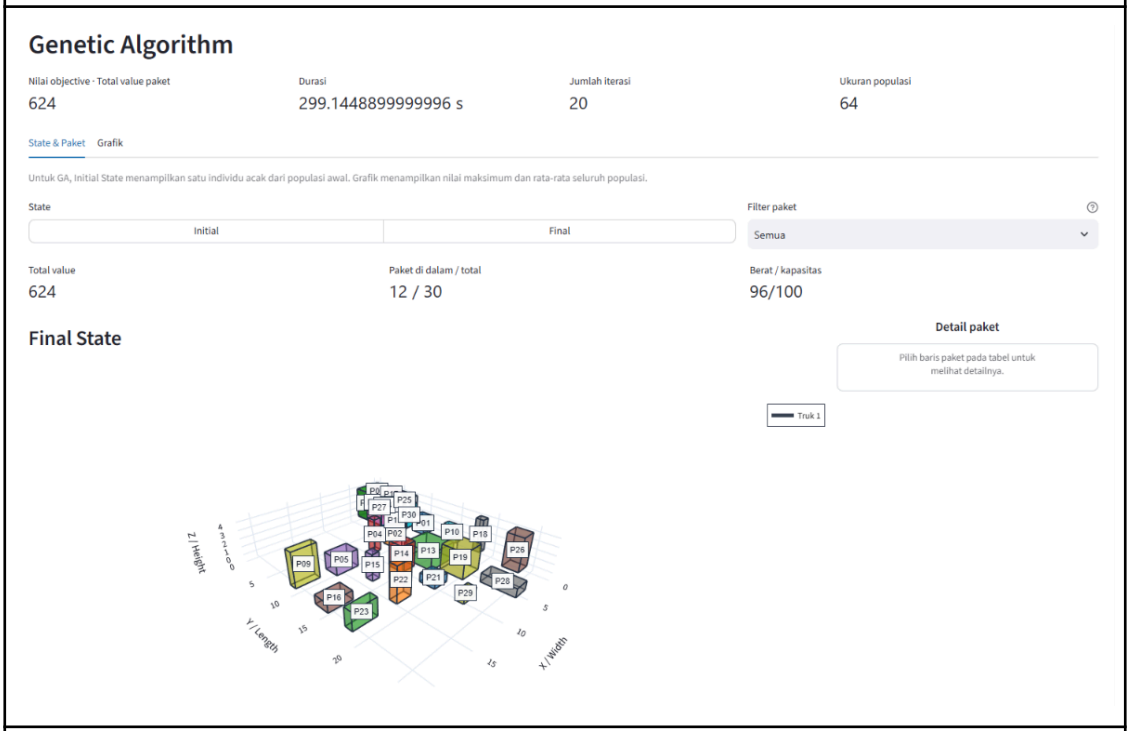
Grafik Objektif



Iterasi Ke-2
Seed: 6281335308362
Initial State

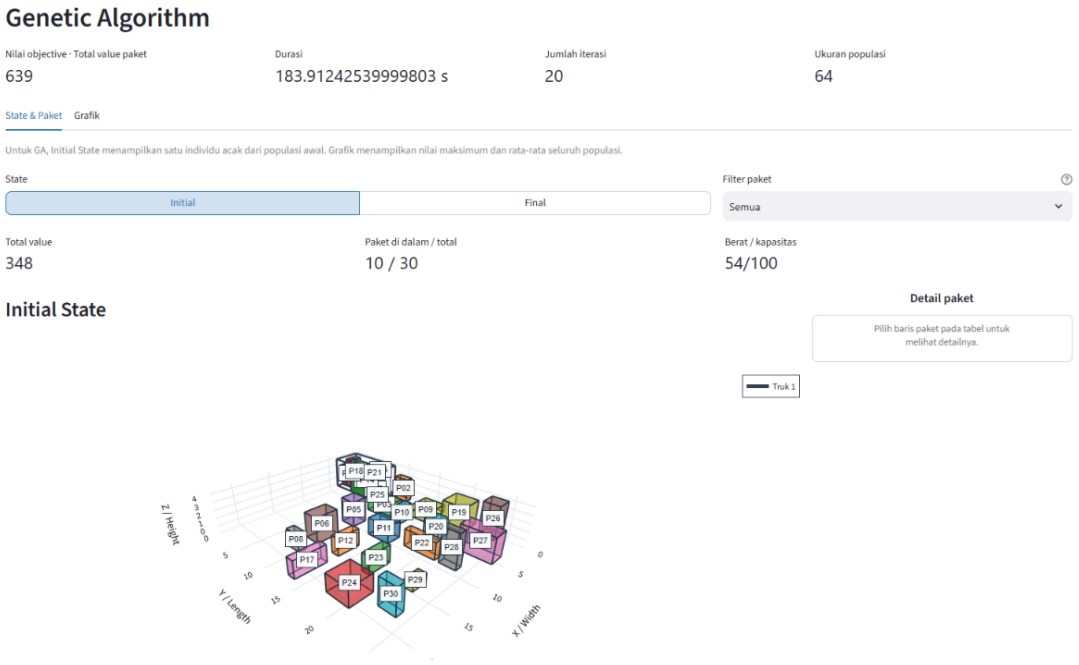


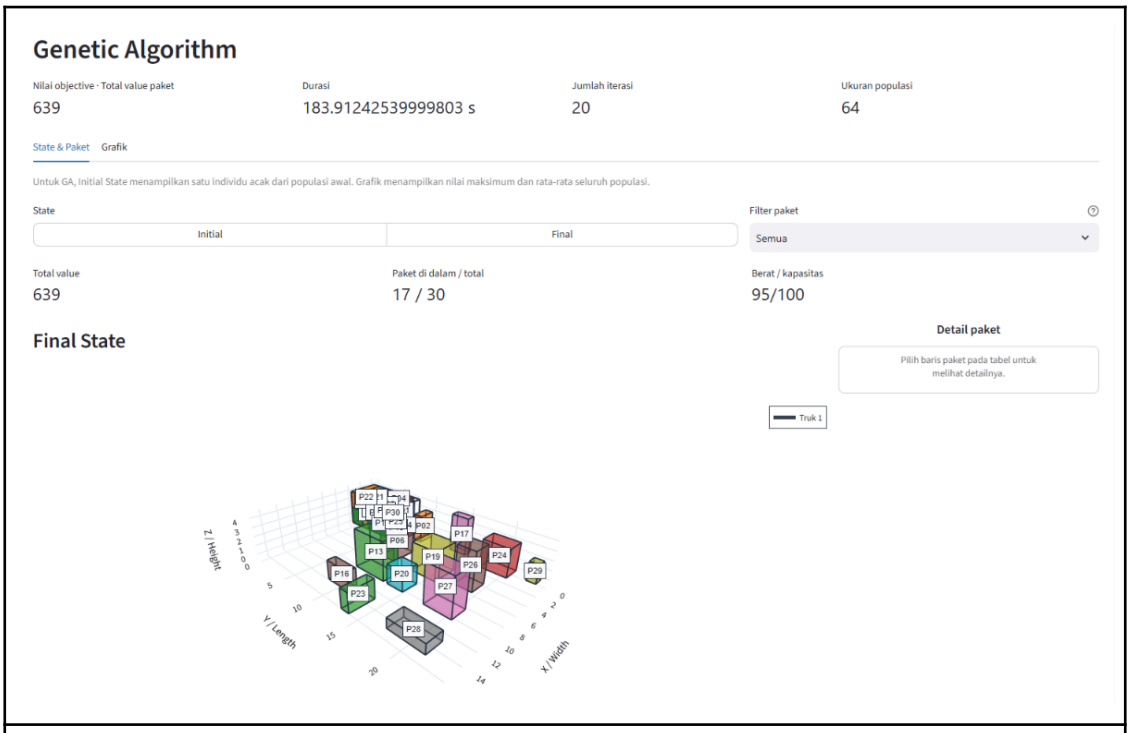
Final State



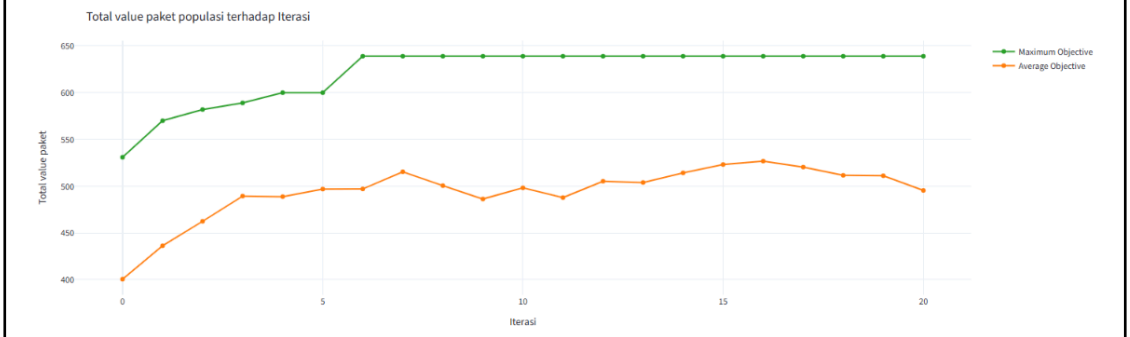
Grafik Objektif



Iterasi Ke-3
Seed: 6285808426741
Initial State

Final State



Grafik Objektif



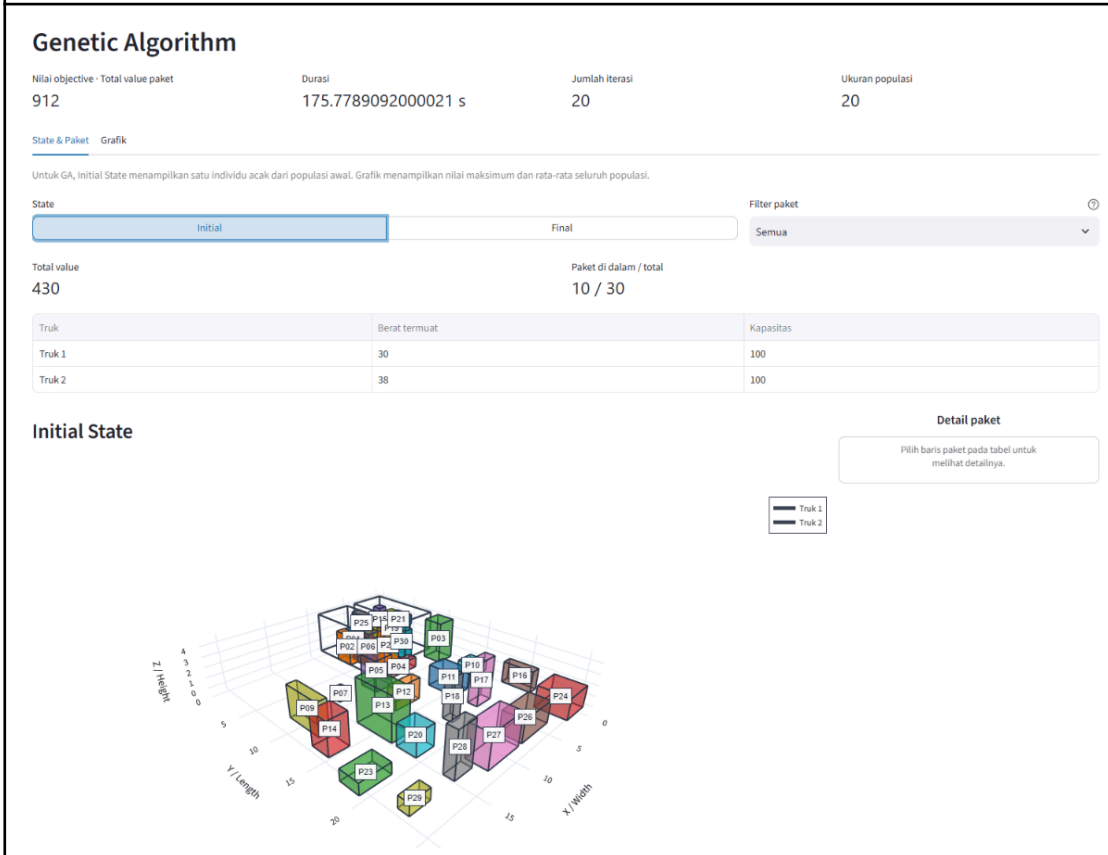
D. Pengujian Bonus Multi Truck

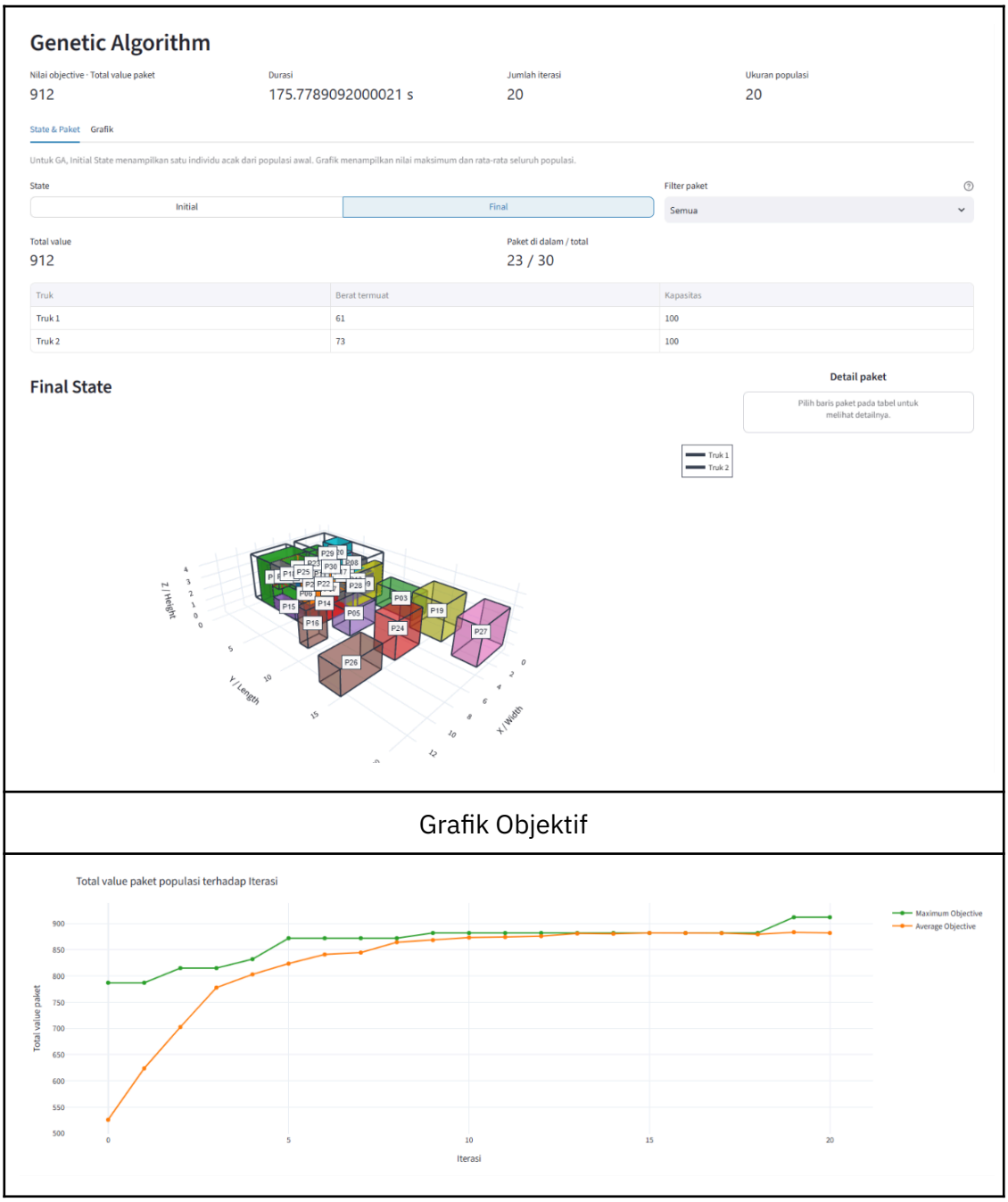
[Bridging]

Tabel 3.8 Variasi Parameter Pengujian Multi Truck

Parameter	Pengujian 1	Pengujian 2	Pengujian 3
Seed	42	42	42
Algoritma	Genetic	Genetic	Sideways HC
Jumlah Populasi	20	20	-
Jumlah Iterasi	20	20	100
Jumlah truk	2	3	4

1. Pengujian MT-01

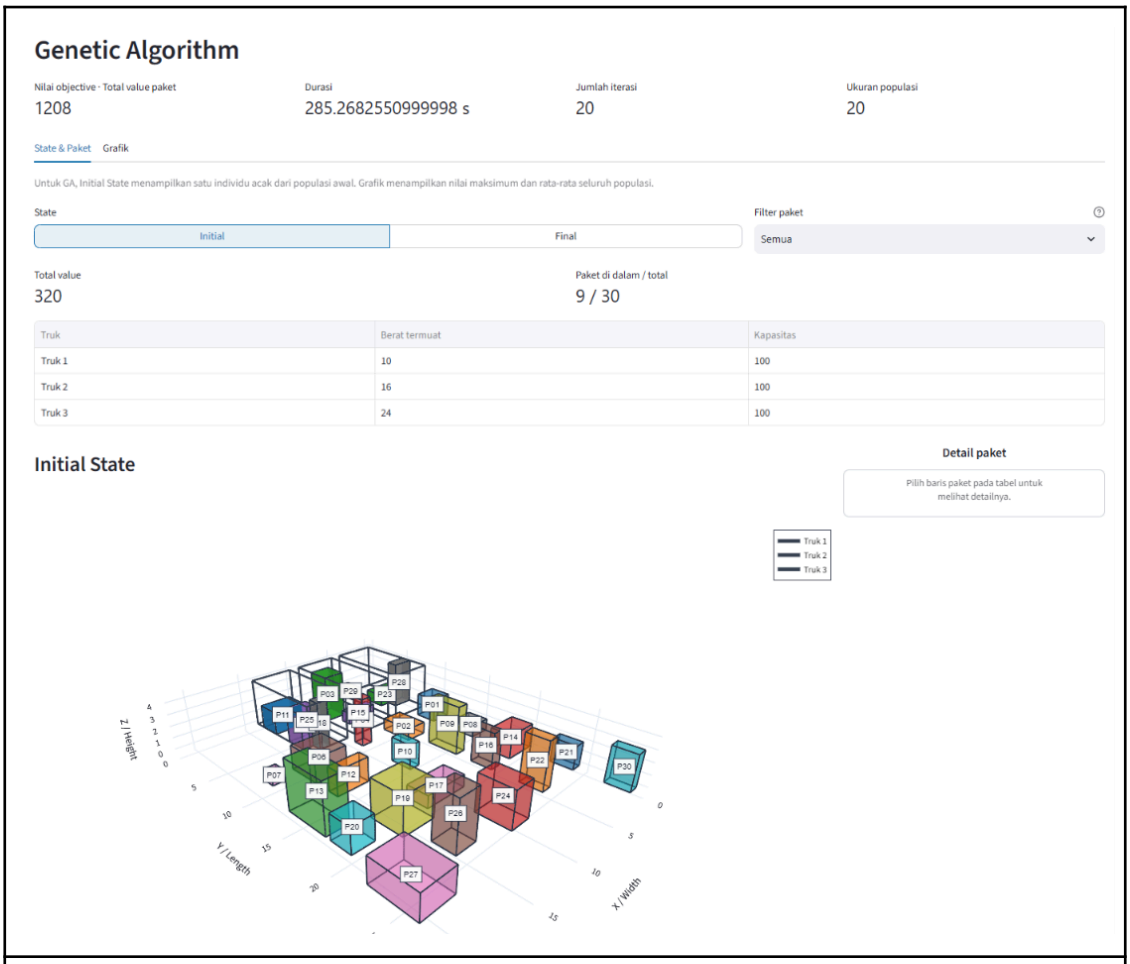
Seed: 42
Initial State

Final State



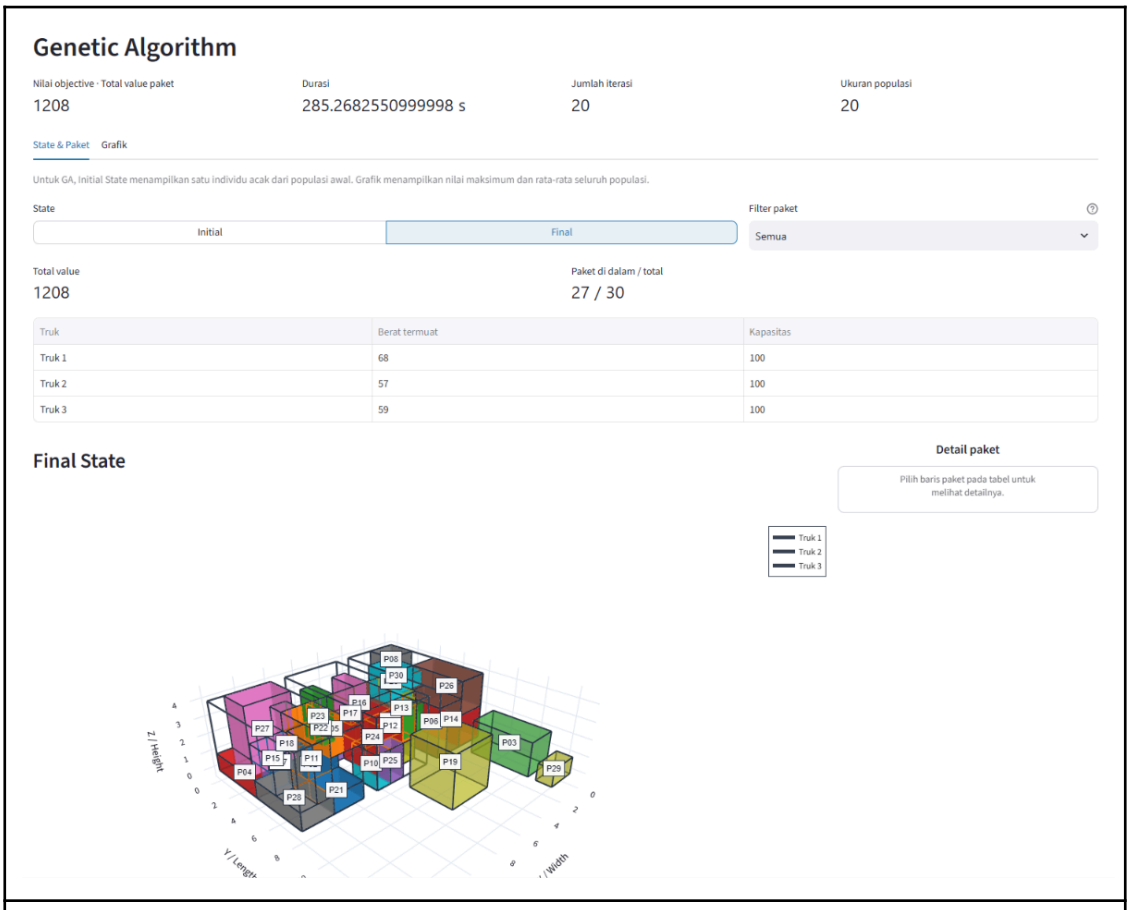
Grafik Objektif

2. Pengujian MT-02

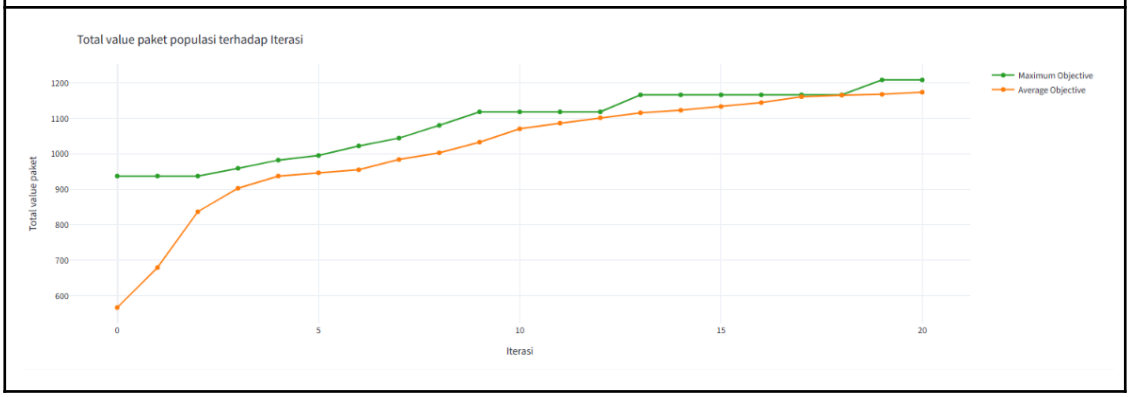
Seed: 42
Initial State


Genetic Algorithm
Nilai objective · Total value paket
1208
Durasi
285.2682550999998 s
Jumlah iterasi
20
Ukuran populasi
20
State & Paket
Grafik
Untuk GA, Initial State menampilkan satu individu acak dari populasi awal. Grafik menampilkan nilai maksimum dan rata-rata seluruh populasi.
State
Initial
Final
Filter paket
Semua
Total value
320
Paket di dalam / total
9 / 30
Truk
Berat termuat
Kapasitas
Truk 1
10
100
Truk 2
16
100
Truk 3
24
100
Initial State
Detail paket
Pilih baris paket pada tabel untuk melihat detailnya.
Truk 1
Truk 2
Truk 3
Z / Height
Y / Length
X / Width

Final State

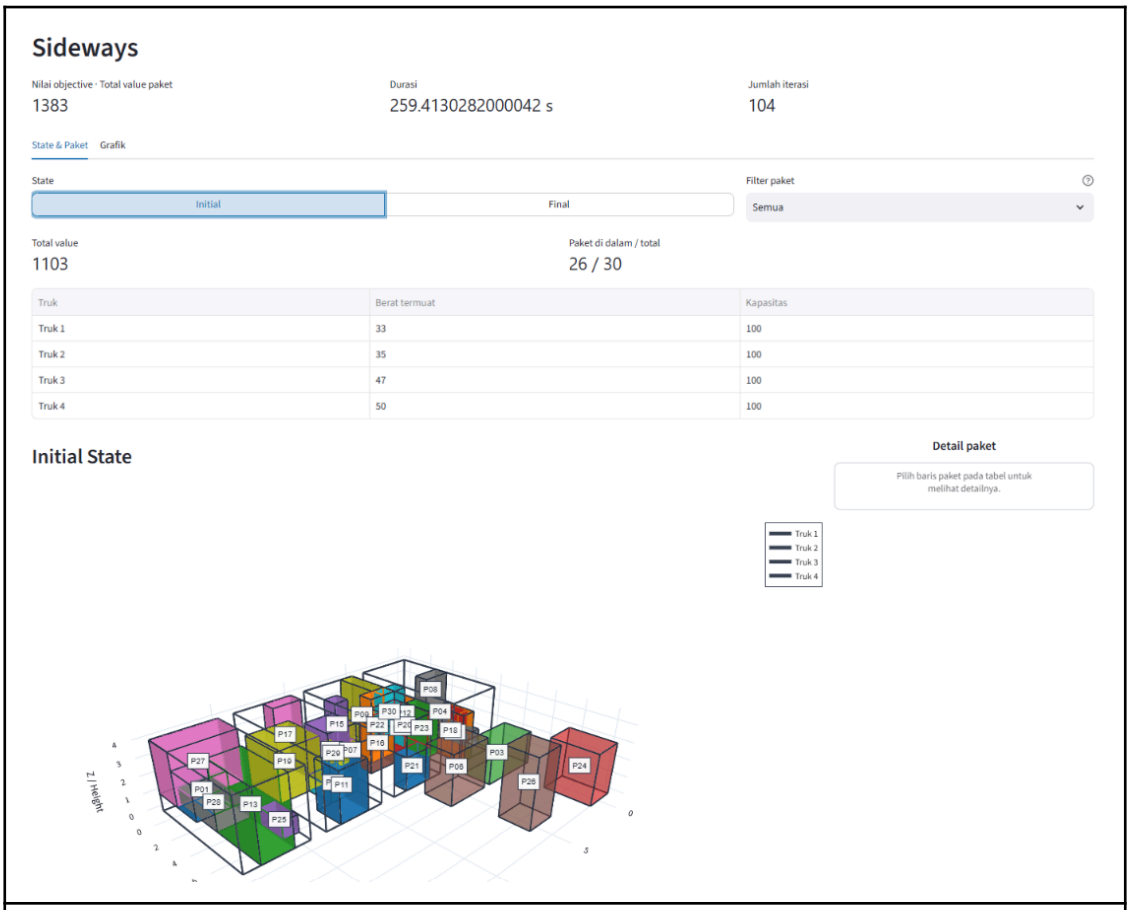


Grafik Objektif

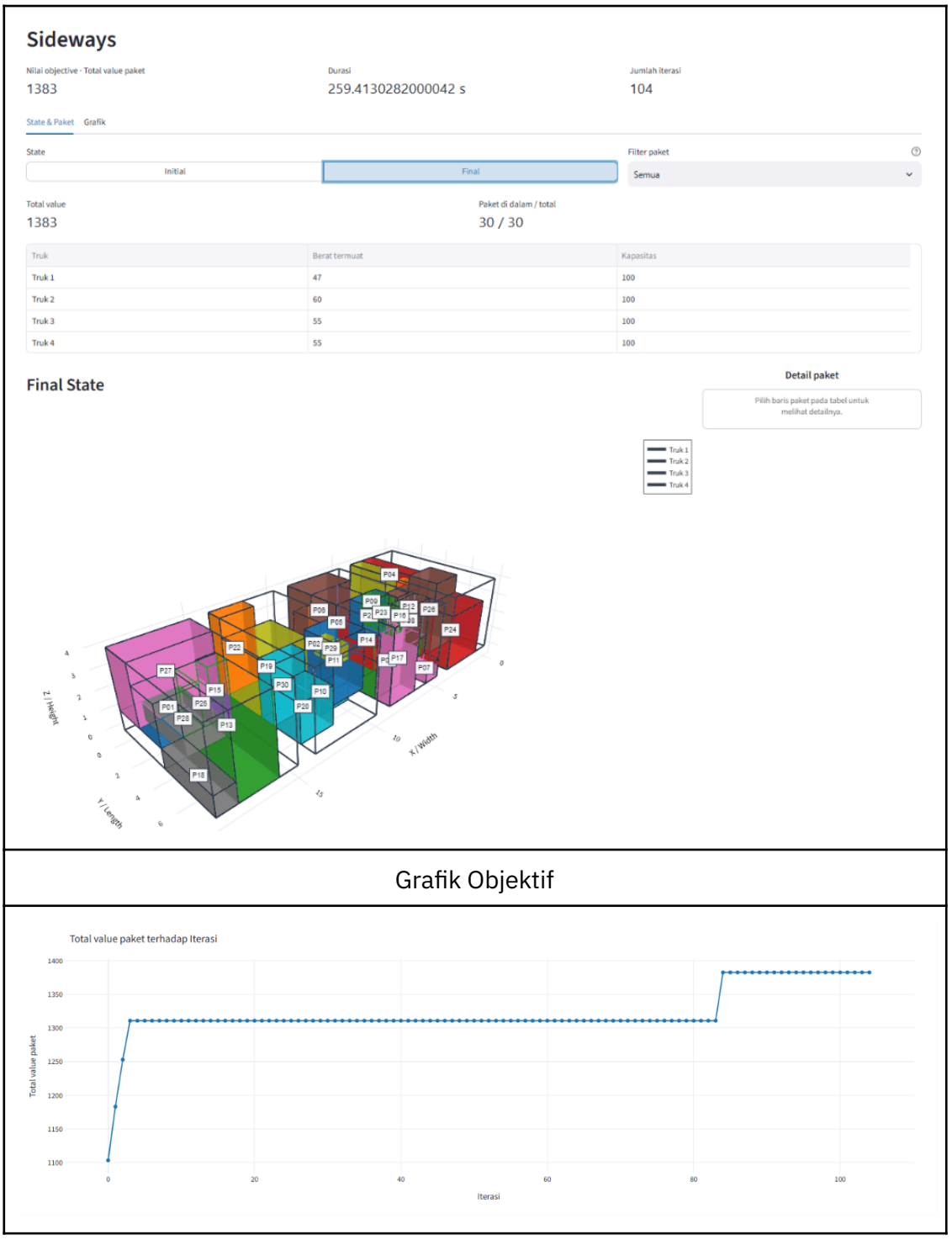


3. Pengujian MT-03

Seed: 42
Initial State


Sideways
Nilai objective · Total value paket
1383
Durasi
259.4130282000042 s
Jumlah iterasi
104
State & Paket
Grafik
State
Initial
Final
Filter paket
Semua
Total value
1103
Paket di dalam / total
26 / 30
Truk
Berat termuat
Kapasitas
Truk 1
33
100
Truk 2
35
100
Truk 3
47
100
Truk 4
50
100
Initial State
Detail paket
Pilih baris paket pada tabel untuk melihat detailnya.
Truk 1
Truk 2
Truk 3
Truk 4
Z / Height

Final State



Grafik Objektif

E. Pengujian Bonus Objective Function (ETA)

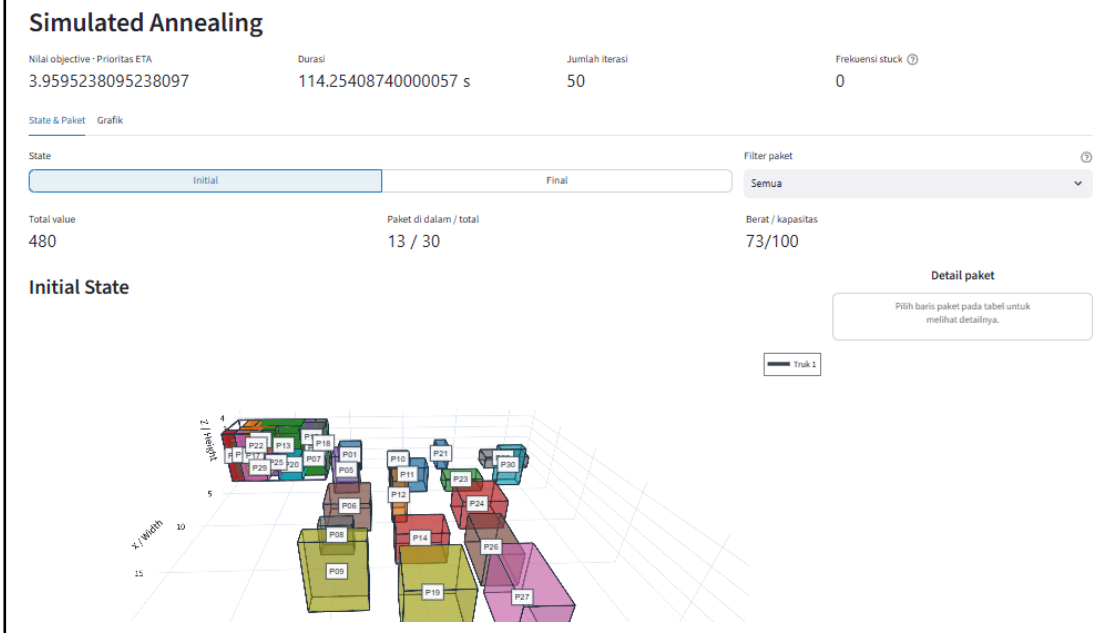
Selain fungsi objektif *value*, dilakukan pengujian tambahan menggunakan fungsi objektif ETA untuk memprioritaskan paket dengan waktu pengiriman lebih dekat. Skor ETA dihitung dari paket yang berhasil dimuat ke dalam truk. Pengujian

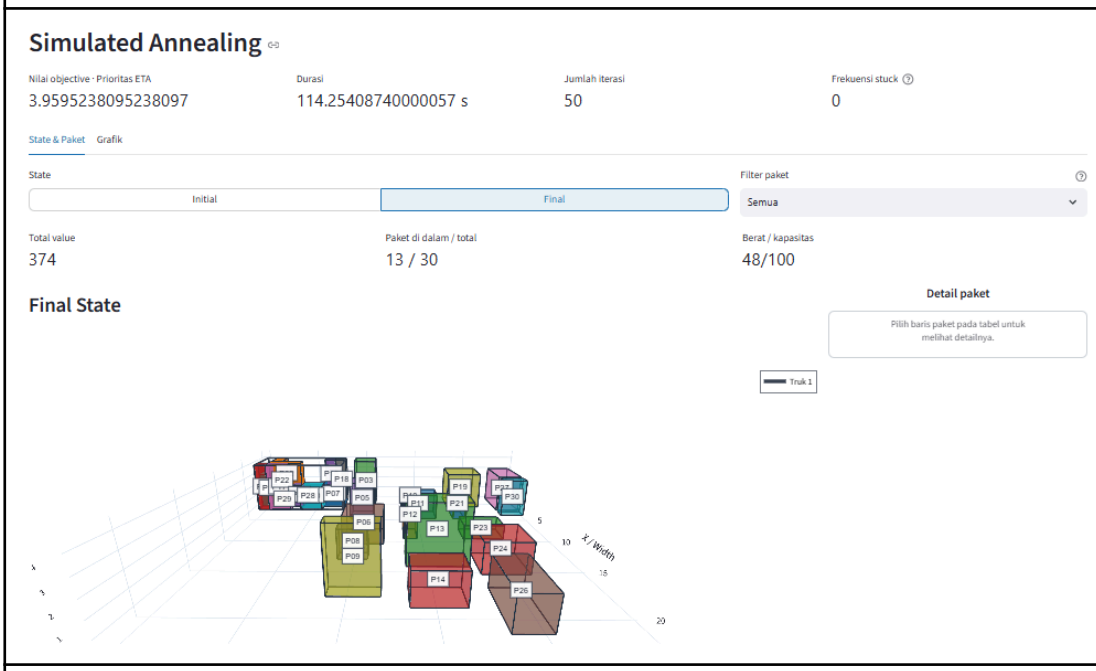
dilakukan tiga kali dengan parameter Simulated Annealing yang sama, sementara seed dibedakan agar setiap pencarian dapat diulang dengan kondisi awal berbeda.

Tabel 3.9 Variasi Parameter Pengujian Simulated Annealing dengan ETA

Parameter	Pengujian 1	Pengujian 2	Pengujian 3
Seed	42	43	44
Temperatur Awal	100	100	100
Cooling rate	0.95	0.95	0.95
Maksimum iterasi	50	50	50
Fungsi objektif	eta	eta	eta
Jumlah truk	1	1	1

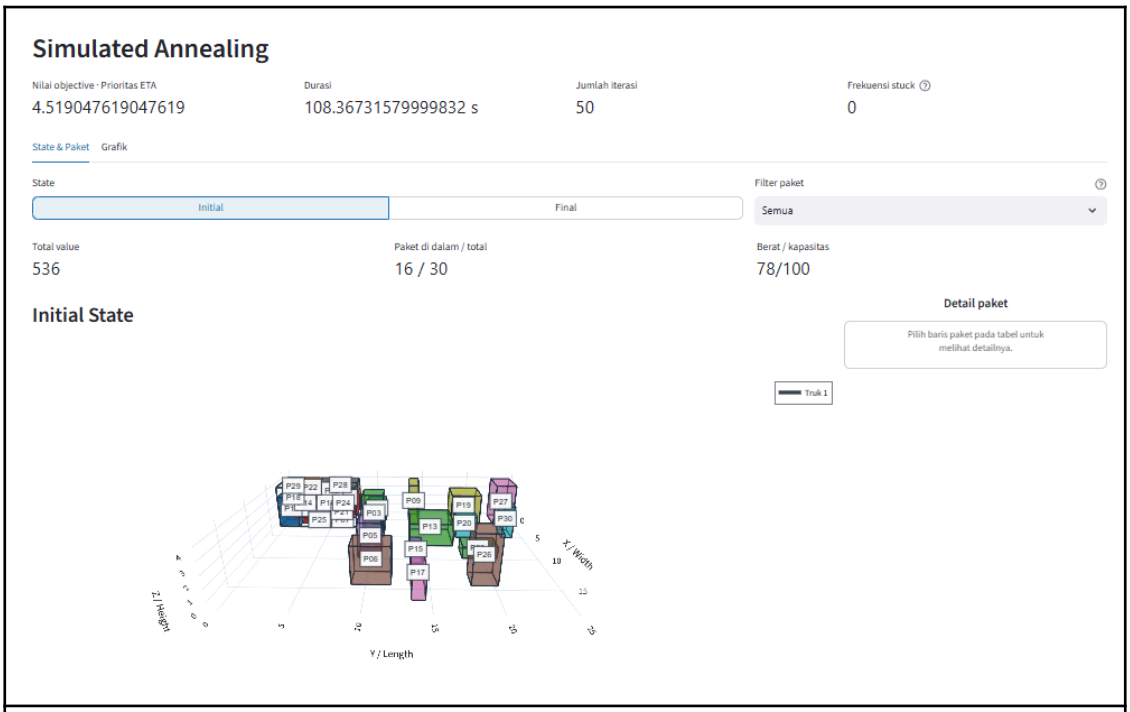
1. Pengujian ETA-01

Initial State

Final State

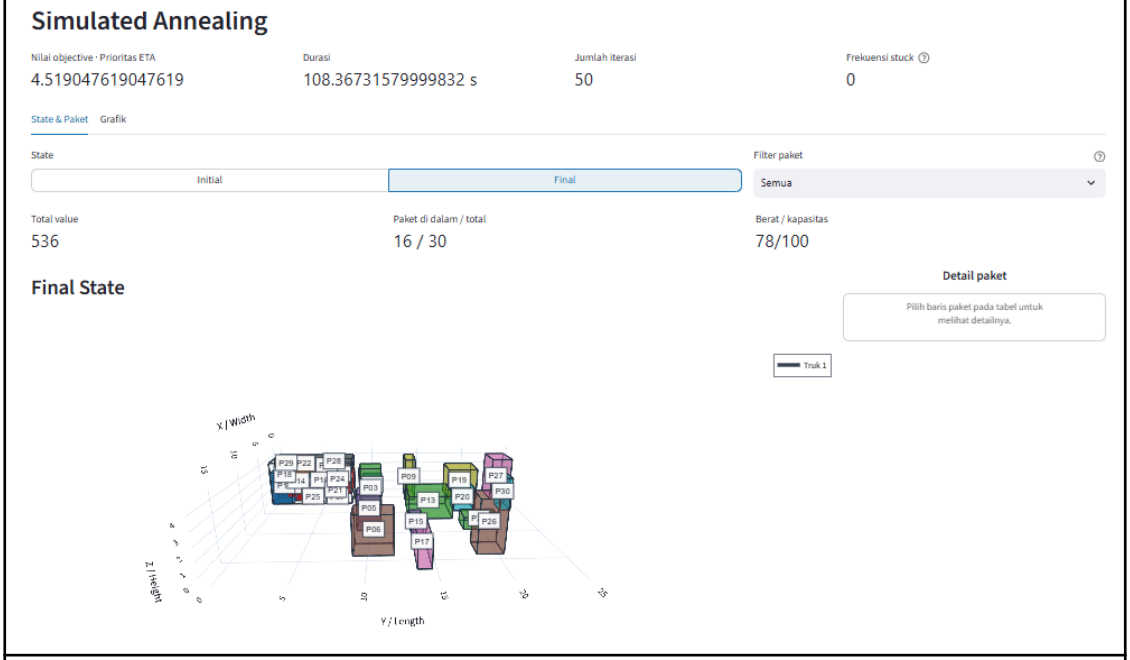
Initial State

Grafik Objektif


2. Pengujian ETA-02

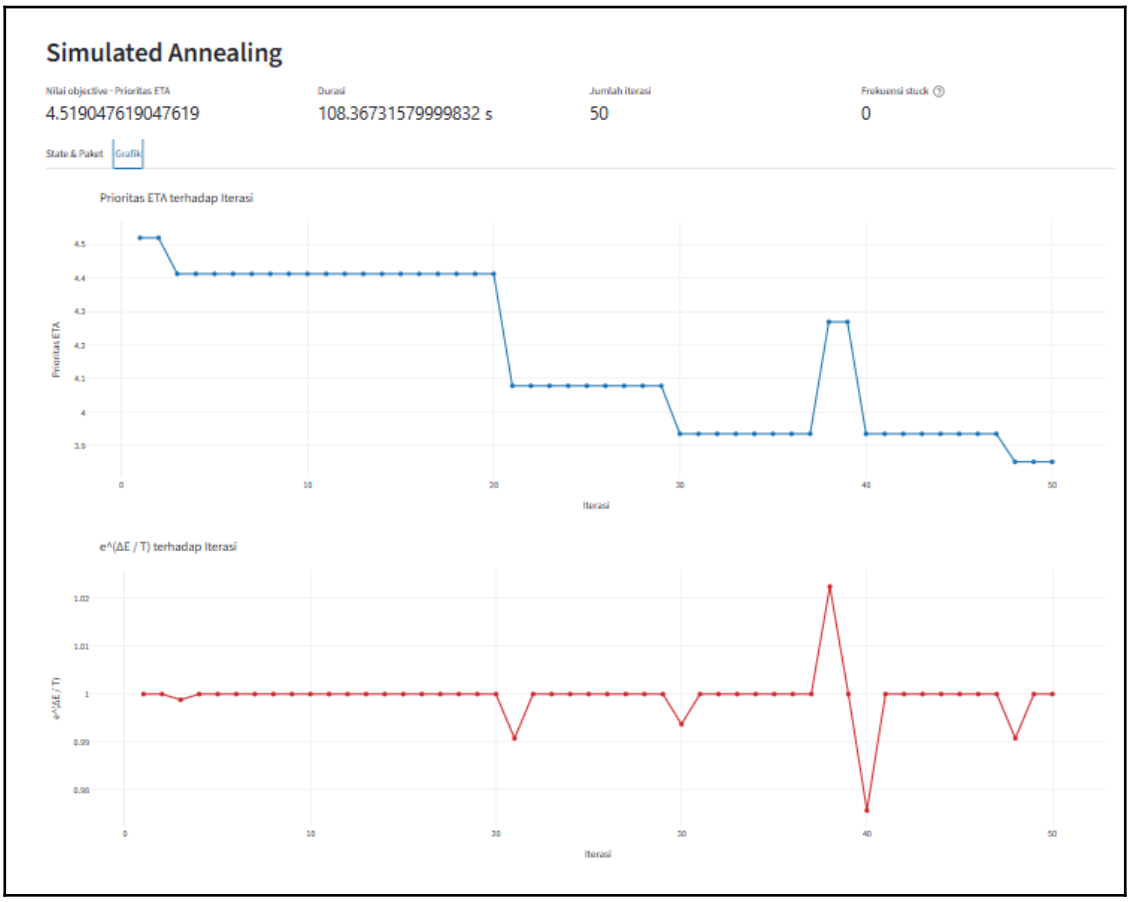
Initial State



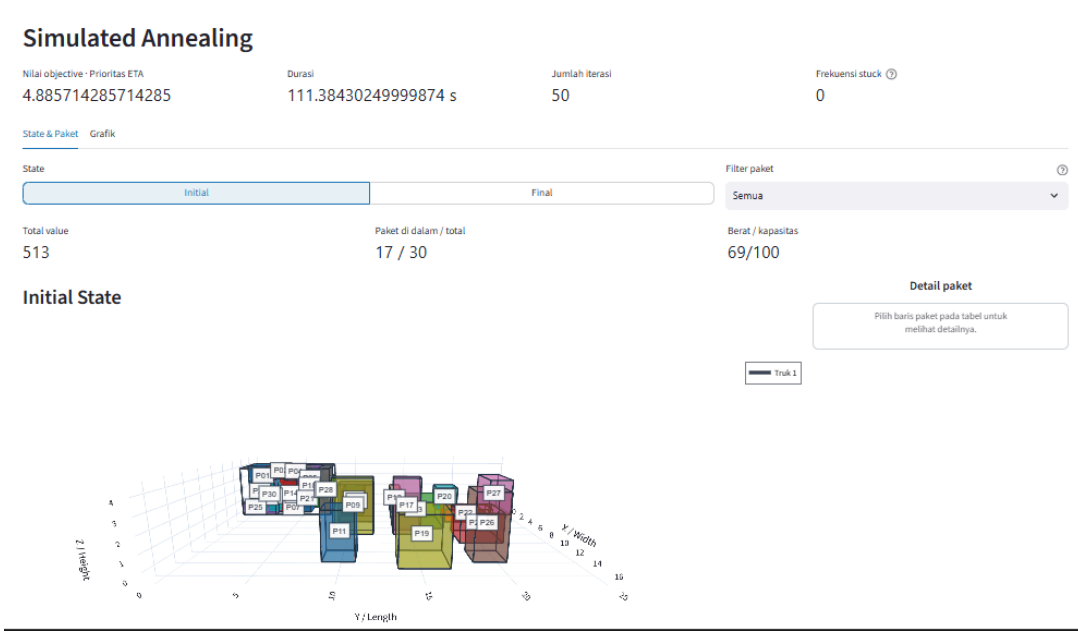
Final State

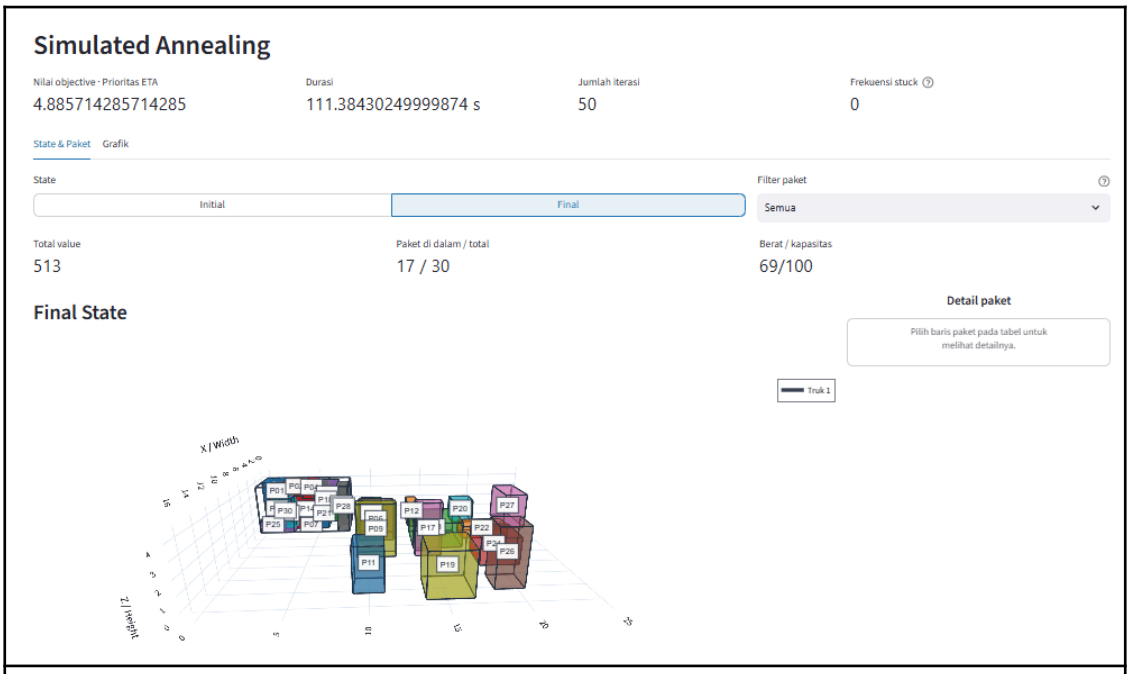


Grafik Objektif

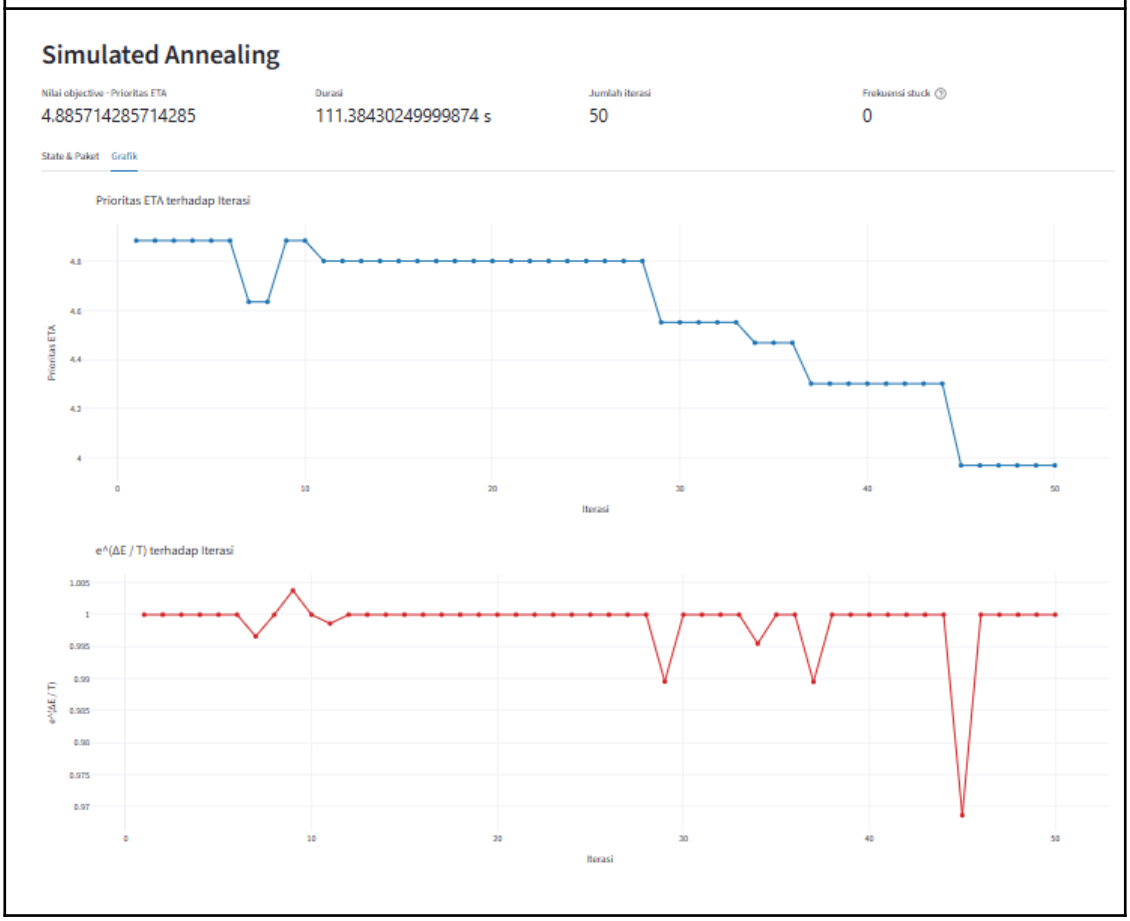


3. Pengujian ETA-03

Initial State

Final State



Grafik Objektif



Pada pengujian SA dengan fungsi objektif ETA, nilai objective terbaik dari tiga pengujian adalah 3,96, 4,52, dan 4,89. Nilai tertinggi diperoleh pada pengujian ketiga. Ketiganya mencapai batas 50 iterasi, dan frekuensi *stuck* tercatat 0. Durasi

pencarian berkisar antara 108,37 hingga 114,25 detik, dengan rata-rata 111,33 detik.

Pada ketiga grafik, nilai objective state saat iterasi terakhir lebih rendah daripada nilai terbaik yang dicapai: masing-masing 3,30, 3,85, dan 3,97. Hal ini menunjukkan SA sempat berpindah ke state dengan skor ETA lebih rendah setelah menemukan solusi yang lebih baik. Perbedaan nilai terbaik antar-pengujian juga menunjukkan hasil pencarian bervariasi berdasarkan seed yang digunakan.

F. Analisis Algoritma

Analisis dilakukan terhadap hasil eksperimen untuk membandingkan hasil akhir dari masing-masing algoritma dengan menjawab pertanyaan-pertanyaan berikut:

1. **Seberapa dekat tiap-tiap algoritma bisa mendekati *global optima* dan mengapa hasilnya demikian?**

   Untuk objective value, nilai tertinggi yang ditemukan adalah **667** oleh **Sideways Move**. Sebagai pembanding, batas atas berdasarkan kapasitas berat dan volume truk adalah **679**. Jadi, solusi 667 berjarak paling banyak 12 poin dari batas tersebut atau sekitar **98,2%**. Namun, batas 679 belum memperhitungkan apakah paket-paketnya dapat disusun secara geometris dan memenuhi aturan support, sehingga global optimum belum dapat dipastikan. Nilai tertinggi tiap algoritma adalah Sideways Move 667, GA 652, Random Restart 639, Steepest Ascent 635, Simulated Annealing 542, dan Stochastic 536.

2. **Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma *local search* yang lain?**

   Rata-rata objective akhir tertinggi diperoleh Sideways Move, yaitu 639,3, diikuti Random Restart sebesar 636,0 dan GA pada konfigurasi terbaiknya sebesar 638,3. Rata-rata Steepest Ascent adalah 602,3, sedangkan SA dan Stochastic masing-masing 522,0 dan 510,7. Sideways Move menghasilkan nilai tertinggi pada satu pengujian, sementara Random Restart memperoleh rata-rata yang hampir sama tinggi dengan hasil yang lebih konsisten. Pada GA, hasil terbaik muncul saat populasi berukuran 64 dengan 20 iterasi.

3. **Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?**

   Stochastic memiliki durasi rata-rata terendah, sekitar 0,60 detik, disusul Steepest Ascent 12,39 detik, SA 79,80 detik, Random Restart 97,52 detik, dan Sideways Move 181,02 detik. Durasi GA bergantung pada ukuran populasi dan iterasi: pada populasi 20, rata-rata durasi untuk 10, 20, dan 40 iterasi adalah 27,29, 47,10, dan 52,96 detik. Dengan 20 iterasi, rata-rata durasi untuk populasi 8, 32, dan 64 adalah 16,96, 84,22, dan 215,31 detik. Dengan demikian, penambahan iterasi dan populasi meningkatkan waktu pencarian, terutama ketika populasi diperbesar.

4. **Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?**

   Rentang nilai akhir tiga pengujian terkecil diperoleh Random Restart, yaitu 634–639, sehingga hasilnya paling konsisten pada pengujian ini. Rentang Sideways Move adalah 607–667, Steepest Ascent 551–635, Stochastic 483–536, dan SA 503–542. Rentang hasil GA berbeda-beda menurut konfigurasi, yaitu 17–58 poin. Hasil ini menunjukkan bahwa seed dan konfigurasi dapat memengaruhi hasil; kesimpulan konsistensi tetap perlu dilihat sebagai indikasi karena setiap konfigurasi hanya diuji tiga kali. Dengan populasi tetap 20, rata-rata objective akhir untuk 10, 20, dan 40 iterasi berturut-turut adalah 597,0, 614,0, dan 613,7. Penambahan iterasi dari 10 ke 20 meningkatkan rata-rata hasil, tetapi penambahan hingga 40 iterasi tidak memberikan peningkatan rata-rata pada pengujian ini. Dengan iterasi tetap 20, rata-rata objective untuk populasi 8, 32, dan 64 meningkat dari 583,0 menjadi 622,3 dan 638,3. Populasi yang lebih besar menghasilkan nilai rata-rata dan nilai terbaik yang lebih tinggi, tetapi durasi pencariannya juga meningkat.

5. **Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?**

   Dengan populasi tetap 20, rata-rata objective akhir untuk 10, 20, dan 40 iterasi berturut-turut adalah 597,0, 614,0, dan 613,7. Penambahan iterasi dari 10 ke 20 meningkatkan rata-rata hasil, tetapi penambahan hingga 40 iterasi tidak memberikan peningkatan rata-rata pada pengujian ini.

Dengan iterasi tetap 20, rata-rata objective untuk populasi 8, 32, dan 64 meningkat dari 583,0 menjadi 622,3 dan 638,3. Populasi yang lebih besar menghasilkan nilai rata-rata dan nilai terbaik yang lebih tinggi, tetapi durasi pencariannya juga meningkat. Selain itu, jumlah populasi yang tinggi juga menyebabkan perbedaan yang tinggi antara value maksimum dan rata-rata karena tidak mampu mempertahankan konvergensi akibat jumlah elit yang terlalu kecil.

BAB IV

KESIMPULAN DAN SARAN

A. Kesimpulan

Program berhasil menerapkan Hill Climbing, Simulated Annealing, dan Genetic Algorithm untuk mencari susunan paket dengan memperhatikan batasan truk. Pada pengujian satu truk dengan objective value, nilai tertinggi yang diperoleh adalah 667 oleh Sideways Hill Climbing. Hasil tersebut belum membuktikan optimum global karena setiap algoritma menggunakan mekanisme dan batas pencarian yang berbeda.

Pada Genetic Algorithm, rata-rata nilai meningkat saat populasi bertambah dari 8 menjadi 64, sedangkan penambahan iterasi dari 20 menjadi 40 pada populasi 20 tidak menunjukkan peningkatan berarti. Pengujian bonus menunjukkan nilai total lebih tinggi saat jumlah truk bertambah. Hasil Simulated Annealing juga bervariasi antar-seed, baik pada objective value maupun ETA.

B. Saran

Pengujian berikutnya sebaiknya menggunakan lebih banyak seed dan pengulangan dengan kondisi yang setara agar hasil setiap algoritma dapat dibandingkan secara adil. Variasi parameter Genetic Algorithm dan jumlah truk juga perlu diperluas, lalu hasilnya dirangkum menggunakan rata-rata dan simpangan baku. Kedekatan solusi terhadap optimum global dapat dinilai dengan membandingkan hasil heuristik terhadap batas atas atau solusi eksak pada problem berukuran kecil.

REFERENSI

Institut Teknologi Bandung. (2026). Beyond Classical Search. Salindia perkuliahan IF3170 Inteligensi Artifisial, Semester I Tahun Akademik 2026/2027.

Python Software Foundation. Python 3 documentation. Dokumentasi resmi Python. Diakses pada 20 Oktober 2026.

Streamlit. Streamlit documentation. Dokumentasi resmi Streamlit. Diakses pada 20 Oktober 2026.

LAMPIRAN

Form Penggunaan AI: [Form Penggunaan AI]

Tabel Pembagian Tugas:

Nama/NIM	Pembagian Tugas
Wafiq Hibban Robbany / 13524016	Representasi State, Initial State, Objective Function utama dan Constraint, Pemilihan Neighbor, Keempat Algoritma Hill Climbing.
Moh. Hafizh Irham Perdana / 13524025	Genetic Algorithm, Bonus Multitruk.
A. Fawwaz Azam Wicaksono / 13524070	Algoritma Simulated Annealing, Visualisasi, Bonus Objective Function Tambahan.